

Kacper Cömlek

**Entwicklung und Evaluation eines
KI-gestützten Python-Lernsystems nach 4C/ID**

Bachelorarbeit

im Studiengang Informatik

am Fachbereich Elektrotechnik und Informationstechnik

der FH Aachen – University of Applied Sciences

Erstgutachter: Prof. Dr. rer. pol. Christian Drumm

Zweitgutachter: Christian Kaiser M.A.

Vorgelegt von: Kacper Cömlek

Matrikelnummer: 3544613

E-Mail: kacper74@hotmail.com

Abgabedatum: 11.03.2026



Erklärung

Ich versichere hiermit, dass ich die vorliegende Arbeit selbständig verfasst und keine anderen als die im Literaturverzeichnis angegebenen Quellen benutzt habe.

Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder noch nicht veröffentlichten Quellen entnommen sind, sind als solche kenntlich gemacht.

Die Zeichnungen oder Abbildungen in dieser Arbeit sind von mir selbst erstellt worden oder mit einem entsprechenden Quellennachweis versehen.

Diese Arbeit ist in gleicher oder ähnlicher Form noch bei keiner anderen Prüfungsbehörde eingereicht worden.

Aachen, im Monat Jahr

(vollständige, handschriftliche Unterschrift)



Aufbewahrungspflicht der Bachelorarbeit

Die Bachelorarbeit muss aus rechtlichen Gründen fünf Jahre lang nach dem Abschluss des Studiums aufbewahrt werden. Das Unterzeichnen dieses Schriftstückes berechtigt den Verfasser der Bachelorarbeit, dass in der Fachhochschule aufzubewahrende Exemplar vorzeitig mit der Pflicht zu übernehmen, es fünf Jahre aufzuheben und notfalls der Fachhochschule wieder zur Verfügung zu stellen.

Name: _____

Vorname: _____

Straße: _____

Ort: _____

Hiermit verpflichte ich mich, ein Exemplar meiner Bachelorarbeit mindestens fünf Jahre nach dem Abschluss meines Studiums aufzubewahren.

Aachen, den _____

Unterschrift der/des Studierenden

Abstract*Kann ein KI-gestützter Tutor den Einstieg ins Programmieren erleichtern?*

Die vorliegende Bachelorarbeit untersucht, inwiefern die Integration eines KI-gestützten Chat-Tutors und einer adaptiven Drill-Auswahl die Lernerfahrung beim Erwerb von Python-Grundlagen verbessert. Hierzu wurde ein webbasiertes Lernsystem entwickelt, das Einsteigende schrittweise durch die Implementierung eines vollständigen BMI-Rechner-Programms führt und dabei KI-basierte Assistenz über die OpenAI-API integriert. Als theoretischer Rahmen dient das 4C/ID-Modell (Four-Component Instructional Design) nach van Merriënboer, wobei der Fokus auf den Komponenten Learning Tasks und Part-task Practice liegt.

Eine kontrollierte empirische Studie mit 18 Teilnehmenden im Between-Subjects-Design untersuchte, ob KI-gestützte Komponenten zu günstigeren Lernindikatoren führen. Die Kontrollgruppe (Gruppe A) arbeitete ohne KI-Funktionen, während die Experimentalgruppe (Gruppe B) Zugang zu KI-gestützter Aufgabenauswahl und einem KI-Chat-Tutor erhielt. Die Datenerhebung umfasste System-Logs, NASA-TLX-Fragebögen, Usability-Skalen sowie KI-spezifische Evaluationsitems.

Die Ergebnisse zeigen keine signifikanten Unterschiede in objektiven Leistungsindikatoren wie Fehlerrate oder Abschlussquote. Auf subjektiver Ebene wies Gruppe B konsistent günstigere Werte auf: geringere Frustration im NASA-TLX, höhere hedonische Qualität im UEQ-S sowie einen besseren Net Promoter Score. Eine qualitative Analyse der Chat-Verläufe identifizierte sowohl produktive Nutzungsmuster als auch systemseitige Schwachstellen, aus denen konkrete Verbesserungsmaßnahmen abgeleitet werden.

Danksagung

Die vorliegende Bachelorarbeit bildet den vorerst letzten Meilenstein meines Informatik Studiums und schließt eine prägende Lebensphase ab. Auch wenn dieser Abschluss einen vorläufigen Endpunkt setzt, blicke ich mit Neugier auf zukünftige akademische Herausforderungen, die folgen mögen. Die Erstellung dieser Arbeit wäre ohne die fachliche und persönliche Unterstützung vieler Wegbegleiter nicht möglich gewesen.

Mein besonderer Dank gilt meinem Betreuer, Herrn Prof. Dr. rer. pol. Christian Drumm, für die Ermöglichung dieses spannenden Themas sowie für die wertvollen Impulse und die stets konstruktive fachliche Begleitung während des gesamten Prozesses.

Des Weiteren möchte ich meinen Freunden und Kommilitonen für den steten Austausch und die gegenseitige Motivation danken. Ein herzlicher Dank geht dabei insbesondere an meinen Mitbewohner für sein gewissenhaftes Korrekturlesen und das wertvolle Feedback, welches maßgeblich zur Qualität dieser Arbeit beigetragen hat.

Ein ganz persönlicher Dank gebührt meiner Familie und meiner Partnerin. Danke, dass ihr mich nicht nur in den letzten Monaten, sondern über die vergangenen Jahre hinweg unermüdlich unterstützt habt. Euer Rückhalt und eure Geduld waren die Basis, auf der ich dieses Studium erfolgreich absolvieren konnte.

Erkelenz, den 13.03.2026

Kacper Cömlek

Inhaltsverzeichnis

1	Einleitung	1
1.1	Problemstellung	1
1.2	Zielsetzung und Forschungsfrage	2
1.3	Aufbau der Arbeit	3
2	Theoretische Grundlagen	4
2.1	Programmierausbildung für Einsteigende	4
2.1.1	Herausforderungen beim Programmierenlernen.....	4
2.1.2	Bestehende Ansätze und deren Grenzen	5
2.1.3	Bedeutung von Übung und Feedback	6
2.2	Didaktischer Rahmen: Das 4C/ID-Modell	7
2.2.1	Cognitive Load Theory	7
2.2.2	Selbstreguliertes Lernen	8
2.2.3	Das 4C/ID-Modell im Überblick	8
2.2.4	Die vier Komponenten im Detail	9
2.2.5	Eignung für die Programmierausbildung	11
2.3	KI im Bildungskontext.....	11
2.3.1	Large Language Models	11
2.3.2	KI in der Bildung: Chancen und Risiken.....	12
2.4	Verwandte Arbeiten	13
2.4.1	PyTutor: ChatGPT-basiertes Tutoring für Python	13
2.4.2	Erweiterung des 4C/ID-Modells durch Learning Analytics	14
2.4.3	Positionierung der eigenen Arbeit	14
3	Konzeption des Lernsystems	15
3.1	Anforderungsanalyse	15
3.1.1	Funktionale Anforderungen	15
3.1.2	Nicht-funktionale Anforderungen.....	17
3.2	Didaktisches Konzept.....	18
3.2.1	Lernpfad nach 4C/ID	18

3.2.2	Rolle der KI.....	21
3.3	Systemarchitektur.....	24
3.3.1	Überblick	24
3.3.2	KI-Anbindung und Prompt-Design	25
4	Implementierung	26
4.1	Technologieauswahl	26
4.2	Umsetzung der Kernkomponenten	27
4.2.1	Web-Oberfläche	27
4.2.2	Aufgaben-Engine und KI-Integration.....	28
4.2.3	Nutzerverwaltung und Logging	32
4.3	Herausforderungen und Designentscheidungen	33
4.3.1	Technische Herausforderungen	33
4.3.2	Verworfenen Alternativen	33
4.3.3	Bewusste Abwägungen	34
4.3.4	Limitierungen des Prototyps	34
4.4	Werkzeugunterstützung und KI-Nutzung	35
5	Empirische Studie	36
5.1	Forschungsdesign und Hypothesen	36
5.1.1	Studiendesign.....	36
5.1.2	Unabhängige Variable	36
5.1.3	Abhängige Variablen (Outcomes).....	36
5.1.4	Forschungsfrage	37
5.1.5	Hypothesen	37
5.2	Stichprobe und Rekrutierung.....	37
5.3	Datenschutz und ethische Aspekte	39
5.4	Ablauf der Studie.....	39
5.4.1	Lernaufgaben (Learning Tasks).....	40
5.4.2	Drill Tasks (Part-task Practice)	40
5.5	Erhobene Daten und Messinstrumente	40
5.5.1	Log-Daten	40

5.5.2 Fragebogen	41
5.6 Datenaufbereitung und Auswertungsregeln	42
5.7 Auswertungsstrategie	43
6 Ergebnisse	44
6.1 Datenbasis und Auswertungsregeln	44
6.2 Demografische Merkmale der Stichprobe	44
6.3 Teilnahme, Abschluss und Abbruch	46
6.4 Bearbeitungszeit im BMI-Kurs	46
6.5 Fehlerprofile im BMI-Kurs	48
6.6 Drill-Auswahl und Leistung	49
6.7 KI-Nutzung in Gruppe B	49
6.8 Post-Evaluation: Fragebogenauswertung	51
6.8.1 Kognitive Belastung (NASA-TLX)	52
6.8.2 Gebrauchstauglichkeit (SUS) und Weiterempfehlung (NPS)	52
6.8.3 User Experience Questionnaire Short (UEQ-S)	54
6.8.4 KI-spezifischer Fragebogen (nur Gruppe B)	54
6.9 Korrelationsanalysen	55
6.10 Qualitative Ergebnisse (Freitext)	57
6.11 Zusammenfassung der Ergebnisse	58
7 Diskussion	59
7.1 Interpretation der Ergebnisse	59
7.1.1 Das Effizienz-Paradoxon: Mehrzeit als Zeichen vertiefter Auseinandersetzung	59
7.1.2 Das Belastungs-Dilemma: Höherer Aufwand, geringere Frustration	60
7.1.3 Fehlerkultur und Explorationsbereitschaft	60
7.1.4 Drill-Adaptivität: Fokussierung statt Streuung	61
7.1.5 Nutzungserfahrung und Akzeptanz	62
7.1.6 Wahrnehmung des KI-Tutors in Gruppe B	62
7.1.7 Kursabschluss und Abbruchquote	65
7.1.8 Bewertung der Haupthypothese	65
7.2 Einordnung in den Forschungsstand	66

7.3	Limitationen	68
7.4	Implikationen für Forschung und Praxis	69
8	Fazit und Ausblick	70
8.1	Zusammenfassung	70
8.2	Ausblick.....	71
	Literaturverzeichnis	76
	Abkürzungsverzeichnis	77
	Abbildungsverzeichnis	79
	Tabellenverzeichnis	80
	Anhang	81

1 Einleitung

Dieses Kapitel beschreibt die Ausgangslage und Motivation der Arbeit, formuliert die Forschungsfrage sowie die zentrale Hypothese und erläutert den Aufbau der vorliegenden Ausarbeitung.

1.1 Problemstellung

Python gehört zu den meistgenutzten Programmiersprachen weltweit und ist in Bereichen wie Datenanalyse, Automatisierung und Künstlicher Intelligenz zur Standardsprache geworden [1]. Die Nachfrage nach Personen mit grundlegenden Programmierkenntnissen wächst branchenübergreifend und immer mehr Bildungseinrichtungen reagieren darauf mit eigenen Einführungskursen in Python. Gleichzeitig zeigt die Forschung zur Programmierausbildung konsistent, dass gerade der Einstieg mit hohen Abbruchquoten und ausgeprägten Frustrationserlebnissen verbunden ist [2]. Die binäre Fehlerkultur der Programmierung, in der bereits ein fehlendes Klammerpaar zum vollständigen Programmabbruch führt, stellt viele Einsteigende vor eine Hürde, die sie schon vor dem Verständnis grundlegender Konzepte wie Variablen, Schleifen oder Listenstrukturen kapitulieren lässt [3]. Dijkstra beschrieb dieses Phänomen treffend als *Radical Novelty*: Programmieren unterscheidet sich in seiner Fehlerintoleranz fundamental von anderen Lerngegenständen, da selbst minimale Tippfehler das gesamte Programm zum Absturz bringen.

Ein zentrales Problem bestehender Lernangebote ist das Fehlen unmittelbarer, individualisierter Unterstützung im Fehlerfall. Klassische Lehrformate wie Lehrbücher oder Vorlesungen können auf individuelle Fehler nicht in Echtzeit eingehen. Online-Lernplattformen bieten zwar interaktive Aufgaben, arbeiten aber mit statischem, vordefiniertem Feedback, das nicht auf den spezifischen Code einer lernenden Person eingeht. MOOCs im Hochschulbereich leiden unter demselben Problem in noch stärkerem Ausmaß: Bei Tausenden eingeschriebenen Lernenden ist individuelle Betreuung strukturell nicht möglich. Wenn Einsteigende in einer Aufgabe feststecken und keine Orientierung finden, beschreibt Jenkins [3] das entstehende Erleben als *Learned Helplessness*, also das Gefühl der Hilflosigkeit, das schnell in Frustration und Kursabbruch umschlägt.

Gleichzeitig haben sich die technischen Möglichkeiten zur KI-gestützten Lernunterstützung in den letzten Jahren fundamental verändert. Große Sprachmodelle wie GPT-4 sind in der Lage, Programmiercode zu verstehen, Fehlermeldungen zu interpretieren und didaktisch sinnvolles Feedback auf natürlichsprachliche Anfragen zu geben [4]. Während frühere Tutorsysteme aufwendig von Hand zu kuratieren waren, ermöglicht die OpenAI-API heute die Integration dieser Fähigkeiten in webbasierte Lernanwendungen mit vertretbarem Entwicklungsaufwand. Die entscheidende offene Frage ist dabei nicht mehr, ob solche Systeme technisch realisierbar sind, sondern ob sie tatsächlich besser lernen lassen als Systeme ohne KI-Unterstützung und welche didaktischen Designentscheidungen dafür ausschlaggebend sind.

1.2 Zielsetzung und Forschungsfrage

Diese Arbeit verfolgt zwei eng miteinander verbundene Ziele. Erstens wird ein webbasiertes Lernsystem für Python-Grundlagen entwickelt und implementiert, das auf dem didaktischen Rahmen des *Four-Component Instructional Design* (4C/ID-Modell) nach van Merriënboer basiert. Der Schwerpunkt liegt auf zwei Komponenten des Modells: authentischen *Learning Tasks* in Form eines geführten Programmierkurses, der Lernende schrittweise durch ein vollständiges BMI-Rechner-Programm führt, sowie adaptiver *Part-task Practice* durch KI-gestützte Drill Tasks¹. Als KI-Komponenten kommen ein kontextgebundener Chat-Tutor mit progressivem Hint-System und ein auf der OpenAI-API (Modell `gpt-4o-mini`) basierender Algorithmus zur Drill-Auswahl zum Einsatz.

Zweitens wird dieses System in einer kontrollierten empirischen Studie mit 18 Teilnehmenden erprobt. In einem *Between-Subjects-Design* (d. h. jede Person gehört genau einer Gruppe an und sieht nur eine der beiden Varianten) bearbeiten zwei Gruppen denselben Kurs: Die Kontrollgruppe (Gruppe A) nutzt das System ohne KI-Funktionen und die Experimentalgruppe (Gruppe B) mit KI-Tutor und adaptiver Drillauswahl. Um zu verstehen, ob und wie sich diese beiden Lernumgebungen unterscheiden, werden Verhaltens- und Erlebensdaten aus mehreren Perspektiven erfasst: objektive Leistungsdaten aus den System-Logs, die subjektiv wahrgenommene kognitive Belastung, die Gebrauchstauglichkeit (Usability) sowie KI-spezifische Evaluationsaspekte.

Die dabei leitende Forschungsfrage lautet:

Inwiefern verbessert die Integration einer KI-gestützten Drill-Auswahl und eines KI-Tutors die lernbezogenen Indikatoren und die wahrgenommene Lernunterstützung beim Erlernen von Python-Grundlagen im Vergleich zu einem Basis-System ohne diese Komponenten?

Aus dieser Frage wird eine gerichtete Haupthypothese abgeleitet: Lernende in der Experimentalgruppe (Gruppe B) weisen im Vergleich zur Kontrollgruppe (Gruppe A) günstigere lernbezogene Indikatoren in den Systemlogs sowie eine höhere subjektive Zufriedenheit mit der Lernunterstützung auf. Die Operationalisierung dieser Hypothese und das vollständige Studiendesign werden in Kapitel 5 beschrieben.

¹Unter *Drill Tasks* werden kurze, isolierte Wiederholungsaufgaben verstanden, die gezielt einzelne Teilfähigkeiten trainieren, z. B. das Schreiben einer einfachen `print()`-Anweisung oder das Konkatenieren von Strings.

1.3 Aufbau der Arbeit

Die vorliegende Arbeit gliedert sich in acht Kapitel:

Kapitel 2 legt die theoretischen Grundlagen dar. Es werden zunächst die Herausforderungen der Programmierausbildung für Einsteigende beleuchtet, anschließend das 4C/ID-Modell als didaktischer Rahmen vorgestellt und schließlich der Einsatz von KI im Bildungskontext diskutiert. Das Kapitel schließt mit einer Abgrenzung zu verwandten Arbeiten.

Kapitel 3 beschreibt die Konzeption des Lernsystems. Ausgehend von einer Anforderungsanalyse wird das didaktische Konzept entwickelt und die Systemarchitektur einschließlich der KI-Anbindung dargestellt.

Kapitel 4 dokumentiert die technische Umsetzung des Prototyps. Es werden die Technologieauswahl begründet, die Kernkomponenten beschrieben und auf Herausforderungen während der Entwicklung eingegangen.

Kapitel 5 erläutert das Design der empirischen Studie. Hier werden Forschungsdesign, Hypothesen, Stichprobe, Studienablauf sowie die erhobenen Daten und Messinstrumente beschrieben.

Kapitel 6 präsentiert die Ergebnisse der Studie in deskriptiver und vergleichender Form, ergänzt durch qualitative Erkenntnisse aus den Freitextantworten.

Kapitel 7 interpretiert die Ergebnisse, ordnet sie in den Forschungsstand ein und beleuchtet auf Basis einer qualitativen Sichtung der Chat-Verläufe konkrete Stärken und Schwachstellen des KI-Tutors. Abschließend werden Limitationen benannt und Implikationen für Forschung und Praxis abgeleitet.

Kapitel 8 fasst die zentralen Erkenntnisse zusammen und gibt einen Ausblick auf weiterführende Forschung und Systementwicklung.

Sämtliche anonymisierten Studiendaten, wie Chat-Verläufe, Aufgaben-Log-Daten und Fragebogenantworten, sind zusammen mit dem Quellcode des Systems im öffentlichen GitHub-Projekt archiviert und für weiterführende Auswertungen verfügbar (vgl. Anhang D).

2 Theoretische Grundlagen

Dieses Kapitel bildet die theoretischen Grundlagen für die Konzeption und Evaluation des in dieser Arbeit entwickelten Lernsystems. Zunächst werden die Herausforderungen der Programmierausbildung für Einsteigende dargestellt. Darauf aufbauend wird das 4C/ID-Modell als didaktischer Rahmen eingeführt, einschließlich seiner lerntheoretischen Grundlagen. Anschließend wird der Einsatz von KI im Bildungskontext beleuchtet. Das Kapitel schließt mit einer Einordnung verwandter Arbeiten ab.

2.1 Programmierausbildung für Einsteigende

Das Erlernen von Programmierung gilt als anspruchsvolle kognitive Tätigkeit, die Einsteigende vor spezifische Hürden stellt. Im Folgenden werden diese Herausforderungen dargestellt, bestehende Lösungsansätze eingeordnet und die Rolle von Übung und Feedback für den Kompetenzerwerb beleuchtet.

2.1.1 Herausforderungen beim Programmierenlernen

Das Erlernen von Programmierung stellt Einsteigende vor erhebliche Herausforderungen. Tomasschitz beschreibt den Programmierprozess als „vielschichtig und komplex“ und betont, dass analytische, mathematische und logische Fertigkeiten erforderlich sind, „um Spezifikationen zu lesen, in Algorithmen umzuwandeln und syntaktisch korrekten Code zu schreiben“ [5]. Dabei müssen drei kognitive Anforderungen simultan bewältigt werden. Erstens erfordert das *Problemverständnis* die Fähigkeit, die Anforderungen einer Programmieraufgabe zu erfassen und in eine mentale Repräsentation zu überführen. Zweitens ist *algorithmisches Denken* notwendig, um die Schritte zur Lösung des Problems zu planen und in eine logische Abfolge zu bringen. Drittens muss die *Codeproduktion* erfolgen, also die Umsetzung der geplanten Lösung in syntaktisch korrekten Code. Diese Gleichzeitigkeit führt insbesondere bei Einsteigenden zu kognitiver Überlastung, da keine der drei Anforderungen isoliert bewältigt werden kann.

Programmieren wird als hierarchische Kompetenz charakterisiert: „Programming is not a single skill. It is also not a simple set of skills; the skills form a hierarchy“ [3]. Diese Hierarchie reicht von der Syntax über die Semantik bis hin zur Programmstruktur und zum Programmierstil. Einsteigende müssen diese Ebenen nacheinander aufbauen und verbleiben häufig auf der syntaktischen Ebene, ohne ein tieferes Verständnis zu entwickeln. Als zentrale Überforderungsquelle wird dabei die Gleichzeitigkeit zweier Lernprozesse beschrieben. Einerseits erfordert *Surface Learning* das Auswendiglernen syntaktischer Regeln, andererseits verlangt *Deep Learning* das Durchdringen der zugrundeliegenden Logik [3].

Ein besonderes Merkmal der Programmierung ist ihre Fehlerintoleranz. Dijkstra beschreibt Programmierung als „precision intensive – the smallest possible perturbation of one single bit can render a program totally worthless“ [3].

In kaum einem anderen Fach führt ein minimaler Fehler, wie etwa ein fehlender Doppelpunkt oder eine falsche Einrückung in Python, zum vollständigen Scheitern des Programms. Diese von Dijkstra als *Radical Novelty* bezeichnete Andersartigkeit des Programmierens unterscheidet es von anderen Lerndisziplinen und erklärt die hohe Frustration unter Einsteigenden.

Die empirische Forschung bestätigt diese Schwierigkeiten auf breiter Basis. Qian und Lehman identifizieren in ihrer Literaturübersicht zahlreiche *Misconceptions*¹ bei Programmierneulingen, insbesondere bei Variablenzuweisungen, die häufig als mathematische Gleichungen statt als Speicheroperationen interpretiert werden [6].

Auch das mentale Modell, das als *Notional Machine*² bezeichnet wird, bereitet nachweislich Schwierigkeiten. Lernende müssen dabei ein Verständnis dafür aufbauen, wie der Computer Anweisungen ausführt, um Programmabläufe nachvollziehen zu können [7]. Bennedsen und Caspersen berichten von durchschnittlichen Durchfallquoten von 33 % in einführenden Programmierkursen weltweit [2].

Diese Herausforderungen werden durch das typische Lehrsetting verschärft. Einführungskurse werden als „high-speed train with no brakes“ beschrieben, in denen Lernende, die einmal den Anschluss verlieren, in einen Zustand der *Learned Helplessness* geraten [3]. Das feste Kurstempo lässt wenig Raum für individuelles Lerntempo, welches ein zentrales Argument für adaptive und technologiegestützte Lernsysteme ist.

Die vorliegende Arbeit fokussiert sich auf Python als Programmiersprache. Python eignet sich als Einstiegssprache, da die Syntax sich an natürlicher Sprache orientiert, auf geschweifte Klammern und Semikolons verzichtet und durch dynamische Typisierung die initiale Hürde senkt [8]. Gleichzeitig ist Python die weltweit meistgenutzte Programmiersprache [9] und besitzt hohe praktische Relevanz für Datenanalyse, Automatisierung und für einen Einsatz außerhalb der professionellen Softwareentwicklung.

2.1.2 Bestehende Ansätze und deren Grenzen

Für das Erlernen von Programmierung existiert eine Vielzahl von Ansätzen und Plattformen. Online-Plattformen wie Codecademy oder freeCodeCamp bieten interaktive Kurse mit eingebetteten Programmierübungen und haben die Zugänglichkeit der Programmierausbildung erheblich verbessert. MOOCs (*Massive Open Online Courses*) ermöglichen den Zugang zu Hochschulkursen für ein breites Publikum. Visuelle Programmierumgebungen wie Scratch adressieren den niedrighwelligen Einstieg, beschränken sich jedoch auf blockbasierte Programmierung und bereiten nicht unmittelbar auf textbasierte Sprachen vor [8].

Aus Sicht des in dieser Arbeit gewählten didaktischen Rahmens weisen diese Ansätze jedoch gemeinsame Limitationen auf. Erstens bieten die meisten Plattformen wenig Individualisierung:

¹Mit *Misconceptions* sind typische Fehlvorstellungen gemeint, die Lernende bei grundlegenden Programmierkonzepten entwickeln.

²Die *Notional Machine* beschreibt ein vereinfachtes mentales Modell darüber, wie ein Programm intern ausgeführt wird (z. B. Zustandsänderungen von Variablen Schritt für Schritt).

Alle Lernenden durchlaufen dieselbe Aufgabensequenz unabhängig von ihrem Lernstand oder ihren spezifischen Schwierigkeiten. Zweitens beschränkt sich das Feedback häufig auf eine binäre Rückmeldung (richtig/falsch), ohne detaillierte Erklärungen zu liefern, warum eine Lösung fehlerhaft ist. Drittens ist die Aufgabenreihenfolge bei vielen Plattformen didaktisch nur schwach begründet, da sie meist einer thematischen Abfolge entspricht und nicht auf einem lerntheoretisch fundierten Modell basiert. Diese Limitationen machen eine Lücke deutlich, die die vorliegende Arbeit schließt: die Kombination eines evidenzbasierten didaktischen Modells mit KI-gestützter Individualisierung von Übungsaufgaben und Feedback.

2.1.3 Bedeutung von Übung und Feedback

Nicht die bloße Übungszeit, sondern die Qualität des Übens und die Unmittelbarkeit des Feedbacks entscheiden über den Lernerfolg. Ericsson, Krampe und Tesch-Römer prägen den Begriff der *Deliberate Practice*³: Gezielte, wiederholte Übung mit informativem Feedback ist wirksamer als passives Aufnehmen oder unstrukturiertes Üben [10]. Entscheidend ist dabei nicht die Anzahl der Wiederholungen allein, sondern die Qualität der Übung, diese muss auf spezifische Schwächen ausgerichtet sein und korrigierendes Feedback bereitstellen.

Hattie und Timperley identifizieren in ihrer Meta-Analyse Feedback als einen der einflussreichsten Faktoren für den Lernerfolg [11]. Besonders wirksam ist das sogenannte *Process Feedback*⁴, das nicht nur die Korrektheit bewertet, sondern Hinweise auf den Lösungsweg gibt und zur Selbstregulation anregt. Während unmittelbares Feedback insbesondere auf der Aufgabenebene (*Task Level*) vorteilhaft ist, ermöglicht die zeitnahe Verfügbarkeit in automatisierten Systemen eine kontinuierliche Begleitung des Lernprozesses, die durch Lehrkräfte in dieser Frequenz kaum zu leisten ist [11].

Für die Programmierausbildung bedeutet dies, dass Übungsaufgaben nicht primär als Prüfungsinstrument dienen sollten. Stattdessen erfüllen sie die Funktion eines *formativen Assessments* und unterstützen Lernende dabei, ihren aktuellen Wissensstand einzuschätzen.

Tomaschitz bestätigt dies empirisch: Der Einsatz eines Überprüfungs-tools, das visuelles Feedback über den Code lieferte, steigerte die Leistung der Studierenden und führte zu erhöhtem Lernerfolg [5]. KI-basierte Systeme eröffnen hier neue Möglichkeiten, da sie über vordefinierte Regeln hinaus kontextbezogene, individuelle Rückmeldungen generieren und damit die Funktion eines persönlichen Tutors annähern können.

³*Deliberate Practice* bezeichnet gezieltes, wiederholtes Üben mit klaren Lernzielen und zeitnahe, informativem Feedback.

⁴*Process Feedback* bezieht sich auf den Lösungsprozess (z.B. Strategie und Vorgehen) und nicht nur auf das Ergebnis (richtig/falsch).

2.2 Didaktischer Rahmen: Das 4C/ID-Modell

Aufbauend auf den in Abschnitt 2.1 dargestellten Herausforderungen wird nun der didaktische Rahmen eingeführt, der das Lernsystem dieser Arbeit strukturiert. Zunächst werden die lern-theoretischen Grundlagen gelegt, bevor das 4C/ID-Modell im Detail vorgestellt wird.

2.2.1 Cognitive Load Theory

Die *Cognitive Load Theory* nach Sweller, im Folgenden als CLT bezeichnet, bildet das theo-retische Fundament des 4C/ID-Modells [12]. Die CLT geht von der Annahme aus, dass das menschliche Arbeitsgedächtnis eine begrenzte Kapazität besitzt und gleichzeitig nur eine be-grenzte Anzahl von Informationselementen verarbeiten kann. Wird diese Kapazität überschritten, ist effektives Lernen nicht möglich. Sweller, van Merriënboer und Paas unterscheiden drei Arten kognitiver Belastung [13]:

- **Intrinsic Load** entsteht durch die enthaltene Komplexität des Lernstoffs. Er wird durch die Anzahl der untereinander interagierenden Elemente bestimmt und kann durch geschick-te Anordnung gesteuert werden. Dies lässt sich beispielsweise erreichen, indem einfache Konzepte vor komplexeren eingeführt werden.
- **Extraneous Load** entsteht aus der Gestaltung der Lernumgebung selbst und trägt nicht zum Lernen an sich bei. Er sollte minimiert werden, etwa durch klare Instruktionsdesigns und bedarfsgerechte Hilfestellungen.
- **Germane Load** bezeichnet die produktive kognitive Belastung, die für den Aufbau und die Automatisierung mentaler Schemata genutzt wird. Ein effektives Instruktionsdesign zielt darauf ab, kognitive Ressourcen für diesen Lernprozess optimal zu nutzen.

Für die Programmierausbildung ist die CLT besonders relevant, da Einsteigende typischerweise einem hohen Intrinsic Load ausgesetzt sind, wie Abschnitt 2.1.1 zeigt. Das 4C/ID-Modell adres-siert alle drei Load-Typen systematisch. Aufgabenklassen mit steigender Komplexität steuern den Intrinsic Load, unterstützende Strukturen wie *Scaffolding*⁵ und klare Instruktionen reduzie-ren den Extraneous Load und gezielte Übungen fördern die Schemaautomatisierung und damit den Germane Load.

⁵*Scaffolding* bezeichnet unterstützende Hilfen, die Lernenden anfangs Orientierung geben und mit zunehmender Kompetenz schrittweise reduziert werden.

2.2.2 Selbstreguliertes Lernen

Da das in dieser Arbeit entwickelte Lernsystem vollständig ohne Lehrkraft genutzt wird, ist das Konzept des *selbstregulierten Lernens* nach Zimmerman von zentraler Bedeutung [14–16].

Selbstreguliertes Lernen, im Folgenden als SRL bezeichnet, beschreibt den Prozess, in dem Lernende ihre eigenen Lernaktivitäten planen, überwachen und bewerten.

In digitalen, selbstgesteuerten Lernumgebungen müssen Lernende diese metakognitiven Fähigkeiten eigenständig einsetzen, was speziell bei Einsteigenden zu Überforderung führen kann [16]. Ohne externe Regulation besteht das Risiko, dass Lernende schwierige Inhalte überspringen oder ihren Lernstand falsch einschätzen. Dazu zählen auch systematische Fehlschlüsse und Illusionen beim selbstregulierten Lernen [17]. In KI-gestützten Umgebungen besteht zudem die Gefahr, dass Lernende den KI-Chatbot vorrangig als unmittelbares Lösungswerkzeug nutzen, anstatt ihn als lernförderliche Unterstützung einzubinden [18]. Digitale Lernumgebungen müssen daher Mechanismen der externen Regulation bereitstellen. Dazu gehören etwa verpflichtende Übungsphasen, gestufte Hilfestellungen oder Fortschrittsanzeigen, um SRL auch bei Einsteigenden zu ermöglichen [19]. Wie das vorliegende System diesen Anforderungen konkret begegnet, wird in Kapitel 3 beschrieben.

2.2.3 Das 4C/ID-Modell im Überblick

Das *Four-Component Instructional Design*, kurz 4C/ID, wurde von van Merriënboer entwickelt und bietet einen systematischen Ansatz zur Gestaltung von Lernumgebungen für komplexe kognitive Fähigkeiten [20, 21]. Das Modell basiert explizit auf der CLT und strukturiert den Lernprozess in vier miteinander verknüpfte Komponenten:

1. **Learning Tasks:** Authentische, ganzheitliche Aufgaben in realitätsnahen Kontexten. Sie sind in Aufgabenklassen mit steigender Komplexität organisiert.
2. **Supportive Information:** Konzeptuelles Wissen und theoretische Hintergründe, die den Aufbau mentaler Modelle fördern und vor oder während der Aufgabenbearbeitung bereitgestellt werden.
3. **Just-in-time Information:** Prozedurale Anleitungen und Handlungsanweisungen, die genau dann bereitgestellt werden, wenn sie benötigt werden.
4. **Part-task Practice:** Isoliertes Üben von Teilfertigkeiten zur Automatisierung wiederkehrender Prozesse.

Ausgehend von den dargestellten Herausforderungen beim Programmierenlernen und der Cognitive Load Theory wird das 4C/ID-Modell in dieser Arbeit als didaktischer Rahmen gewählt [20, 21].

Erstens enthält das Modell mit der Part-task Practice einen expliziten Mechanismus für isoliertes Üben, der sich gut für die technologische Umsetzung und Automatisierung eignet [20]. Zweitens strukturiert 4C/ID den Lernstoff in Aufgabenklassen mit steigender Komplexität und ermöglicht dadurch eine nachvollziehbare Progression [20, 21]. Drittens ist das Modell evidenzbasiert und in der Hochschuldidaktik etabliert [22]. Viertens adressiert 4C/ID durch seine Fundierung in der CLT das Risiko kognitiver Überlastung, das insbesondere bei ProgrammierEinstiegenden relevant ist [13, 20]. Die konkrete Umsetzung dieser Prinzipien im vorliegenden System wird in Kapitel 3 beschrieben.

2.2.4 Die vier Komponenten im Detail

Das Modell gliedert sich in vier aufeinander abgestimmte Komponenten, die im Folgenden einzeln dargestellt werden.

Learning Tasks: Lernaufgaben Lernaufgaben bilden das Kernstück des 4C/ID-Modells. Sie basieren auf authentischen, ganzheitlichen Aufgaben, die in der Literatur als *Whole Tasks* beschrieben werden und Wissen, Fertigkeiten sowie Haltungen integrieren, die für die Bewältigung von Aufgaben im zukünftigen Beruf oder Alltag benötigt werden [23]. Im Gegensatz zu isolierten Teilübungen konfrontieren sie die Lernenden mit der gesamten Komplexität einer Problemstellung.

Lernaufgaben werden in *Aufgabenklassen* organisiert, die eine steigende Komplexität abbilden. Innerhalb einer Klasse erhalten Lernende zunächst Unterstützung, die als *Scaffolding* beschrieben wird. Mit zunehmender Kompetenz wird diese Unterstützung schrittweise zurückgenommen, was als *Fading*⁶ bezeichnet wird. Niegemann und Weinberger betonen, dass Lernen durch Entdecken nur dann effektiv ist, wenn die Lernaufgaben untereinander „eine gewisse Variation aufweisen“ [23].

Supportive Information: Unterstützende Informationen Unterstützende Informationen helfen Lernenden bei den nicht-routinisierten Aspekten der Lernaufgaben, die Problemlösen und Entscheidungen erfordern. Van Merriënboer und Kirschner beschreiben sie als das konzeptuelle Wissen, das den Aufbau *mentaler Modelle* fördert [20]. Typische Formen sind theoretische Erklärungen, Analogien sowie *Worked Examples*, also ausgearbeitete Lösungsbeispiele.

Die unterstützenden Informationen sind nicht an einzelne Aufgaben gebunden, sondern an die jeweilige Aufgabenklasse: Sie werden idealerweise vor Beginn einer neuen Klasse bereitgestellt und können während der Bearbeitung abgerufen werden [23]. Bei steigender Kompetenz werden sie schrittweise reduziert.

⁶*Fading* beschreibt den schrittweisen Abbau anfänglicher Unterstützung, damit Lernende Aufgaben zunehmend selbstständig lösen.

Just-in-time Information: Prozedurale Information Im Gegensatz zur unterstützenden Information wird die prozedurale Information genau dann bereitgestellt, wenn sie benötigt wird. Sie umfasst konkrete Handlungsanweisungen, die als *How-to-Instruktionen* bezeichnet werden und für die Ausführung routinisierbarer Aspekte der Aufgabe erforderlich sind [23].

Niegemann und Weinberger heben einen Vorteil menschlicher Lehrkräfte gegenüber statischen Anleitungen hervor: „Der Vorteil eines Lehrers gegenüber einer Benutzeranleitung besteht darin, dass ein Lehrer gewissermaßen über die Schulter schauen kann und Anweisungen und korrekatives Feedback genau dann geben kann, wenn es benötigt wird“ [23]. Für digitale Lernumgebungen liegt nahe, dass ein KI-Chatbot eine solche Form der Unterstützung zumindest teilweise leisten kann. Dazu kann er die aktuellen Eingaben der Lernenden als Kontext nutzen und genau dann Hinweise oder korrekatives Feedback geben, wenn es benötigt wird.

Part-task Practice: Übung von Teilfertigkeiten Die Part-task Practice dient der Automatisierung wiederkehrender Teilfertigkeiten, die als *Recurrent Skills* bezeichnet werden, durch gezieltes und wiederholtes Üben. Van Merriënboer und Kirschner betonen, dass diese Übungen erst dann sinnvoll sind, wenn die entsprechende Teilfertigkeit bereits im Kontext einer Lernaufgabe eingeführt wurde [20]. Durch die Automatisierung von Routinefertigkeiten wird kognitive Kapazität freigesetzt, die für die komplexeren Anforderungen der Lernaufgaben genutzt werden kann.

Die Übungen lassen sich nach der revidierten Bloom'schen Taxonomie [24] in vier kognitive Anforderungsstufen differenzieren: von *Erinnern* (Wissensabfragen) über *Verstehen* (Ausgabe vorhersagen) und *Anwenden* (Lücken füllen, Code ergänzen) bis hin zum *Analysieren* (Fehler lokalisieren). Diese Abstufung stellt sicher, dass Übungsaufgaben schrittweise höhere kognitive Prozesse einfordern. Idealerweise wird die kognitive Anforderung dabei an den individuellen Lernstand angepasst.

Die Part-task Practice adressiert primär den *Near Transfer*, also den Transfer auf sehr ähnliche Aufgaben. Durch wiederholtes Üben isolierter Teilfertigkeiten werden syntaktische und prozedurale Muster automatisiert [20]. Der *Far Transfer*, also die Übertragung gelernter Konzepte auf neuartige Problemkontexte, wird hingegen durch die ganzheitlichen Lernaufgaben adressiert. Das Zusammenspiel beider Komponenten ist zentral. Im vorliegenden System wird die Part-task Practice als *Drill Tasks* bezeichnet. Drill Tasks richten sich gezielt an rekurrente Teilfertigkeiten, also wiederkehrende syntaktische und prozedurale Grundmuster, die durch wiederholtes Üben automatisiert werden sollen. Auf diese Weise entlasten sie das Arbeitsgedächtnis und schaffen freie kognitive Kapazität für die höherstufigen Anforderungen der Lernaufgaben. Wie Drill Tasks in der Lernplattform konkret gestaltet und ausgewählt werden, wird in Kapitel 3 beschrieben.

2.2.5 Eignung für die Programmierausbildung

Das 4C/ID-Modell eignet sich in besonderer Weise für die Programmierausbildung, da es die in Abschnitt 2.1.1 beschriebenen Herausforderungen systematisch adressiert. Die Sequenzierung in Aufgabenklassen begegnet der kognitiven Überlastung, authentische Lernaufgaben fördern den Transfer in reale Anwendungskontexte und die Part-task Practice ermöglicht die gezielte Automatisierung syntaktischer Grundfertigkeiten. Besonderes Potenzial entsteht durch die Kombination des Modells mit KI-Technologien.

Das 4C/ID-Modell fordert eine individualisierte Part-task Practice, abgestimmt auf den Lernstand des Einzelnen, was in klassischen Settings einen erheblichen Aufwand für Lehrkräfte bedeutet [20]. KI-Technologien bieten hier Ansatzpunkte: Sie können die Individualisierung der Übungsauswahl automatisieren und kontextbezogene Hilfestellungen im Sinne der Just-in-time Information bereitstellen. Während das 4C/ID-Modell als didaktischer Rahmen für komplexes Lernen etabliert ist, fehlt es an Studien, die untersuchen, wie KI-Technologien die Umsetzung der Modellkomponenten unterstützen können. Die vorliegende Arbeit leistet hierzu einen Beitrag.

2.3 KI im Bildungskontext

Im Folgenden werden Large Language Models als technologische Basis vorgestellt sowie Chancen und Risiken ihres Einsatzes im Bildungskontext diskutiert.

2.3.1 Large Language Models

Large Language Models (LLMs) sind neuronale Sprachmodelle. Sie basieren auf der Transformer-Architektur und werden auf umfangreichen Textsammlungen trainiert. Durch dieses Training entwickeln sie die Fähigkeit, natürlichsprachliche Eingaben zu verarbeiten und kontextbezogene Antworten zu generieren. Modelle der GPT-Reihe (*Generative Pre-trained Transformer*) von OpenAI gehören zu den bekanntesten Vertretern und werden über eine API für die Integration in Anwendungen bereitgestellt [25].

Für die Einbindung in Lernsysteme sind mehrere Eigenschaften von LLMs relevant. Erstens können sie natürlichsprachliche Erklärungen generieren, die auf die spezifische Frage eines Lernenden zugeschnitten sind. Zweitens erlaubt *Prompt Engineering*, also die gezielte Gestaltung von Systemaufforderungen, eine Steuerung des Antwortverhaltens, etwa die Einschränkung auf eine sokratische Gesprächsführung statt direkter Lösungsnennung. Dabei werden Lernende durch gezielte Rückfragen dazu angeleitet, den Lösungsweg selbst zu entwickeln. Drittens ermöglichen *Structured Outputs* die Erzeugung typischerer Ausgaben im JSON-Format, die eine programmatische Weiterverarbeitung erlauben [26].

Gleichzeitig weisen LLMs systematische Limitationen auf. Zentral für den Bildungskontext ist das Phänomen der *Halluzination*: LLMs können syntaktisch korrekte, aber inhaltlich falsche Antworten generieren, die für Einsteigende schwer als solche zu erkennen sind [4].

Diese Limitation erfordert bei der Integration in Lernsysteme besondere Vorkehrungen.

2.3.2 KI in der Bildung: Chancen und Risiken

Die Nutzung KI-gestützter Systeme in der Bildung hat eine längere Tradition als die aktuelle LLM-Debatte vermuten lässt. *Intelligente Tutorsysteme* (ITS) werden seit den 1970er Jahren erforscht und setzen regelbasierte oder statistische Modelle ein, um Lernende individuell zu unterstützen. Meta-Analysen belegen, dass ITS Lerneffekte erzielen können, die mit individueller menschlicher Betreuung vergleichbar sind und besonders in Kombination mit klassischem Unterricht wirksam sind [27, 28]. Eine aktuelle Forschungssynthese des BMBF⁷ kommt zu dem Ergebnis, dass der Einsatz von KI-Technologien insgesamt einen positiven Effekt auf Lernleistungen hat und dass die Effektivität „wesentlich von ihrem didaktischen Design und Einsatz“ abhängt. Die Lernwirksamkeit lässt sich daher nicht über die Technologie allein definieren, sondern nur über die Qualität ihrer didaktischen Integration [28]. Chatbot-basierte Systeme erzielen dabei besonders gute Effekte, wenn sie spezifische Unterstützung bieten, zur Reflexion anleiten oder elaboriertes Feedback geben [28].

Mit der Verfügbarkeit von LLMs eröffnen sich Möglichkeiten, die über traditionelle ITS hinausgehen. Zu den zentralen Chancen zählen:

- **Personalisierung:** LLM-basierte Tutoren können Erklärungen individuell an den Wissensstand und die Schwierigkeiten eines Lernenden anpassen [4].
- **Skalierbarkeit:** Im Gegensatz zu menschlichen Tutoren kann ein KI-System viele Lernende gleichzeitig betreuen.
- **Sofortiges Feedback:** Gemäß der Feedback-Forschung begünstigen Rückmeldungen ohne Wartezeit den Lernerfolg [11].
- **Niedrige Hemmschwelle:** Lernende stellen einem KI-System eher Fragen, die sie einer Lehrkraft nicht stellen würden [4].
- **Adaptive Aufgabenauswahl:** KI kann den Lernstand analysieren und die Auswahl von Übungsaufgaben individuell anpassen.

⁷BMBF steht für „Bundesministerium für Bildung und Forschung“.

Diesen Chancen stehen Risiken gegenüber, die im Systemdesign berücksichtigt werden müssen:

- **Halluzinationen:** LLMs können fehlerhafte Informationen generieren. In der Programmierausbildung äußert sich dies als falscher Code oder fehlerhafte Erklärungen [29].
- **Lösungs-Leak:** Ohne geeignete Maßnahmen kann ein KI-Chatbot die Lösung direkt preisgeben, wodurch der Lerneffekt entfällt. Becker et al. diskutieren diese Problematik im Kontext der Frage, wie KI-Code-Generierung die Programmierausbildung verändert [29].
- **Qualitätssicherung:** KI-generierte Aufgaben können fehlerhaft oder in ihrer Schwierigkeit unangemessen sein und bedürfen einer Validierung.

Sarsa et al. demonstrieren die grundsätzliche Machbarkeit KI-gestützter Aufgabengenerierung für die Programmierausbildung und zeigen, dass LLMs Übungen erzeugen können, die von Lernenden als angemessen und lehrreich bewertet werden [30]. MacNeil et al. untersuchen LLM-generierte Code-Erklärungen in einem interaktiven Lehrbuch und berichten von positiver Aufnahme durch Studierende [31].

Diese Studien belegen das Potenzial von LLMs als didaktisches Werkzeug, betonen aber zugleich die Notwendigkeit einer bewussten Integration in einen didaktischen Rahmen.

Diese Risiken erfordern beim Entwurf eines KI-gestützten Lernsystems bewusste Gegenmaßnahmen, etwa die Einschränkung der generativen Freiheit des Modells, gestufte Hilfestellungen und eine Qualitätssicherung der Aufgabeninhalte. Die konkreten Designentscheidungen, mit denen das vorliegende System diesen Risiken begegnet, werden in Abschnitt 3.2.2 dargestellt.

2.4 Verwandte Arbeiten

Aufbauend auf den dargestellten Grundlagen werden im Folgenden zwei verwandte Arbeiten vorgestellt, die zentrale Schnittmengen mit dem in dieser Arbeit entwickelten System aufweisen: Ein LLM-basiertes Tutorsystem für Python-Einsteiger sowie ein Framework zur Erweiterung des 4C/ID-Modells durch Learning Analytics⁸.

2.4.1 PyTutor: ChatGPT-basiertes Tutoring für Python

Yang, Lin, Lin und Ogata entwickelten mit *PyTutor* ein auf ChatGPT basierendes Intelligent Tutoring System für die Python-Ausbildung [32]. Das System bietet adaptives Feedback und ein vierstufiges Hint-System, das von Pseudocode über Lückentexte bis hin zu vollständigen Lösungen eskaliert. In einer 11-wöchigen Studie mit 71 Studierenden (35 mit PyTutor, 36 ohne) zeigten insbesondere leistungsschwächere Lernende höheres Engagement und bessere Abschlussquoten. Ein methodisches Merkmal ist die semi-synchrone Validierung: KI-generierte Erklärungen wurden vor der Auslieferung durch Lehrende geprüft.

⁸*Learning Analytics* bezeichnet die Erfassung und Auswertung von Lerndaten, um Lernprozesse zu verstehen und Lernangebote adaptiv zu steuern.

PyTutor ist die nächstverwandte Arbeit zum vorliegenden System. Beide Systeme setzen auf einen LLM-basierten Tutor mit progressivem Hint-System für Python-Einsteigende und evaluieren die Wirksamkeit in einem Zwei-Gruppen-Design. Das vorliegende System unterscheidet sich in drei Aspekten: Erstens ist es in das 4C/ID-Modell eingebettet, das die strikte Trennung von ganzheitlichen Lernaufgaben und isolierter Part-task Practice strukturiert. PyTutor verwendet hingegen kein explizites didaktisches Rahmenmodell. Zweitens bietet das vorliegende System eine KI-gestützte Aufgabenauswahl aus einem statischen Pool, die auf individuellen Leistungsdaten basiert. PyTutor verfügt über keine vergleichbare Komponente. Drittens unterscheiden sich beide Systeme in ihrer Strategie zur Qualitätssicherung. PyTutor setzt auf einen *Human-in-the-Loop*-Ansatz, bei dem KI-generierte Inhalte vor der Auslieferung durch Lehrende geprüft werden. Das vorliegende System verfolgt stattdessen eine constraints-basierte Architektur: Die KI ist auf einen vorab geprüften Aufgabenpool und restriktive System-Prompts beschränkt, sodass es vollständig autonom ohne Lehrkraft-Validierung operieren kann. Dieser Ansatz erhöht die Skalierbarkeit, verlagert die Qualitätssicherung jedoch in die Entwicklungsphase.

2.4.2 Erweiterung des 4C/ID-Modells durch Learning Analytics

Mun und Jo schlagen ein Framework vor, das das 4C/ID-Modell durch Methoden des Learning Analytics erweitert [33]. Durch die Erfassung und Analyse von Lernverlaufsdaten auf digitalen Plattformen demonstrieren sie, wie die Bereitstellung unterstützender Informationen und die Zuweisung von Teilaufgaben automatisiert an den kognitiven Bedarf der Lernenden angepasst werden kann. Das Framework adressiert damit eine historische Limitation des 4C/ID-Modells: die aufwendige manuelle Individualisierung von Lernpfaden.

Für die vorliegende Arbeit ist diese Publikation methodisch relevant, da sie die Verbindung zwischen quantitativen Leistungsdaten und der 4C/ID-Architektur wissenschaftlich fundiert. Das vorliegende System verfolgt ein ähnliches Ziel, individuelle Lernverläufe für die Steuerung der Part-task Practice zu nutzen. Die Abgrenzung liegt im technologischen Instrumentarium: Während Mun und Jo auf statistische Learning-Analytics-Dashboards setzen, baut das vorliegende System auf generative KI und ergänzt die datengestützte Aufgabenauswahl um einen LLM-basierten KI-Tutor für die Just-in-time Information.

2.4.3 Positionierung der eigenen Arbeit

Die Analyse der verwandten Arbeiten zeigt, dass LLM-basierte Tutorsysteme für die Programmierausbildung zunehmend erforscht werden und dass das 4C/ID-Modell durch datengetriebene Ansätze skalierbar gemacht werden kann. Es fehlt jedoch ein evaluiertes System, das beide Stränge verbindet: die didaktische Struktur des 4C/ID-Modells mit KI-gestützter Individualisierung sowohl bei der Aufgabenauswahl als auch bei der Lernunterstützung. Die vorliegende Arbeit adressiert diese Lücke. Die konkreten Designentscheidungen, mit denen das System diese Verbindung umsetzt, werden in Kapitel 3 beschrieben.

3 Konzeption des Lernsystems

Dieses Kapitel führt den in Kapitel 2 dargestellten theoretischen Rahmen weiter und leitet daraus konkrete Entwurfsentscheidungen ab. Ausgehend von einer Anforderungsanalyse wird das didaktische Konzept auf Grundlage des in Abschnitt 2.2 eingeführten 4C/ID-Modells entwickelt und die Systemarchitektur auf konzeptioneller Ebene dargestellt. Die technische Umsetzung der hier beschriebenen Konzepte folgt in Kapitel 4.

3.1 Anforderungsanalyse

Die Anforderungsanalyse definiert die funktionalen und nicht-funktionalen Anforderungen an das Lernsystem. Sie bildet die Grundlage für die nachfolgenden Entwurfsentscheidungen.

3.1.1 Funktionale Anforderungen

Die folgenden funktionalen Anforderungen beschreiben, welche Kernfunktionen das Lernsystem bereitstellen muss.

Kursstruktur und Lernpfad

Das System umfasst zwei projektbasierte Kurse mit steigender Komplexität: einen Pflichtkurs (BMI-Rechner, 12 Aufgaben) und einen optionalen Aufbaukurs (Zinsrechner, 8 Aufgaben). Der Pflichtkurs muss vollständig abgeschlossen werden, bevor der Aufbaukurs freigeschaltet wird. Nach Abschluss des Pflichtkurses können Nutzende wahlweise den Aufbaukurs bearbeiten oder direkt zum Evaluationsbogen wechseln.

Innerhalb eines Kurses ist die Progression streng linear: Jede Aufgabe baut auf der vorherigen auf, wobei der Startercode der aktuellen Aufgabe auf der erwarteten Lösung der vorherigen basiert. Ein Vorwärts-Springen zu ungelösten Aufgaben ist nicht möglich, Rückwärts-Navigation zu bereits gelösten Aufgaben hingegen erlaubt. Diese lineare Struktur stellt eine strukturierte Lernprogression sicher und gewährleistet vergleichbare Bedingungen für alle Nutzenden im Rahmen der empirischen Studie.

Aufgabenpräsentation

Jede Aufgabe muss gleichzeitig einen Code-Editor mit vorgegebenem Startercode, die Aufgabenstellung mit Code-Beispielen und Hinweisen sowie eine Konsolenausgabe für das Ergebnis der Code-Ausführung bereitstellen. Die Darstellung soll so gestaltet sein, dass Lernende ohne Wechsel zwischen Ansichten arbeiten können.

Code-Ausführung und Validierung

Nutzende können Python-Code direkt im Browser schreiben und ausführen, ohne zusätzliche Software installieren zu müssen. Die Ausführung muss in einer sicheren Umgebung stattfinden, die den eingegebenen Code isoliert und vor Endlosschleifen schützt.

Die Lösungsvalidierung muss drei Aspekte abdecken: Erkennung von Python-Fehlern, Prüfung auf tatsächliche Verwendung der gelernten Konzepte (um Hardcoding zu verhindern) und Abgleich der Ausgabe mit einer vordefinierten Musterlösung. Die technische Umsetzung dieser Anforderungen wird in Kapitel 4 beschrieben.

Part-task Practice durch Drill Tasks

Nach jeder erfolgreich gelösten Aufgabe folgt obligatorisch eine Drill Task als Part-task Practice im Sinne des 4C/ID-Modells [20]. Jede Drill Task besteht aus zwei Komponenten: einer Multiple-Choice-Frage und einer Mini-Code-Aufgabe. Beide beziehen sich auf Konzepte, die in der zuletzt gelösten Aufgabe behandelt wurden. Die Drill Tasks sind nicht überspringbar; erst nach Abschluss beider Komponenten erfolgt die Weiterleitung zur nächsten Aufgabe. Nutzende haben bei beiden Teilen unbegrenzt viele Versuche.

Die Drill Tasks stammen aus einem statischen, vom Autor erstellten und geprüften Aufgabenpool. Die Auswahl der jeweils angezeigten Aufgabe erfolgt je nach Versuchsgruppe unterschiedlich. Die didaktische Begründung und die Auswahlverfahren werden in Abschnitt 3.2.2 beschrieben.

KI-Funktionen

Das System integriert zwei KI-Funktionen, die ausschließlich für Gruppe B, also die Experimentalgruppe mit KI-Unterstützung, verfügbar sind (Gruppe A dient als Kontrollgruppe ohne KI-Funktionen):

1. **KI-gestützte Drill-Task-Auswahl:** Ein Sprachmodell wählt aus dem statischen Aufgabenpool die didaktisch passendste Kombination aus Multiple-Choice-Frage und Code-Aufgabe, basierend auf der individuellen Performance der Nutzenden.
2. **KI-Tutor mit progressivem Hint-System¹:** Ein kontextsensitiver KI-Tutor unterstützt Lernende während der Aufgabenbearbeitung. Lernende können in natürlicher Sprache auf ihrem eigenen Niveau mit ihm kommunizieren. Er ist jederzeit abrufbar und ermöglicht es, über Aufgaben, Fehler und Missverständnisse zu sprechen. Die Hilfestellung eskaliert über fünf Stufen von reinen Verständnisfragen bis zur vollständigen Lösung.

¹Ein progressives Hint-System stellt Hilfestellungen gestuft bereit: Mit jeder weiteren Hilfeanfrage werden die Hinweise konkreter, bis hin zur vollständigen Lösung.

Nutzerverwaltung und Gruppenzuordnung

Die Anmeldung muss anonym sein, ohne Erhebung personenbezogener Identifikationsmerkmale wie Vorname, Nachname oder Geburtsdatum. Die Zuordnung zu einer Versuchsgruppe erfolgt automatisch und balanciert, wobei die Zuweisung permanent ist, um Störvariablen durch Gruppenwechsel zu vermeiden. Nutzende werden transparent über ihre Gruppenzugehörigkeit informiert.

Onboarding

Ein Erklärvideo, das die Funktionen des Systems erklärt und den Hintergrund dieser Studie erläutert, soll den Einstieg vor dem Login unterstützen. Neue Nutzende müssen durch ein mehrstufiges Onboarding in die Bedienung der Plattform eingeführt werden. Für Gruppe B umfasst dies zusätzlich eine Einführung in den KI-Tutor sowie die Erfassung von Programmiervorkenntnissen, die als Kontext für die KI-Interaktionen dienen.

Datenerhebung

Das System muss alle relevanten Aktivitäten protokollieren, um die empirische Auswertung zu ermöglichen. Dazu gehören Code-Ausführungen, Aufgabenfortschritt, Chat-Verläufe (Gruppe B), Drill-Ergebnisse und Evaluationsbogen-Antworten. Zusätzlich wird ein persistentes Lernprofil benötigt, das den aktuellen Leistungsstand und häufige Fehlertypen erfasst und als Grundlage für die KI-gestützte Aufgabenauswahl dient.

3.1.2 Nicht-funktionale Anforderungen

Die nicht-funktionalen Anforderungen beschreiben Qualitätseigenschaften des Systems, die unabhängig von konkreten Funktionen gelten und den Rahmen für sämtliche technischen und gestalterischen Entscheidungen bilden.

Bedienfreundlichkeit

Da die Zielgruppe Personen ohne jede Programmiererfahrung umfasst, ist eine niedrige Einstiegshürde essentiell. Die Bedienoberfläche setzt auf ein klares, übersichtliches Design mit ausreichenden Schriftgrößen und gut lesbaren Code-Beispielen. Farbkodierung dient als unmittelbares Feedback: Richtige Lösungen lösen eine positive Rückmeldung aus, fehlerhafte Lösungen zeigen kontextbezogene Hinweise.

Datenschutz

Die in der Nutzerverwaltung vorgesehene anonyme Anmeldung minimiert die Verarbeitung personenbezogener Daten. Eine Datenschutzseite informiert transparent über Art und Zweck der

erhobenen Daten. Die Datenerhebung erfolgt konform zur Datenschutz-Grundverordnung [34] auf Basis der Einwilligung bei Anmeldung (Art. 6 Abs. 1 lit. a DSGVO). Sämtliche Nutzerdaten werden nach Abschluss des Prüfungsverfahrens gelöscht oder anonymisiert.

Sprache

Die Plattform muss primär auf Deutsch ausgelegt sein, da die Zielgruppe deutschsprachige Studierende umfasst.

Aufbauend auf diesen Anforderungen beschreibt der folgende Abschnitt, wie die funktionalen Kernelemente, insbesondere Lernpfad, Drill Tasks und KI-Funktionen, im Rahmen des in Abschnitt 2.2 eingeführten 4C/ID-Modells didaktisch begründet und gestaltet werden.

3.2 Didaktisches Konzept

Das didaktische Konzept basiert auf dem in Abschnitt 2.2 eingeführten 4C/ID-Modell. Die dort beschriebenen vier Komponenten sind Learning Tasks, Supportive Information, Just-in-time Information und Part-task Practice. Diese werden im Folgenden auf das Lernsystem abgebildet. Dabei wird aufgezeigt, wie die lerntheoretischen Prinzipien der Cognitive Load Theory (vgl. Abschnitt 2.2.1) in konkreten Designentscheidungen umgesetzt werden.

3.2.1 Lernpfad nach 4C/ID

Der Lernpfad setzt die Sequenzierung in Aufgabenklassen des 4C/ID-Modells um und bildet das strukturelle Grundgerüst der Lernplattform.

Aufgabenklassen und Learning Tasks

Das 4C/ID-Modell organisiert Lernaufgaben in Aufgabenklassen mit steigender Komplexität (vgl. Abschnitt 2.2.4). Im vorliegenden System bilden die beiden Kurse jeweils eine Aufgabenklasse:

- **Aufgabenklasse 1 (BMI-Rechner):** Grundlagen der Python-Programmierung: Textausgabe, Variablen, Datentypen, arithmetische Operationen, Typkonvertierung und bedingte Anweisungen.
- **Aufgabenklasse 2 (Zinsrechner):** Aufbauende Konzepte: Schleifen mit Laufvariablen, das Akkumulator-Pattern, Listen und deren Kombination mit Schleifen und formatierte Zeichenketten. Bedingte Anweisungen aus Aufgabenklasse 1 werden in neuem Kontext wiederholt.

Beide Kurse sind als komplexe, authentische Learning Tasks im Sinne des 4C/ID-Modells konzipiert (vgl. Abschnitt 2.2.4). Der BMI-Rechner spiegelt eine reale Anforderung wider: die Berechnung des Body-Mass-Index. Sie erfordert die Integration mehrerer Konzepte, nämlich Eingabe,

Variablen, Berechnungen und bedingte Logik, in einem sinnvollen Kontext mit Bezug zur Lebenswelt der Lernenden. Der Zinsrechner erweitert diesen Ansatz um iterative Berechnungen und Datensammlung in einem finanzmathematischen Kontext.

Der qualitative Komplexitätssprung zwischen den Kursen liegt darin, dass im BMI-Kurs eine einmalige Berechnung ausreicht, während der Zinsrechner dieselbe Berechnung wiederholt erfordert. Dadurch wird der Einsatz von Schleifen motiviert. Zudem müssen im Zinsrechner mehrere Zwischenergebnisse gespeichert werden, was den Einsatz von Listen begründet.

Komplexitätssteigerung innerhalb der Kurse

Die Komplexitätssteigerung innerhalb des BMI-Kurses verläuft in vier Phasen. In der ersten Phase (Aufgaben 1–2) erzeugen Lernende reine Textausgaben ohne Zustandsverwaltung. In der zweiten Phase (Aufgaben 3–7) werden Variablen, Datentypen, Typkonvertierung und arithmetische Operationen eingeführt. Die dritte Phase (Aufgaben 8–11) umfasst bedingte Anweisungen und das Refactoring bestehenden Codes. Die vierte Phase (Aufgabe 12) konsolidiert das Gesamtprogramm durch das Testen verschiedener Eingabewerte.

Der Zinsrechner steigert die Komplexität in drei Phasen: formatierte Ausgabe und Berechnung (Aufgaben 1–2), Iteration und Datensammlung mit Schleifen und Listen (Aufgaben 3–6) sowie Wiederholung und Konsolidierung (Aufgaben 7–8).

Die Reihenfolge der Python-Konzepte folgt einer Abhängigkeitskette, in der jedes neue Konzept das vorherige voraussetzt und durch die Aufgabenstellung motiviert wird. Textausgabe steht am Anfang, da sie die Feedback-Schleife ermöglicht. Ohne Ausgabemöglichkeit kann der Lernende kein Ergebnis überprüfen. Variablen benötigen die Textausgabe zur Sichtbarmachung. Arithmetik erzeugt Zahlenwerte, über die erst dann per bedingter Anweisung entschieden werden kann. Schleifen werden erst im Zinsrechner eingeführt, wo die Aufgabenstellung sie natürlich erfordert und Listen ergeben sich als Antwort auf das Problem, mehrere Werte in einer Schleife zu speichern.

Scaffolding

Das Scaffolding im Lernsystem basiert auf drei statischen und einem dynamischen Mechanismus.

Der zentrale Mechanismus ist der *additive Startercode*: Jede Aufgabe enthält den vollständigen, funktionierenden Code aller vorherigen Aufgaben als Vorgabe. Lernende schreiben nur den neuen Teil. Der vorgegebene Code wächst von wenigen Zeilen in der ersten Aufgabe auf über zwanzig Zeilen in späteren Aufgaben. Dadurch können sich Lernende auf das jeweils neue Konzept konzentrieren, ohne vorherige Lösungen erneut eingeben zu müssen. Aus Sicht der Cognitive Load Theory (vgl. Abschnitt 2.2.1) reduziert dies den Extraneous Load und schafft kognitive Kapazität für den Aufbau neuer Schemata.

Kommentar-Markierungen im Startercode sind als Kommentare formulierte Platzhalter, die den

Bereich kennzeichnen, in dem Lernende eigenen Code ergänzen oder bestehenden Code anpassen sollen.

Hinweisblöcke mit unterschiedlichen Dringlichkeitsstufen warnen vor typischen Fehlern oder zeigen strukturelle Vorlagen. Die Hinweisanzahl zeigt eine leichte Abnahme in späteren Aufgaben, wobei dies eine manuelle Autorenentscheidung ist und kein programmierter Fading-Mechanismus.

Das einzige *dynamische Scaffolding* wird durch den KI-Tutor (nur Gruppe B) realisiert: Sein progressives Hint-System eskaliert die Hilfestellung während der Aufgabenbearbeitung über fünf Stufen von reinen Rückfragen bis zur vollständigen Lösung. Der KI-Tutor ist für Lernende in Gruppe B jederzeit verfügbar; der Hint-Zähler wird bei jeder neuen Aufgabe zurückgesetzt.

Part-task Practice

Die in Abschnitt 3.1.1 beschriebenen Drill Tasks entsprechen der 4C/ID-Komponente Part-task Practice (vgl. Abschnitt 2.2.4). Sie folgen dem Prinzip der *Deliberate Practice*, das gezielte, wiederholte Übung mit informativem Feedback zur Automatisierung von Teilfertigkeiten beschreibt (vgl. Abschnitt 2.1.3).

Der Aufgabenpool umfasst 278 Aufgaben zu sieben Teilfertigkeiten: Textausgabe, Variablen, Datentypen, Zeichenketten, Listen, Schleifen und bedingte Anweisungen. Die Multiple-Choice-Fragen nutzen vier Fragetypen: Wissensabfrage, Ausgabe vorhersagen, Lücken füllen und Fehler finden. Damit decken sie verschiedene Stufen der revidierten Bloom'schen Taxonomie ab (vgl. Abschnitt 2.2.4), von Erinnern über Verstehen und Anwenden bis zu Analysieren. Die Kombination mit einer Code-Aufgabe ergänzt dieses passive Erkennen um aktives Anwenden.

Die Platzierung direkt nach jeder gelösten Aufgabe stellt die zeitliche Nähe zum gelernten Konzept sicher und gewährleistet eine ausreichende Datenbasis für die empirische Auswertung. Die Auswahl erfolgt kumulativ: In späteren Aufgaben stehen alle bisherigen Themen zur Verfügung, sodass auch frühere Konzepte wiederholt werden. Die Part-task Practice adressiert damit primär den *Near Transfer*: Durch wiederholtes Üben werden syntaktische und prozedurale Muster automatisiert, während die komplexen Learning Tasks, also die konkreten Aufgaben im Kurs, den *Far Transfer* auf neuartige Problemkontexte fördern (vgl. Abschnitt 2.2.4).

Supportive Information

Supportive Information unterstützt den Aufbau mentaler Modelle und wird gemäß dem 4C/ID-Modell vor oder während der Aufgabenbearbeitung bereitgestellt (vgl. Abschnitt 2.2.4). Im Lernsystem wird diese Komponente durch vier Elemente realisiert.

Konzeptuelle *Textblöcke* zu Beginn jeder Aufgabe erläutern das zugrunde liegende Konzept, bevor die eigentliche Aufgabenstellung folgt. Beispielsweise wird bei der Einführung von Variablen die Metapher einer beschrifteten Box verwendet, bei bedingten Anweisungen das Grundprinzip einer Ja/Nein-Entscheidung erklärt.

Worked Examples als eingebettete, annotierte Code-Beispiele zeigen Konzepte in Aktion, ohne dass Lernende selbst aktiv werden müssen. Diese Beispiele demonstrieren die korrekte Syntax und Struktur und sind mit Erklärungen versehen.

Das *Onboarding* baut das mentale Modell der Plattform auf und vermittelt die Bedienung der Bedienoberfläche sowie grundlegende Python-Konventionen anhand annotierter Screenshots und eines vollständigen Beispielprogramms.

Für Gruppe B dient das *KI-Intro-Dialogfenster* als zusätzliche Orientierung: Der KI-Assistent begrüßt die Nutzenden, fragt nach Vorkenntnissen und motiviert, was im 4C/ID-Sinne als Elaboration und Aktivierung von Vorwissen dient.

Just-in-time Information

Just-in-time Information liefert prozedurale Hilfe genau dann, wenn sie benötigt wird (vgl. Abschnitt 2.2.4). Im Lernsystem wird diese Komponente durch mehrere Elemente realisiert, die sich in ihrer Auslösung unterscheiden.

Hinweisblöcke im Aufgabenbereich liefern prozedurale Rezepte für den aktuellen Schritt, etwa Warnungen vor typischen Fehlern oder strukturelle Vorlagen für die erwartete Syntax. Diese Hinweise sind technisch immer sichtbar und nicht erst bei einem Fehler. Das ist eine pragmatische Entscheidung, die präventive Hilfe gegenüber strenger Bedarfssteuerung priorisiert.

Kommentar-Markierungen im Startercode markieren die Stelle, an der Lernende handeln müssen und liefern prozedurale Anweisungen direkt am Punkt der Aktion im Editor.

Die *Konsolenausgabe* nach jeder Code-Ausführung zeigt das Ergebnis oder die vollständige Fehlermeldung. Kontextbezogene Kurzmeldungen klassifizieren Fehler und geben Hinweise zur Korrektur. Eine visuelle Rückmeldung bei korrekter Lösung dient als nonverbales Erfolgssignal.

Der *KI-Tutor* (nur Gruppe B) ist die einzige echte bedarfsgesteuerte Just-in-time-Komponente: Er reagiert erst auf Anfrage der Nutzenden und berücksichtigt den aktuellen Code, die aktuelle Fehlermeldung und den bisherigen Gesprächsverlauf. Damit übernimmt er die in Abschnitt 2.2.4 beschriebene Funktion des „über die Schulter schauenden Lehrers“ in digitaler Form. Damit wird die im 4C/ID-Modell vorgesehene Trennung zwischen aufgabenklassenbezogenem Wissen und aufgabenspezifischer Anleitung funktional umgesetzt, auch wenn einzelne Just-in-time-Elemente aus Gründen der Bedienbarkeit und technischen Praktikabilität permanent angezeigt werden.

3.2.2 Rolle der KI

Die KI übernimmt im Lernsystem zwei distinkte Aufgaben: die adaptive Auswahl von Drill Tasks und die kontextsensitive Hilfestellung über einen KI-Tutor.

KI-gestützte Drill-Task-Auswahl

Beide Versuchsgruppen greifen auf denselben, kuratierten Drill-Aufgabenpool zurück. Der Pool ist auf Drills bis zum aktuellen Kurs-Step beschränkt und enthält damit ausschließlich Aufgaben zu bereits eingeführten Konzepten. So wird verhindert, dass Lernende mit noch nicht behandelten Inhalten konfrontiert werden und die Vergleichbarkeit zwischen den Gruppen bleibt gewahrt. Jede Drill-Aufgabe ist mit Metadaten versehen, unter anderem mit einem vom Autor vergebenen didaktischen *Prioritätswert* (Gewichtung). Dieser Prioritätswert beschreibt, wie zentral eine Aufgabe für die Automatisierung der jeweiligen Teilfertigkeit eingeschätzt wird und operationalisiert damit die didaktische Relevanz im Aufgabenpool.

Gruppe A (gewichtete Zufallsauswahl): In der Kontrollgruppe wird pro Drill Task eine Aufgabe per Zufall gezogen, wobei die Ziehungswahrscheinlichkeit durch den Prioritätswert gewichtet wird. Zentralere Übungsschritte werden dadurch häufiger präsentiert, während Randthemen seltener erscheinen.

Gruppe B (KI-gestützte Auswahl): In der Experimentalgruppe übernimmt ein Sprachmodell die Auswahl der Drill-Kombination. Um die unabhängige Variable der Studie sauber zu isolieren, bleibt der zugrunde liegende Aufgabenpool identisch; lediglich das Auswahlverfahren unterscheidet sich. Dem Modell werden der aktuelle Kursfortschritt, die behandelten Themen, Performance-Daten (z. B. Erfolgsrate, häufige Fehlertypen, bisherige Drill-Versuche) sowie der verfügbare Aufgabenpool inklusive Metadaten übergeben. Es wird angewiesen, bei niedriger Erfolgsrate einfachere Aufgaben zu bevorzugen und bei hoher Erfolgsrate schwierigere, noch nicht gezeigte Aufgaben zu priorisieren und die Fragetypen zu variieren. Die ausgewählte Multiple-Choice-Frage und die Code-Aufgabe sollen thematisch aufeinander abgestimmt sein. Das Modell muss seine Auswahl begründen. Diese Begründungen werden protokolliert und mit einer anonymisierten ID für die spätere Auswertung und Qualitätssicherung gespeichert; sie werden den Lernenden nicht angezeigt.

Die Schwierigkeit wird dabei über einen zusammengefassten Performance-Score (Prozentwert) gesteuert, den das Sprachmodell bei der Auswahl berücksichtigt. Es gibt keine festen Schwellen (z. B. „leicht/mittel/schwer“); das Modell trifft stattdessen eine heuristische Entscheidung, wie der Score zu interpretieren ist. Diese einfache, eindimensionale Steuerung wurde bewusst gewählt, um die unabhängige Variable der Studie kontrollierbar zu halten.

Die KI-gestützte Auswahl erhöht die Flexibilität, reduziert jedoch die Reproduzierbarkeit einzelner Auswahlentscheidungen.

KI-Tutor mit progressivem Hint-System

Der KI-Tutor folgt der Philosophie, nicht die Lösung vorzugeben, sondern zum selbstständigen Denken anzuregen. Dies ist eine zentrale Maßnahme gegen die in Abschnitt 2.2.1 beschriebenen Risiken selbstregulierten Lernens. Dies wird durch ein progressives Hint-System mit fünf Eskalationsstufen umgesetzt:

- **Stufe 1:** Der KI-Tutor stellt ausschließlich Verständnisfragen.
- **Stufe 2:** Der KI-Tutor erklärt die relevanten Grundkonzepte.
- **Stufe 3:** Der KI-Tutor zeigt allgemeine Code-Muster, die nicht aufgabenspezifisch sind aber den Nutzenden helfen sollen.
- **Stufe 4:** Der KI-Tutor gibt eine Code-Struktur mit Lücken und generischen Variablennamen vor.
- **Stufe 5:** Der KI-Tutor liefert die vollständige Lösung mit ausführlicher Erklärung.

Die Eskalation erfolgt durch den Zähler der Hilfe-Anfragen pro Aufgabe und wird bei jeder neuen Aufgabe zurückgesetzt. Ein zentraler Schutzmechanismus adressiert das in Abschnitt 2.3.2 beschriebene Lösungs-Leak-Risiko: Die Musterlösung wird dem Sprachmodell erst ab Stufe 5 im Kontext mitgegeben. Unterhalb dieser Stufe hat das Modell keinen Zugriff auf die Lösung und kann sie folglich nicht weitergeben, selbst wenn die Nutzenden direkt danach fragen.

Der KI-Tutor erhält bei jeder Anfrage umfassenden Kontext: die aktuelle Aufgabe, den Code im Editor, die Konsolenausgabe mit Fehlerklassifikation sowie das aktuelle Hint-Level. Über einen Mechanismus zur Konversationskontinuität bleiben vorherige Nachrichten der Nutzenden im Kontext erhalten, ohne bei jeder Anfrage erneut mitgesendet werden zu müssen.

Entscheidung gegen dynamische Aufgabengenerierung

Die Entscheidung für eine Pool-Auswahl anstelle dynamischer Generierung war das Ergebnis praktischer Erfahrung. In einer frühen Entwicklungsphase wurde die dynamische Generierung erprobt, jedoch verworfen. Die Erfahrungen aus der Implementierung des KI-Tutors zeigten, dass es bereits anspruchsvoll ist, ein Sprachmodell mit dem Nutzungskontext vertraut zu machen und thematisch im definierten Bereich zu halten. Die eigenständige Bewertung, ob eine generierte Aufgabe qualitativ hochwertig, thematisch passend und frei von noch nicht behandelten Konzepten ist, erwies sich im zeitlichen und methodischen Rahmen dieser Bachelorarbeit als nicht zuverlässig lösbar.

Zudem hätte eine dynamische Generierung die Vergleichbarkeit der Ergebnisse der Studie erschwert, da verschiedene Nutzende unterschiedliche Aufgabeninhalte erhalten hätten. Durch den statischen Pool variiert ausschließlich die Auswahlmethode, wodurch die unabhängige Variable der Studie sauber isoliert wird.

Die endgültige Lösung besteht aus einem statischen, vom Autor geprüften Aufgaben-Pool mit KI-gesteuerter Auswahl. Sie adressiert das in Abschnitt 2.3.2 beschriebene Risiko mangelnder Qualitätssicherung und stellt einen Kompromiss zwischen Zuverlässigkeit und Innovation dar. Eine dynamische Generierung mit verbesserter Validierung, etwa durch ein zweites Modell zur Qualitätsprüfung, bleibt eine Option für zukünftige Iterationen.

Abgrenzung der Versuchsgruppen

Das System unterstützt zwei Modi, deren Konfiguration über die Gruppenzugehörigkeit gesteuert wird. Tabelle 1 fasst die Unterschiede zusammen.

Merkmal	Gruppe A	Gruppe B
Aufgabeninhalte	identisch	identisch
Drill-Aufgabenpool	identisch	identisch
Drill-Auswahl	gewichteter Zufall	KI-gestützt
KI-Tutor	nicht verfügbar	verfügbar
Onboarding	3 Schritte	4 Schritte
KI-Begrüßung	keine	interaktives Dialogfenster

Tabelle 1: Vergleich der Versuchsgruppen

Die Aufgabeninhalte, der Startercode und die Hinweisblöcke sind für beide Gruppen identisch. Diese Abgrenzung ermöglicht einen kontrollierten Vergleich, der ausschließlich den Einfluss der KI-gestützten Funktionen misst. Das Studiendesign und die Hypothesen werden in Kapitel 5 beschrieben.

Aufbauend auf der didaktischen Konzeption und der definierten Rolle der KI wird im Folgenden die Systemarchitektur beschrieben, welche die zuvor erläuterten Funktionen technisch trägt.

3.3 Systemarchitektur

Die Systemarchitektur legt fest, wie die zuvor beschriebenen Anforderungen auf technischer Ebene umgesetzt werden.

3.3.1 Überblick

Die Systemarchitektur besteht aus vier Hauptkomponenten: dem Client (Browser), dem Server, einer Datenbank und einem externen KI-Dienst. Eine Besonderheit der Architektur ist die direkte Kommunikation zwischen Client und Datenbank ohne Umweg über den Server. Der Server fungiert ausschließlich als Vermittler für die KI-Kommunikation.

Die Kommunikationswege sind wie folgt organisiert:

- **Client ↔ Datenbank:** Direkte Lese- und Schreiboperationen für Nutzerdaten, Fortschritt, Events und Chat-Verläufe
- **Client → Server:** Anfragen für KI-Funktionen (Drill-Auswahl, KI-Tutor-Antworten)

- **Server** → **KI-Dienst**: Weiterleitung der KI-Anfragen mit serverseitig geschütztem API-Schlüssel
- **Client** → **Code-Sandbox**: Lokale Python-Ausführung im Browser ohne Netzwerkinteraktion

Die funktionale Aufteilung zwischen Client und Server ist bewusst asymmetrisch.

Der Client übernimmt die gesamte Geschäftslogik: Darstellung der Bedienoberfläche, Code-Eingabe, Python-Ausführung in der Sandbox, Lösungsvalidierung, Drill-Anzeige, Datenbankoperationen, Gruppenzuweisung bei der Anmeldung und Event-Logging. Der Server beschränkt sich auf zwei Funktionen: die Weiterleitung von KI-Tutor-Anfragen und die Weiterleitung von Drill-Auswahl-Anfragen an den externen KI-Dienst. Diese Architektur minimiert Serverabhängigkeiten und gewährleistet, dass die Kernfunktionalität der Lernumgebung auch bei einem Ausfall des KI-Dienstes für Gruppe A vollständig funktionsfähig bleibt.

Die Aufgaben und Kursinhalte werden als statische Dateien im Quellcode gespeichert, nicht in der Datenbank. Diese Entscheidung gewährleistet Reproduzierbarkeit und Versionskontrolle. Nutzerdaten, Fortschritt, Event-Logs und Chat-Verläufe werden in der Datenbank persistiert. Chat-Verläufe werden zusätzlich beim KI-Dienst gespeichert, um die Konversationskontinuität über einen Referenz-Mechanismus zu ermöglichen.

Nach dem Architekturüberblick wird im nächsten Schritt die KI-Anbindung als zentraler technischer Baustein der Experimentalgruppe B konzeptionell eingeordnet.

3.3.2 KI-Anbindung und Prompt-Design

Die KI-Funktionen werden über einen externen Sprachmodell-Dienst realisiert. Auf ein selbst gehostetes oder fein abgestimmtes Modell wurde verzichtet, da der Aufwand für Infrastruktur, Training und Wartung den Rahmen eines Prototyps übersteigt. Ein spezialisiertes, auf den Anwendungsfall trainiertes Modell hätte potenziell bessere Ergebnisse liefern können, war jedoch zeitlich nicht realisierbar.

Das Prompt-Design wird konzeptionell durch drei Leitprinzipien bestimmt: klare Rollen- und Aufgabenbeschreibung, kontextsensitive Anfrageaufbereitung sowie explizite Verhaltensgrenzen.

Damit wird sichergestellt, dass die KI-Antworten pädagogisch ausgerichtet, auf den aktuellen Lernstand bezogen und mit den Studienzielen kompatibel bleiben. Für den Fall fehlerhafter oder ausbleibender KI-Antworten wurde ein Fallback-Konzept vorgesehen, um die Nutzbarkeit der Lernplattform während der Studie aufrechtzuerhalten. Die konkrete technische Ausgestaltung der Prompts, Antwortformate, Restriktionen und Fallback-Logik für Drill-Auswahl und KI-Tutor wird in Kapitel 4 beschrieben. Screenshots der fertigen Lernplattform, die die im Konzept beschriebenen Oberflächen illustrieren, sind in Anhang D zusammengestellt.

4 Implementierung

Aufbauend auf der in Kapitel 3 beschriebenen Konzeption und dem in Kapitel 2 dargestellten theoretischen Rahmen dokumentiert dieses Kapitel die technische Umsetzung des Prototyps. Es werden die Technologieauswahl begründet, die Kernkomponenten beschrieben und auf Herausforderungen während der Entwicklung eingegangen.

4.1 Technologieauswahl

Die Technologieauswahl orientierte sich an den Anforderungen aus Abschnitt 3.1.2: niedrige Einstiegshürden für Programmierneinsteigende, Nutzbarkeit ohne Softwareinstallation und effiziente Entwicklung des Prototyps. Entsprechend wurden etablierte Bibliotheken und Frameworks gewählt, um den Entwicklungsaufwand auf die fachlichen Kernanforderungen zu konzentrieren.

Die Lernplattform wurde als Webanwendung auf Basis von Next.js entwickelt, einem quelloffenen Framework auf Grundlage der JavaScript-Bibliothek React [35] für Fullstack-Webanwendungen [36]. Next.js bietet eine solide Grundlage für interaktive Bedienoberflächen und ermöglicht gleichzeitig die Integration serverseitiger Funktionalitäten. Die Wahl fiel auf Next.js aufgrund des integrierten Routings, der flexiblen Komposition von client- und serverseitigen Funktionen sowie bestehender Erfahrung des Autors. Als Programmiersprache kam TypeScript [37] zum Einsatz, dessen statische Typisierung die Wartbarkeit und Fehlersicherheit des Codes verbessert und durch den konsistenten Einsatz sowohl im Client als auch im Server eine einheitliche Codebasis gewährleistet.

Für die Gestaltung der Bedienoberfläche wurde eine Kombination aus Tailwind CSS [38] und Chakra UI [39] gewählt. Tailwind CSS ermöglicht durch seinen Utility-First-Ansatz die Erstellung responsiver Layouts, während Chakra UI vorgefertigte, barrierefreie Komponenten bereitstellt. Für die Code-Eingabe im Browser wurde der Monaco Editor [40] integriert, der auch in Visual Studio Code [41] zum Einsatz kommt und Syntax-Highlighting, Code-Vervollständigung sowie weitere Funktionen einer professionellen Entwicklungsumgebung bietet.

Die Datenhaltung erfolgt über Firebase Firestore [42], eine NoSQL-basierte Dokumentendatenbank, die die effiziente Speicherung und Abfrage von Nutzerdaten, Aufgabenfortschritten und Log-Daten ermöglicht. Die Wahl fiel auf Firestore aufgrund des kostenlosen Kontingents für prototypische Anwendungen, der integrierten Verwaltungsoberfläche sowie der serverlosen Architektur, die den Implementierungsaufwand reduziert. Die Authentifizierung wird über Firebase Authentication [42] realisiert, das E-Mail/Passwort-Anmeldung und die Integration mit Firestore für die Zugriffskontrolle bietet.

Die Ausführung von Python-Code erfolgt clientseitig über Pyodide [43], eine auf WebAssembly [44] basierende Python-Portierung für den Browser. Die direkte Ausführung ohne Serverinteraktion reduziert Latenz, und die integrierte Sandbox-Umgebung gewährleistet eine sichere Code-Ausführung.

Die KI-Integration basiert auf der OpenAI API [25], deren Grundlagen und Limitationen in Abschnitt 2.3.1 dargestellt wurden. Diese werden zur Auswahl individualisierter Übungsaufgaben sowie zur Unterstützung der Lernenden eingesetzt. Die Nutzung der API ermöglichte die Integration leistungsstarker KI-Funktionen ohne den Aufbau eigener Infrastruktur.

Die gewählten Technologien bilden eine kohärente Fullstack-Architektur: Next.js orchestriert Präsentationsschicht und serverseitige Logik, während Pyodide eine vollständig clientseitige Code-Ausführung ermöglicht. Die OpenAI API wird ausschließlich serverseitig über Next.js Server Actions angesprochen, um API-Schlüssel vor dem Client zu schützen.

Die Lernplattform wird auf einem Virtual Private Server mit Ubuntu 22.04 LTS [45] bei Hetzner [46] gehostet, gewählt aufgrund zuverlässiger Dienste, attraktiver Preisgestaltung und des Standorts in Deutschland. Für das Deployment kommt Coolify [47] zum Einsatz, das die automatisierte Bereitstellung als Docker-Container [48] einschließlich SSL-Zertifikaten, Reverse-Proxy-Konfiguration und Git-Integration übernimmt.

4.2 Umsetzung der Kernkomponenten

Die Kernkomponenten der Plattform umfassen die Web-Oberfläche, die Aufgaben-Engine mit KI-Integration sowie die Nutzerverwaltung und das Logging-System.

4.2.1 Web-Oberfläche

Die Bedienoberfläche der Lernplattform wurde mit Fokus auf Zugänglichkeit und Bedienfreundlichkeit für Programmierneinsteigende gestaltet. Der Einstieg erfolgt über eine Login-Seite, auf der Nutzende einen selbstgewählten Benutzernamen eingeben und ein gemeinsames Passwort verwenden. Diese Seite enthält zusätzlich ein deutschsprachiges Erklärvideo, das den Ablauf der Anwendung und ihre Funktionen vorstellt, um technisch unerfahrenen Nutzenden den Einstieg zu erleichtern.

Nach der Anmeldung durchlaufen Nutzende ein mehrstufiges Onboarding, das schrittweise die zentralen Komponenten der Plattform erklärt. Dazu gehören der Code-Editor, der Aufgabenbereich, die Konsolenausgabe sowie das Konzept der Drill Tasks mit Multiple-Choice-Fragen und Mini-Code-Aufgaben. Im Onboarding wird den Nutzenden mitgeteilt, zu welcher Versuchsgruppe sie gehören. Gruppe A arbeitet ohne KI-Unterstützung, Gruppe B mit KI-Tutor. Das Onboarding nutzt Screenshots zur visuellen Unterstützung und vermittelt grundlegende Hinweise zur Python-Programmierung. Nutzende der Gruppe B werden nach dem Onboarding aufgefordert, den KI-Tutor zu personalisieren, indem sie ihr Erfahrungslevel mit Programmierung und Python angeben. Diese Informationen dienen dazu, die KI-gestützten Hilfestellungen an das individuelle Niveau anzupassen.

Nach Abschluss des Onboardings gelangen Nutzende zum Dashboard, das eine Übersicht aller verfügbaren Kurse und Aufgaben bietet. Das Dashboard visualisiert den Lernfortschritt durch

farbliche Markierungen: Gelöste Aufgaben werden grün hervorgehoben, offene Aufgaben neutral dargestellt. Die Kursfreischaltung folgt der in Abschnitt 3.1.1 beschriebenen linearen Progression. Der Aufgabenbereich ist in einem zweiseitigen Layout organisiert. Links befindet sich der Monaco Editor für die Code-Eingabe, rechts die Aufgabenstellung im Markdown-Format [49] mit Code-Blöcken und farblich hervorgehobenen Hinweisboxen für wichtige Informationen und Warnungen. Unterhalb des Editors wird die Konsolenausgabe angezeigt, die der originalen Python-Ausgabe entspricht. Nutzende lösen Aufgaben, indem sie die erwartete Ausgabe per `print`-Funktion erzeugen, woraufhin das System diese automatisch gegen vordefinierte Lösungsstrings validiert.

Für Gruppe B steht ein permanenter blauer Button am rechten Bildschirmrand zur Verfügung, über den ein seitliches Dialogfenster mit dem KI-Tutor geöffnet werden kann. Dieses Fenster gleitet von rechts in den Bildschirm und ermöglicht kontextsensitive Hilfestellungen während der Aufgabenbearbeitung. Innerhalb der Drill Tasks ist der Chat jedoch nicht verfügbar, was in der empirischen Studie teilweise zu Frustration führte.

Die Navigation innerhalb eines Kurses erfolgt über eine Navigationsleiste mit nummerierten Buttons. Gelöste Schritte sind visuell hervorgehoben und können erneut aufgerufen werden. Der Navigationsfluss setzt die in Abschnitt 3.1.1 beschriebene lineare Progression um: Nach erfolgreicher Lösung öffnet sich die zugehörige Drill Task, erst nach deren Abschluss erfolgt die Weiterleitung zur nächsten Aufgabe.

Farbcodierung dient als unmittelbares visuelles Feedback: Richtige Lösungen werden positiv hervorgehoben, fehlerhafte Lösungen erzeugen kontextbezogene Hinweismeldungen. Nutzende haben für alle Drill Tasks unbegrenzt viele Versuche. Fehlermeldungen in der Konsolenausgabe entsprechen den originalen Python-Fehlern, die an kritischen Stellen durch zusätzliche Hinweise ergänzt werden.

Der Monaco Editor wurde mit Syntax-Highlighting, statischem Autocomplete für Python-Built-ins und Hover-Dokumentation konfiguriert. Jede Aufgabe beginnt mit einem vorgegebenen Startercode, der in der Regel auf der Lösung der vorherigen Aufgabe aufbaut.

Kommentare im Code markieren die Stellen, an denen Nutzende Änderungen vornehmen sollen. Screenshots der fertigen Bedienoberfläche mit Dashboard, Aufgabenbereich, Drill Tasks und KI-Tutor sind in Anhang D zusammengestellt.

4.2.2 Aufgaben-Engine und KI-Integration

Die Lernplattform basiert auf einer hybriden Architektur, die statische Aufgaben mit KI-gestützter adaptiver Aufgabenauswahl kombiniert.

Statische Aufgaben

Die Hauptlernpfade für den BMI-Rechner mit 12 Aufgaben und den Zinsrechner mit 8 Aufgaben sind als typisierte TypeScript-Objekte in separaten Dateien hinterlegt.¹ Diese Entscheidung für eine Code-basierte Speicherung anstelle einer Datenbank gewährleistet Reproduzierbarkeit für die Forschungsstudie und erleichtert Versionskontrolle über Git.

Jede Aufgabe folgt einem definierten Interface `CourseTask` mit folgenden Kernfeldern:

- `id` als eindeutige Kennung, `step` als Position in der Sequenz sowie `title` und `description`
- `blocks` als Liste aus Content-Blöcken mit Text in Markdown, Code-Beispielen, Hinweisen mit Severity-Levels und Aufgabenstellungen
- `starterCode` als Vorgabe-Code im Editor
- `solutionString` als erwartete Konsolenausgabe, die als String oder als Array mehrerer Strings vorliegen kann
- `solutionCode` als optionales Feld mit erforderlichen Code-Snippets für die Anti-Cheat-Prüfung
- `topics` als Liste abgedeckter Themen für die Drill-Auswahl
- `hasDrill` als Boolean-Wert, der angibt, ob nach Abschluss eine Drill Task folgt

Aufgaben werden zur Laufzeit vom Backend über Hilfsfunktionen wie `getBmiCourseTask` geladen. Dabei werden die Parameter `step` und `locale` übergeben. Die Funktionen unterstützen auch die Lokalisierung in Deutsch und Englisch mit Fallback-Mechanismen.

Code-Ausführung

Die Ausführung des eingegebenen Codes erfolgt vollständig clientseitig über Pyodide v0.27.0, eine WebAssembly-Kompilierung von Python, die über ein Content Delivery Network geladen wird. Der Ablauf ist in der Funktion `runPythonCode` implementiert. Der eingegebene Code wird in einen Wrapper eingebettet, der `sys.stdout` auf ein `StringIO`-Objekt umleitet. Dieses Objekt fungiert als Puffer für String-Ausgaben im Arbeitsspeicher, um alle `print`-Ausgaben abzufangen. Der Code wird innerhalb eines `try/finally`-Blocks ausgeführt, wobei `sys.stdout` anschließend zurückgesetzt wird. Zum Schutz vor Endlosschleifen wird der Code mit einem Iterations-Counter instrumentiert, der nach maximal 100.000 Iterationen abbricht. Das Ergebnis umfasst den Output-String, eine Fehlerklassifikation und den Ausführungsstatus.

Die clientseitige Ausführung eliminiert Server-Abhängigkeiten, reduziert Latenz und gewährleistet durch die Pyodide-Sandbox sichere Code-Ausführung.

¹Die Kursdaten sind in `bmiCalculatorCourseData.ts` und `interestCalculatorCourseData.ts` im Quellcode des Projekts definiert (vgl. Anhang D).

Lösungsvalidierung

Die Validierung erfolgt dreistufig in der Funktion `validateSolution`:

1. **Python-Fehler-Check:** Die Ausgabe wird auf Schlüsselwörter wie „Error“, „Traceback“ und „Exception“ geprüft. Bei Vorhandensein wird die Lösung sofort als ungültig markiert.
2. **Anti-Cheat-Check:** Falls `solutionCode` definiert ist, muss der eingegebene Code bestimmte Snippets enthalten, zum Beispiel den Ausdruck `print`. Dies verhindert Hardcoding der erwarteten Ausgabe ohne Verwendung der gelernten Konzepte.
3. **Output-Vergleich:** Die Konsolenausgabe wird gegen `solutionString` geprüft. Bei Arrays müssen alle Strings enthalten sein. Der Vergleich ist whitespace-tolerant mit Normalisierung von Leerzeichen um Operatoren, Umwandlung einfacher in doppelte Anführungszeichen und Bereinigung von Leerzeichen in Funktionsaufrufen.

Bei korrekter Lösung erscheint eine Konfetti-Animation als positive Rückmeldung. Fehlerhafte Lösungen führen zu kontextbezogenen Toast-Benachrichtigungen mit den Meldungen „Python-Fehler“, „ProgressError“ und „incorrectSolution“.

KI-gestützte Drill-Task-Auswahl

Die in Abschnitt 3.1.1 beschriebenen Drill Tasks wurden vom Autor händisch erstellt und geprüft, teilweise unter Nutzung des KI-gestützten Entwicklungswerkzeugs Claude Code [50]. Sie sind in einem vordefinierten Pool hinterlegt, aus dem die Auswahl entweder algorithmisch für Gruppe A oder KI-gestützt für Gruppe B erfolgt.

Für **Gruppe A** kommt eine gewichtete Zufallsauswahl über `weightedRandomSelect` zum Einsatz, die auf Basis der Aufgaben-Prioritäten und bereits gezeigter Drills eine thematisch passende Auswahl trifft.

Für **Gruppe B** erfolgt die Auswahl über die OpenAI API mit dem Modell `gpt-4o-mini` mittels einer serverseitigen Next.js Server Action namens `selectDrillsFromPoolWithAI`. Der technische Ablauf:

1. Nutzende öffnen Drill Task nach gelöster Aufgabe
2. Frontend sendet Request an das Backend über eine Next.js Server Action
3. Backend bereitet Anfrage für OpenAI auf mit: Kurskontext (Step, Topics), Leistungsprofil der Nutzenden (gelöste Drills, Erfolgsrate, Fehlertypen) und verfügbaren Aufgabenpools mit Metadaten
4. OpenAI wählt didaktisch optimale Kombination aus MCQ und Code-Aufgabe
5. Backend gibt ausgewählte Aufgaben ans Frontend zurück

Die Auswahl dauert zwischen 0,5 und 3 Sekunden, währenddessen wird ein Loading-Skeleton angezeigt. Als Fallback bei ungültiger KI-Auswahl wird die erste verfügbare Aufgabe aus dem Pool gewählt. Bei einem vollständigen Ausfall des KI-Dienstes fällt die Drill-Auswahl auf den Algorithmus der Kontrollgruppe mit gewichteter Zufallsauswahl zurück, um den Lernfluss aufrechtzuerhalten.

Das verwendete Modell `gpt-4o-mini` wurde gewählt aufgrund der Balance zwischen Kosten, Geschwindigkeit und Qualität. Es unterstützt die in Abschnitt 2.3.1 beschriebenen *Structured Outputs* [26], die das Sprachmodell zwingen, Antworten in einem vordefinierten JSON-Schema-Format zu liefern, was vorhersagbare und typsichere Antwortformate gewährleistet. Die Implementierung nutzt die OpenAI Responses API aus dem Jahr 2024 mit Konversationskontinuität über `previous_response_id`, was manuelles Token-Management vermeidet und die Gesprächskontinuität über mehrere Anfragen hinweg sicherstellt.

Das Prompt-Design folgt vier Prinzipien: Rollenweisung zur pädagogischen Ausrichtung, Kontextinjektion mit Aufgaben- und Performancedaten, strukturierte Ausgabe über striktes JSON-Schema und explizite Verhaltensrestriktionen. Die Prompts sind instruktionsbasiert ohne Few-Shot-Beispiele.

KI-Tutor-Integration

Für Gruppe B steht ein KI-Tutor zur Verfügung, der über ein persistentes seitliches Dialogfenster erreichbar ist. Der System-Prompt definiert die Rolle als „hilfreicher AI-Assistent für das Python-Bootcamp“ mit einer klaren Kernphilosophie: nicht die Lösung vorgeben, sondern zum selbstständigen Denken anregen.

Umgesetzt wird dies über das in Abschnitt 3.2.2 beschriebene progressive Hint-System mit fünf Eskalationsstufen, das mit jeder Hilfe-Anfrage der Nutzenden eine Stufe weiter eskaliert.

Der Anfrage-Prompt enthält die Nachricht der Nutzenden sowie umfassenden Kurskontext, darunter die aktuelle Aufgabe mit Titel und Beschreibung, den aktuellen Code im Editor zusammen mit dem Startercode, die Konsolenausgabe mit Fehler-Flag, den Zähler der Hilfeanfragen und das aktuelle Hint-Level. Die Solution-Strings werden erst ab Level 5 mitgesendet, um vorzeitiges Preisgeben der Lösung zu verhindern.

Die KI-Antworten folgen einem strukturierten JSON-Format mit den Types `text`, `admonition` und `codeblock`, um eine formatierte Darstellung im Chat-Drawer zu ermöglichen. Die Antworten enthalten weder LaTeX noch Formeln, sondern ausschließlich Unicode-Text oder Python-Code-Blöcke. Die Sprachauswahl erfolgt basierend auf der Sprachoption der Nutzenden, primär Deutsch und optional Englisch.

Zur Qualitätssicherung wird die KI auf einen händisch geprüften Aufgabenpool restringiert. Dennoch kann es vorkommen, dass die gewählte Aufgabe nicht optimal zum aktuellen Lernstand passt. Bei fehlerhaften KI-Tutor-Antworten wird eine lokalisierte Fehlermeldung angezeigt. Ein dedizierter Wiederholungsmechanismus bei API-Fehlern wurde nicht implementiert.

4.2.3 Nutzerverwaltung und Logging

Dieser Abschnitt beschreibt die technische Umsetzung der anonymen Nutzerverwaltung mit automatischer Gruppenzuweisung sowie des Event-Sourcing-basierten Logging-Systems.

Nutzerverwaltung und Gruppenzuweisung

Die Plattform verzichtet auf eine klassische Registrierung mit E-Mail-Verifizierung. Stattdessen wählen Nutzende einen beliebigen Benutzernamen und geben ein gemeinsames Passwort ein. Das System unterstützt vier Passwort-Varianten, die als Umgebungsvariablen konfiguriert sind. Dazu gehören je ein dediziertes Passwort für Gruppe A ohne KI und Gruppe B mit KI, ein balanciertes Passwort für automatische Gruppenzuweisung sowie ein Admin-Passwort für Verwaltungsfunktionen. Der eingegebene Benutzername wird normalisiert. Dabei werden Kleinbuchstaben verwendet und Leerzeichen durch Bindestriche ersetzt, zum Beispiel „Max Mustermann“ zu `max-mustermann`. Anschließend dient der Name als Firestore-Dokument-ID. Diese Architektur ermöglicht anonyme Teilnahme ohne Erhebung personenbezogener Identifikationsmerkmale und vereinfacht die DSGVO-Konformität.

Die Gruppenzuweisung erfolgt bei Nutzung des balancierten Passworts automatisch: Das System zählt die aktuelle Anzahl an Nutzenden in Gruppe A mit `aiChatTutorIsEnabled = false` und in Gruppe B mit `aiChatTutorIsEnabled = true` und weist die neue Person der kleineren Gruppe zu. Bei Gleichstand wird Gruppe A zugewiesen. Die Zuweisung ist permanent und wird nicht nachträglich geändert, um Störvariablen durch Gruppenwechsel zu vermeiden.

Nutzende sehen ihre Gruppenzugehörigkeit transparent im Onboarding mitgeteilt; Gruppe-B-Nutzende erkennen sie zusätzlich am blauen Chat-Button.

Event-Tracking und Logging

Das System nutzt eine Event-Sourcing-Architektur, bei der alle Aktivitäten der Nutzenden als chronologische, unveränderliche Ereignisse mit Zeitstempel in Firestore gespeichert werden. Die protokollierten Events umfassen fünf Kategorien: Session-Events mit Start, Ende und Metadaten, Aufgaben-Events mit Code-Ausführungen, Bearbeitungsdauer und Lösungsversuchen, Drill-Events mit Auswahl, Versuchen und Erfolg, Chat-Events mit Hint-Level und Nachrichten nur für Gruppe B sowie die abschließende Evaluation.

Zusätzlich wird ein persistentes Lernprofil gepflegt, das einen aggregierten Performance-Score, eine nach Fehlertyp und Thema klassifizierte Fehlerhistorie sowie adaptive Themen-Gewichtungen umfasst. Diese Daten bilden die Grundlage für die in Abschnitt 3.2.2 beschriebene KI-gestützte Drill-Auswahl. Die Datenschutzanforderungen aus Abschnitt 3.1.2 werden technisch durch anonyme Benutzernamen, eine Datenschutzseite unter `/datenschutz` und der dauerhaften Anonymisierung der Daten nach Abschluss der Studie umgesetzt.

4.3 Herausforderungen und Designentscheidungen

Im Verlauf der Implementierung wurden mehrere technische und konzeptionelle Herausforderungen identifiziert, die zu bewussten Designentscheidungen führten und nachfolgend dokumentiert werden.

4.3.1 Technische Herausforderungen

Die Pyodide-Integration verlief insgesamt weniger problematisch als erwartet, erforderte jedoch Anpassungen für die Einbindung in die React-App und die Handhabung asynchroner Operationen. Die Implementierung des Endlosschleifen-Schutzes mittels Iterations-Counter und die Kapselung von `sys.stdout` über `StringIO` erforderten mehrere Iterationen, um eine saubere Output-Capture zu gewährleisten.

Die Firebase-Integration brachte eigene Herausforderungen mit sich, insbesondere die Entwicklung einer geeigneten Dokumentstruktur für die Event-Sourcing-Architektur. Mehrere Iterationen waren nötig, um eine Balance zwischen Abfrage-Effizienz, Datenkonsistenz und überschaubarer Komplexität zu finden. Die finale Lösung mit Events als Sub-Collection und aggregierten Daten im Wurzel-Dokument der Nutzenden erwies sich als tragfähig.

Die größte Herausforderung stellte die Minimierung von Halluzinationen des KI-Tutors dar, wie in Abschnitt 2.3.2 beschrieben. Gemeint sind Antworten, die plausibel klingen, jedoch faktisch falsch oder frei erfunden sind. Besonders kritisch war die Aufrechterhaltung eines stabilen Nutzungskontexts über mehrere Interaktionen hinweg, sodass der KI-Tutor stets über Nutzernamen, bisherige Fehler, gestellte Fragen und die aktuelle Aufgabe informiert bleibt. Die Lösung erfolgte durch eine Kombination aus drei Mechanismen:

erstens die Nutzung von `previous_response_id` für echte Konversationskontinuität statt manueller Nachrichtenhistorie, zweitens Structured Outputs mit `strict: true` JSON Schema für vorhersagbare und typsichere Antwortformate sowie drittens das progressive Hint-System, das vorzeitiges Preisgeben der Lösung verhindert.

4.3.2 Verworfen Alternativen

Eine serverseitige Code-Ausführung wurde verworfen, da die ursprünglich vorgesehene Hosting-Plattform Vercel keine native Python-Runtime unterstützt und Stabilitätsrisiken durch einen dedizierten Python-Server bestanden. Die clientseitige Ausführung mit Pyodide erwies sich als einfachere und stabilere Lösung ohne Server-Abhängigkeit.

Die Nutzung eines lokalen, selbst gehosteten Sprachmodells anstelle der OpenAI API wurde aufgrund des hohen Aufwands für diesen Prototyp verworfen. Ein spezialisiertes, fein abgestimmtes Modell hätte potenziell qualitativer und näher an der Anwendungsdomäne sein können, war aber zeitlich nicht realisierbar.

Die dynamische Generierung von Drill Tasks zur Laufzeit wurde nach mehreren Versuchen ver-

worfen. Die Validierung dynamisch generierter Aufgaben erwies sich als unzuverlässig, da nicht sichergestellt werden konnte, dass die erwartete Antwort tatsächlich korrekt ist. Statische Aufgaben mit statischen Lösungsstrings waren deutlich einfacher und sicherer.

Die Implementierung eines vollwertigen Python Language Servers für Echtzeit-Syntaxprüfung im Editor wurde als zu komplex für den Prototyp-Umfang bewertet. Statisches Autocomplete für Built-ins erwies sich als pragmatischer Kompromiss.

4.3.3 Bewusste Abwägungen

Eine zentrale Abwägung bestand zwischen einem stringenten Prozessablauf und vollständiger Einstiegsfreundlichkeit. Während die Bedienoberfläche für Nutzende mit grundlegenden Computerkenntnissen gut funktionierte, war sie für Teilnehmende ohne jegliche Programmiererfahrung teils überfordernd. Der zeitliche Rahmen erlaubte keine umfassende Einführung in die Grundlagen der Programmierung, sodass direkt mit Python-Syntax begonnen wurde.

Gleichzeitig wurde bewusst auf eine skalierbare Architektur verzichtet zugunsten einer zügigen Prototyp-Entwicklung, da das primäre Ziel die Durchführung der empirischen Studie war.

Die Entscheidung für einen statischen Aufgabenpool mit KI-gesteuerter Auswahl anstelle dynamischer Generierung priorisierte Zuverlässigkeit gegenüber Flexibilität.

4.3.4 Limitierungen des Prototyps

Der Prototyp weist mehrere Limitierungen auf, die bei einer Weiterentwicklung adressiert werden sollten. Erstens ist der KI-Tutor konzeptionell ausschließlich für die Learning Tasks implementiert; innerhalb der Drill Tasks steht den Nutzenden systemseitig keine Hilfestellung zur Verfügung. Dadurch sind die Möglichkeiten einer durchgängigen tutorierten Begleitung eingeschränkt. Zweitens besteht derzeit keine Möglichkeit, blockierte Aufgaben zu überspringen oder zugeteilte Drill Tasks auszutauschen. Diese fehlende Flexibilität kann bei anhaltenden Schwierigkeiten zu einer Unterbrechung des Lernflusses führen und erschwert eine dynamische Anpassung an den individuellen Lernstand. Drittens fehlt ein dediziertes Fehlerbehandlungssystem für API-Ausfälle. Ein Ausfall der OpenAI API würde Gruppe-B-Nutzende bei der KI-gestützten Drill-Auswahl blockieren. Schließlich handelt es sich nicht um ein verblindetes Experiment: Nutzende wissen, welcher Gruppe sie angehören, was potenzielle Erwartungseffekte begünstigt.

Für einen produktiven Einsatz wären insbesondere niedrigschwellige Hilfeoptionen sowie eine kontrollierte Möglichkeit zum Überspringen oder Austauschen einzelner Aufgaben erforderlich, ohne Lernende zu frühzeitigem Aufgeben oder unreflektiertem Durchklicken einzuladen, damit weiterhin selbstreguliertes Lernen in hoher Qualität unterstützt wird.

4.4 Werkzeugunterstützung und KI-Nutzung

Die Entwicklung dieser Arbeit, sowohl die schriftliche Ausarbeitung als auch die technische Umsetzung des Prototyps, wurde durch den Einsatz digitaler Werkzeuge einschließlich KI-basierter Sprachmodelle unterstützt. Gemäß dem Ansatz der verteilten Kognition wirkten diese Werkzeuge nicht stellvertretend für den Autor, sondern als kognitive Hilfsmittel, die das eigene Denken unterstützten und strukturierten. Alle inhaltlichen Entscheidungen, Quellenangaben, Interpretationen und Schlussfolgerungen wurden vom Autor selbst getroffen und verantwortet. KI-generierte Ausgaben wurden nicht ungeprüft übernommen; insbesondere wurden bibliografische Angaben und DOIs manuell verifiziert (vgl. Anhang E).

Zentrales Werkzeug war die in Visual Studio Code [41] integrierte KI-Erweiterung GitHub Copilot (Microsoft), die die Sprachmodelle GPT-4o und GPT-5.1 sowie Claude Sonnet 4.6 und Claude Opus 4.6 (Anthropic) einbindet. Dieses wurde eingesetzt, um Formulierungsvorschläge zu erarbeiten, Gliederungsideen zu entwickeln, $\LaTeX$ -Syntax zu überprüfen sowie prototypischen Code zu generieren und zu analysieren. Darüber hinaus wurden zur Auseinandersetzung mit dem Forschungsstand und zur Strukturierung von Kapiteln Fragen an das Modell gestellt. Alle so entstandenen Inhalte wurden kritisch geprüft, überarbeitet und an die eigene Argumentation angepasst. Ein detailliertes Protokoll der verwendeten Werkzeuge findet sich in Anhang E. Repräsentative Screenshots der implementierten Web-Anwendung, darunter die Kursansicht, der KI-Chat und die Drill Tasks, sind in Anhang D dokumentiert.

5 Empirische Studie

Dieses Kapitel beschreibt das Design der empirischen Studie, sodass diese theoretisch reproduzierbar wäre. Die Untersuchung wurde als Online-Intervention im Zeitraum von Dezember 2025 bis Januar 2026 durchgeführt. Die Datenerhebung erfolgte systemseitig anonym und ortsunabhängig über die Webplattform.

5.1 Forschungsdesign und Hypothesen

Im Folgenden werden die zentrale Forschungsfrage, das Studiendesign, die Variablen und die Hypothesen der Untersuchung strukturiert dargestellt.

5.1.1 Studiendesign

Die Studie ist als *Between-Subjects-Design* mit zwei Versuchsbedingungen umgesetzt. Beide Gruppen bearbeiten dieselben Lernaufgaben (Learning Tasks) und greifen auf denselben redaktionell erstellten Drill-Aufgabenpool (Part-task Practice) zurück. Die Gruppen unterscheiden sich ausschließlich in der Verfügbarkeit der KI-Funktionen:

- **Gruppe A (ohne KI-Funktionen):** Drill Tasks werden per gewichteter Zufallsauswahl aus dem statischen Pool ausgewählt. Ein KI-Chat-Tutor ist nicht verfügbar.
- **Gruppe B (mit KI-Funktionen):** Drill Tasks werden KI-gestützt aus demselben statischen Pool ausgewählt (OpenAI API, Modell `gpt-4o-mini`). Zusätzlich steht ein KI-Chat-Tutor mit progressivem Hint-System zur Verfügung.

Die Gruppenzuteilung erfolgt als *Quasi-Randomisierung* durch balancierte systemseitige Zuteilung (vgl. Abschnitt 5.2). Der Lernerfolg wird über Logdaten und einen abschließenden Evaluationsbogen operationalisiert. Die daraus folgende deskriptive Auswertungsstrategie ist in Abschnitt 5.7 näher beschrieben.

5.1.2 Unabhängige Variable

Die unabhängige Variable ist der *Zugang zu KI-Funktionen* (ja/nein). Dieser Zugang beeinflusst (i) das Auswahlverfahren der Drill Tasks (Algorithmus vs. KI-gestützt) und (ii) die Verfügbarkeit eines KI-Tutors (nur Gruppe B).

5.1.3 Abhängige Variablen (Outcomes)

Die abhängigen Variablen werden über zwei Datenquellen erfasst, die in Abschnitt 5.5 detailliert beschrieben sind.

- **System-Logs:** Nutzungs- und Leistungsmetriken, die automatisch während der Bearbeitung aufgezeichnet werden, etwa Bearbeitungszeiten, Fehlerprofile und Drill-Ergebnisse. Für Gruppe B kommen zusätzlich KI-Nutzungsmetriken hinzu (vgl. Abschnitt 5.5.1).
- **Post-Evaluationsbogen:** Ein abschließender Fragebogen nach Kursende, der Selbsteinschätzungen zum Lernfortschritt, die wahrgenommene Systemqualität und den Cognitive Load erhebt (vgl. Abschnitt 5.5.2).

5.1.4 Forschungsfrage

Die zentrale Forschungsfrage dieser Arbeit lautet:

Inwiefern verbessert die Integration einer KI-gestützten Drill-Auswahl und eines KI-Tutors die lernbezogenen Indikatoren und die wahrgenommene Lernunterstützung beim Erlernen von Python-Grundlagen im Vergleich zu einem Basis-System ohne diese Komponenten?

5.1.5 Hypothesen

Aus der Forschungsfrage wird eine gerichtete Haupthypothese abgeleitet:

- **H1 (Haupthypothese):** Lernende in Gruppe B weisen im Vergleich zu Gruppe A günstigere lernbezogene Indikatoren in den Systemlogs (z. B. höhere Erfolgsquoten, weniger Fehlversuche, effizientere Bearbeitungszeiten) sowie eine signifikant höhere subjektive Zufriedenheit mit der Lernunterstützung im Post-Fragebogen auf.
- **H0 (Gegenhypothese):** Zwischen Gruppe A und Gruppe B bestehen keine systematischen Unterschiede in den erhobenen Logmetriken und Fragebogenwerten.

5.2 Stichprobe und Rekrutierung

Die Rekrutierung der Teilnehmenden erfolgte im Rahmen einer *Gelegenheitsstichprobe* (Convenience Sampling), primär durch persönliche Ansprache im sozialen Umfeld des Autors sowie über digitale Netzwerke. Die Teilnehmenden wiesen heterogene Hintergründe auf, wobei sowohl Studierende als auch Berufstätige mit unterschiedlichem IT-Vorwissen vertreten waren. Aufgrund der offenen Bereitstellung der Webplattform kann zudem nicht ausgeschlossen werden, dass weitere Teilnehmende über universitäre Kanäle auf das Projekt aufmerksam wurden.

Die Datenerhebung erfolgte *systemseitig anonym*: Die Registrierung setzte lediglich einen selbstgewählten Benutzernamen voraus, ohne dass personenbezogene Identifikationsmerkmale erhoben wurden. Gleichwohl ist kritisch anzumerken, dass durch die Akquise im privaten Umfeld teilweise ein direktes Bekantschaftsverhältnis zwischen den Testpersonen und der Studienleitung bestand. Dies schränkt die Repräsentativität der Ergebnisse ein und begünstigt potenzielle

Verzerrungen durch soziale Erwünschtheit (*Social Desirability Bias*), was bei der Interpretation der Daten in Abschnitt 7.3 reflektiert wird.

Für die statistische Auswertung wird zwischen verschiedenen Datenbeständen differenziert, wobei die zugrundeliegenden Bereinigungsregeln detailliert in Abschnitt 5.6 erläutert werden.

- **Gesamtregistrierungen:** Es registrierten sich insgesamt $N = 27$ Nutzende (Gruppe A: $n_A = 14$, Gruppe B: $n_B = 13$). Diese Rohzahl enthält mehrere interne Testkonten aus der Entwicklungsphase. Nach deren Entfernung verbleiben $N = 25$ reale Registrierungen (Gruppe A: $n_A = 14$, Gruppe B: $n_B = 11$).
- **Analyse-Stichprobe:** Um die Validität der Ergebnisse zu sichern, wurden nur Personen berücksichtigt, die eine *substanzielle Interaktion* mit den Aufgaben zeigten. Nach Anwendung dieses *Aktivitätsfilters* (mindestens drei Code-Ausführungen, vgl. Abschnitt 5.6) umfasst die finale Stichprobe $N = 18$ Teilnehmende, die paritätisch auf beide Gruppen verteilt sind ($n_A = 9$, $n_B = 9$).

Von dieser bereinigten Stichprobe füllten insgesamt $N = 16$ Personen den abschließenden Evaluationsbogen vollständig aus. Tabelle 2 fasst die Fallzahlen auf den jeweiligen Filterebenen zusammen.

Die Gruppenzugehörigkeit wird systemseitig zugewiesen und bleibt dauerhaft stabil. Bei Nutzung eines balancierten Studien-Passworts erfolgt die Zuteilung nach einer 50:50-Logik: Neue Nutzende werden der aktuell kleineren Gruppe zugeordnet, wobei bei Gleichstand Gruppe A zugeordnet wird. Die Zuteilung ist damit quasi-randomisiert.

Nutzende werden im Onboarding transparent über ihre Gruppenzugehörigkeit informiert. Das Experiment ist nicht verblindet.

Tabelle 2: Stichprobengrößen auf den einzelnen Filterebenen

Filterebene	Beschreibung	Gesamt	Gr. A	Gr. B
Gesamtregistrierungen	Alle registrierten Nutzenden inkl. Testkonten	27	14	13
Reale Registrierungen	Bereinigter Datensatz nach Ausschluss interner Testkonten	25	14	11
Analyse-Stichprobe	Anwendung des Aktivitätsfilters (<code>num_runs_code > 2</code>)	18	9	9
Evaluationsbogen	Vollständig abgeschlossene Post-Evaluation	16	8	8

Anmerkung: Gr. A = Kontrollgruppe (statische Auswahl); Gr. B = Experimentalgruppe (KI-gestützt). Die Entfernung der Testkonten erfolgte manuell direkt in der Datenbank. Der Aktivitätsfilter dient der Sicherstellung einer Mindestinteraktion mit dem System (Details vgl. Abschnitt 5.6).

5.3 Datenschutz und ethische Aspekte

Die Plattform ist so konzipiert, dass eine Teilnahme ohne Erhebung direkt personenbezogener Identifikationsmerkmale möglich ist (Pseudonymisierung). Die Registrierung erfolgt ausschließlich über einen selbstgewählten Benutzernamen und ein gemeinsames Studien-Passwort. Die Verarbeitung der Studiendaten basiert auf einer ausdrücklichen Einwilligung, die im Rahmen des Anmeldeprozesses eingeholt wird.

Die erhobenen Daten umfassen insbesondere systemseitige *Interaktionsmuster* (Events) sowie die Antworten aus dem Post-Evaluationsbogen. Ziel der Datenerhebung ist die wissenschaftliche Analyse des *Nutzungsverhaltens* sowie der lernbezogenen Indikatoren im Kontext der Forschungsfragen dieser Arbeit. Die Teilnahme an der Untersuchung war zu jedem Zeitpunkt freiwillig. Ein Abbruch der Studie war ohne Angabe von Gründen und ohne Nachteile für die Teilnehmenden möglich.

5.4 Ablauf der Studie

Der Ablauf der Teilnahme ist vollständig im System abgebildet und folgt einem standardisierten Ablauf:

1. **Login:** Teilnehmende melden sich mit selbstgewähltem Benutzernamen und zentralem Studien-Passwort an.
2. **Gruppenzuweisung und Onboarding:** Das System weist die Gruppe zu und kommuniziert diese im Onboarding. Im Onboarding werden die Bedienoberfläche (Editor, Aufgabenpanel, Konsole) sowie Drill Tasks erläutert. In Gruppe B erfolgt zusätzlich die Personalisierung des KI-Tutors über ein kleines Intro des KI-Tutors, welcher sich vorstellt und das Erfahrungslevel des Lernenden erfragt.
3. **Lernphase (Learning Tasks):** Bearbeitung der Kursaufgaben in fester, linearer Reihenfolge. Jede Aufgabe enthält Startercode, Aufgabenbeschreibung, Hinweise und in den meisten Fällen Worked Examples.
4. **Übungsphase (Part-task Practice):** Nach jeder erfolgreich gelösten Aufgabe folgt obligatorisch eine Drill Task (MCQ + Mini-Code). Drill Tasks sind nicht überspringbar. Beide Teile müssen abgeschlossen werden.
5. **Abschluss:** Nach Abschluss des Pflichtkurses (BMI) wird ein Evaluationsbogen angezeigt. Teilnehmende haben die Möglichkeit, den Evaluationsbogen auszufüllen und den Kurs zu beenden oder vorher noch den optionalen Zinsrechner Aufbaukurs zu bearbeiten.

5.4.1 Lernaufgaben (Learning Tasks)

Als Learning Tasks werden zwei projektbasierte Kurse eingesetzt (didaktische Begründung und Aufbau vgl. Abschnitt 3.2.1):

- **Pflichtkurs (BMI-Rechner):** 12 Steps, strikt lineare Bearbeitung mit additivem Startercode als Scaffolding.
- **Aufbaukurs (Zinsrechner, optional):** 8 Steps, ebenfalls strikt linear mit additivem Startercode; freigeschaltet nach Abschluss des Pflichtkurses.

Für den Gruppenvergleich wird ausschließlich der Pflichtkurs herangezogen, da nur er für alle Teilnehmenden obligatorisch war und damit eine vollständige Datenbasis liefert.

5.4.2 Drill Tasks (Part-task Practice)

Nach jedem erfolgreich gelösten Kurs-Step folgt obligatorisch eine Drill Task als Part-task Practice. Eine Drill Task besteht aus (i) einer Multiple-Choice-Frage und (ii) einer kurzen Mini-Code-Aufgabe. Beide Teile müssen abgeschlossen werden, wobei die Versuche nicht begrenzt sind. Die Drill Tasks entstammen einem statischen, redaktionell erstellten und geprüften Pool (278 Drills). Die Drills sind thematisch in sieben Bereiche gegliedert: Textausgabe, Variablen, Datentypen, Zeichenketten, Listen, Schleifen und bedingte Anweisungen. Die Inhalte sind zwischen den Gruppen identisch; der Unterschied liegt ausschließlich in der Auswahlmethode (vgl. Abschnitt 5.1.1 sowie Abschnitt 3.2.2 für die detaillierte Beschreibung beider Auswahlverfahren). Während des gesamten Ablaufs werden relevante Interaktionen als Events in der Datenbank protokolliert (vgl. Abschnitt 5.5.1).

5.5 Erhobene Daten und Messinstrumente

Im Folgenden werden die in der Plattform erhobenen Datenquellen sowie die verwendeten Messinstrumente beschrieben.

5.5.1 Log-Daten

Die Plattform nutzt eine Event-Sourcing-Architektur¹, in der Interaktionen der Nutzenden als chronologische Ereignisse mit Zeitstempeln gespeichert werden. Zusätzlich wird ein persistentes Lernprofil mit aggregierten Kennzahlen geführt.

¹Event Sourcing ist ein Architekturmuster, bei dem Zustandsänderungen nicht als aktueller Zustand, sondern als unveränderliche Folge von Ereignissen (Events) gespeichert werden. Der aktuelle Zustand ergibt sich durch sequentielle Auswertung aller Ereignisse.

Die erhobenen Daten sind auf drei Granularitätsebenen organisiert, die unterschiedliche Auswertungsperspektiven ermöglichen. Auf *Participant-Level* werden pro Person aggregierte Kennzahlen geführt (z. B. Bearbeitungszeit, Fehlerprofile, Drill-Erfolgsrate), die einen direkten Gruppenvergleich zwischen A und B erlauben. Auf *Task-Level* werden dieselben Metriken pro Kurs-Schritt aufgezeichnet, sodass Schwierigkeitsverläufe und aufgabenspezifische Unterschiede sichtbar werden. Die *Event-Timeline* schließlich liefert die feingranulare Rohdatenbasis (Session-Events, Code-Ausführungen, Drill- und Chat-Events), aus der beide übergeordneten Ebenen rekonstruiert werden können und die eine nachträgliche Überprüfung einzelner Metriken ermöglicht. Für Gruppe B existieren zusätzlich separate Exporte für Chat-Nachrichten (inkl. Zeitstempel und Hint-Level-Zustand) sowie die vom Modell generierten Begründungen der Drill-Auswahl, die zu Qualitätssicherungszwecken gespeichert, den Lernenden jedoch nicht angezeigt werden.

5.5.2 Fragebogen

Nach Abschluss des Pflichtkurses wird ein finaler Evaluationsbogen angezeigt. Der vollständige Fragebogen ist im Anhang dokumentiert (vgl. Anhang A). Er gliedert sich in folgende Abschnitte:

- **Demografie und Selbsteinschätzung:** Dieser Abschnitt erhebt allgemeine Angaben zur Vorerfahrung mit Programmierung sowie die subjektive Einschätzung des eigenen Wissensstandes. Die Angaben dienen der Charakterisierung der Stichprobe und der Einordnung möglicher Gruppenunterschiede.
- **Cognitive Load (CL):** Zwei Items erfassen die wahrgenommene mentale Anstrengung sowie das erlebte Frustrationsniveau während der Bearbeitung. Damit soll eingeschätzt werden, ob die KI-Funktionen die subjektive Belastung der Lernenden beeinflusst haben.
- **System Usability Scale (SUS):** Die SUS [51] besteht aus zehn Items auf einer fünfstufigen Likert-Skala und ergibt einen Gesamtwert zwischen 0 und 100. Sie misst die wahrgenommene Gebrauchstauglichkeit des Systems und ermöglicht einen Vergleich mit etablierten Referenzwerten.
- **User Experience Questionnaire Short (UEQ-S):** Der UEQ-S [52] umfasst acht Items auf einer siebenstufigen bipolaren Skala und erfasst sowohl die pragmatische als auch die hedonische Qualität der Nutzungserfahrung. Er wurde gewählt, weil er gegenüber der Langfassung deutlich kürzer ist und damit die Ausfüllzeit für Teilnehmende gering hält.
- **Net Promoter Score (NPS) und Freitext:** Eine einzelne Frage auf einer Skala von 0 bis 10 erhebt die Weiterempfehlungsbereitschaft. Ein optionales Freitextfeld gibt den Teilnehmenden die Möglichkeit, qualitatives Feedback zu formulieren, das für die Diskussion ergänzend herangezogen wird.

- **KI-spezifischer Abschnitt (nur Gruppe B):** Teilnehmende der Gruppe B bewerten den KI-Chat-Tutor anhand von sechs Items auf einer siebenstufigen Likert-Skala (1 = Stimme überhaupt nicht zu, 7 = Stimme voll zu). Die Items gliedern sich in zwei Subskalen: *Wahrgenommene Nützlichkeit* (drei Items, z. B. „Die KI hat mir geholfen, die Aufgabe schneller zu lösen“) und *Vertrauen und Sicherheit* (drei Items, z. B. „Ich vertraue darauf, dass die Antworten der KI korrekt sind“). Ein Item der Vertrauensskala ist invertiert formuliert und wird vor der Auswertung umgekodiert.

5.6 Datenaufbereitung und Auswertungsregeln

Wie bereits erwähnt, wurden die Rohdaten vor der Auswertung in mehreren Schritten bereinigt. Die folgenden Regeln definieren, welche Datenpunkte in die Analyse eingehen:

- **Ausschluss interner Accounts:** Zur Sicherstellung der Datenintegrität wurden sämtliche während der Entwicklungsphase angelegten internen Accounts manuell aus dem Datensatz entfernt. Die Registrierung dieser Konten erfolgte nicht über die balancierte systemseitige Gruppenzuteilung (vgl. Abschnitt 5.2), sondern durch eine manuelle Zuweisung zu Gruppe B während der Entwicklungsphase. Ihr Ausschluss erklärt, warum Gruppe B von $n_B = 13$ auf $n_B = 11$ zurückgeht, während Gruppe A unverändert bei $n_A = 14$ verbleibt. Nach diesem Schritt umfasst der bereinigte Datensatz $N = 25$ Teilnehmende (Gruppe A: $n_A = 14$, Gruppe B: $n_B = 11$).
- **Aktivitätsfilter:** Als Mindestkriterium für eine ernstzunehmende Teilnahme wurde festgelegt, dass Nutzende mehr als zwei Code-Ausführungen vorgenommen haben müssen ($\text{num_runs_code} > 2$). Personen unterhalb dieses Schwellenwerts haben die Plattform zwar aufgerufen, jedoch keine oder kaum aktive Auseinandersetzung mit den Programmieraufgaben gezeigt. Durch diesen Filter wurden 7 Teilnehmende ausgeschlossen (Gruppe A: 5, Gruppe B: 2), sodass die finale Analyse-Stichprobe $N = 18$ Teilnehmende umfasst (Gruppe A: $n_A = 9$, Gruppe B: $n_B = 9$).
- **AFK-Bereinigung:** Um Verzerrungen durch längere Inaktivität (Away From Keyboard) zu minimieren, wurde ein Grenzwert von 60 Minuten festgelegt ($\text{AFK_CUTOFF_MINUTES} = 60$). Da die Kursaufgaben auf eine Bearbeitungszeit von jeweils 5 bis 15 Minuten ausgelegt sind, gilt eine Unterbrechung von über einer Stunde als eindeutiges Inaktivitätssignal. Bearbeitungszeiten einzelner Kurs-Steps, die diesen Wert überschreiten, werden bei der Berechnung zeitbezogener Kennzahlen ausgeschlossen.

- **Rekonstruktion von KI-Anfragen:** Da die direkten Signale für Hilfeanfragen teilweise unvollständig protokolliert wurden, wurde eine indirekte Metrik mittels Reverse-Engineering erstellt. Dabei werden die Zeitpunkte der Chat-Nachrichten dem jeweils zuletzt gestarteten Bearbeitungsschritt zugeordnet. Die resultierende Kennzahl „Chat per Step“ dient als Proxy für die KI-Inanspruchnahme.

Durch diese Regeln wird sichergestellt, dass trotz technischer Limitierungen beim Logging eine konsistente und vergleichbare Datenbasis für beide Versuchsgruppen vorliegt.

5.7 Auswertungsstrategie

Die Auswertung folgt einem primär deskriptiven, explorativen Ansatz. Gründe hierfür sind (i) die kleine Stichprobe, (ii) die Quasi-Randomisierung sowie (iii) der Verzicht auf einen standardisierten Pre-/Post-Test. Ergebnisse werden daher vorrangig über Verteilungen, Gruppenmittelwerte/-mediane und geeignete Visualisierungen berichtet.

Der Gruppenvergleich (A vs. B) wird für zentrale Logmetriken (z. B. Abschluss/Abbruch, Fehlerprofile pro Step, Zeitmetriken, Drill-Diversität) und für ausgewählte Post-Evaluationsitems durchgeführt. KI-spezifische Kennzahlen werden ausschließlich innerhalb von Gruppe B ausgewertet (z. B. Chat-Nutzung pro nutzender Person, Zeit bis zur ersten KI-Anfrage). Sofern dies zur Interpretation der Daten beiträgt, werden AFK-bereinigte Zeitmetriken ergänzend zu den Rohwerten berichtet, um die Sensitivität der Ergebnisse gegenüber Ausreißern transparent zu machen.

6 Ergebnisse

Im folgenden Kapitel werden die Ergebnisse der empirischen Untersuchung im Hinblick auf die zuvor in Abschnitt 5.1.5 hergeleiteten Hypothesen dargelegt und geprüft. Es werden ausschließlich deskriptive Befunde dargestellt. Die Interpretation und Einordnung in den Forschungsstand erfolgen in Kapitel 7.

6.1 Datenbasis und Auswertungsregeln

Die Ergebnisse basieren auf exportierten Log- und Fragebogendaten aus der Lernplattform. Für zentrale Kennzahlen wurden im Analyse-Workflow vordefinierte Filter- und Robustheitsregeln angewendet. Die im Folgenden berichteten Kennzahlen beziehen sich, sofern nicht anders angegeben, auf den gefilterten Analyse-Datensatz. Die vollständige Stichprobenbeschreibung und die Filterkriterien sind in Kapitel 5 dokumentiert.

- **Analyse-Stichprobe:** $N = 18$ Teilnehmende ($n = 9$ in Gruppe A, $n = 9$ in Gruppe B).
- **Post-Evaluation (Fragebogen):** $N = 16$ abgegebene Fragebögen ($n = 8$ je Gruppe).
- **AFK-Regel:** Step-Zeiten von mehr als 60 Minuten werden als *Away-from-Keyboard-Verdacht*¹ markiert und aus den bereinigten Zeitmetriken ausgeschlossen.
- **Fehlerbalken:** In allen Abbildungen entsprechen Fehlerbalken dem *Standardfehler des Mittelwerts* (SEM, *Standard Error of the Mean*).

6.2 Demografische Merkmale der Stichprobe

Abbildung 1 zeigt die Altersverteilung beider Gruppen für die $N = 16$ Teilnehmenden, die den abschließenden Fragebogen ausgefüllt haben. Die Gruppen unterscheiden sich demografisch deutlich: Gruppe A weist einen Altersmittelwert von 36,1 Jahren (Mdn = 34,5 Jahre) auf und ist über mehrere Altersgruppen gleichmäßig verteilt, mit einem Schwerpunkt bei den 45- bis 54-Jährigen. Gruppe B ist deutlich jünger mit einem Altersmittelwert von 25,1 Jahren (Mdn = 21,0 Jahre). Sie konzentriert sich zu mehr als 60 % auf die Altersgruppe 18 bis 24 Jahre. Damit zeigt die Stichprobe hinsichtlich des Alters eine deutliche Diskrepanz zwischen den beiden Versuchsgruppen.

Abbildung 2 zeigt die Selbsteinschätzung der Programmierkenntnisse auf einer Skala von 1 (keine Kenntnisse) bis 5 (professionelle Programmierkenntnisse). Gruppe A erzielte einen Mittelwert von 3,0 (Mdn = 2,5) und weist eine sehr breite Streuung auf, da zwei Teilnehmende sich mit dem

¹ *Away from Keyboard* (AFK): Pausen ohne Bildschirmaktivität, welche die gemessene Bearbeitungszeit künstlich erhöhen.

Höchstwert 5 einstufen. Gruppe B erreichte einen Mittelwert von 2,6 (Mdn = 3,0) bei einer kompakteren Verteilung ohne Extremwerte. Damit weist Gruppe A eine deutlich breitere Streuung der Programmierkenntnisse auf als Gruppe B.

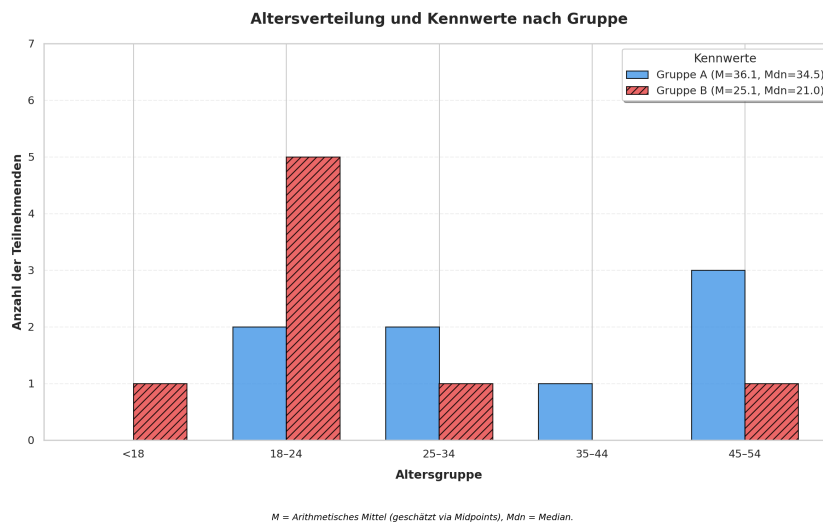


Abbildung 1: Altersverteilung nach Gruppe (Post-Fragebogen, $n = 8$ je Gruppe).
Gruppe A: $M = 36,1$, $Mdn = 34,5$. Gruppe B: $M = 25,1$, $Mdn = 21,0$.

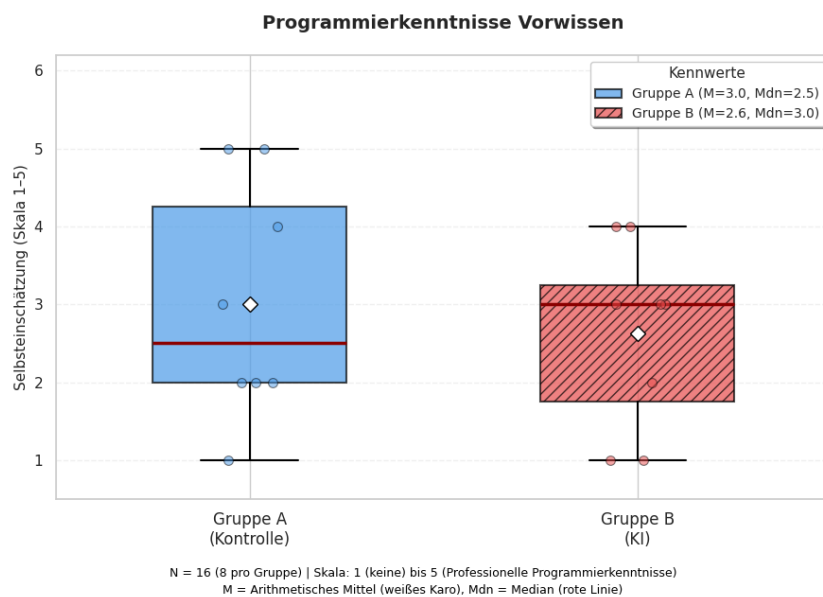


Abbildung 2: Selbsteinschätzung der Programmierkenntnisse nach Gruppe.
Gruppe A: $M = 3,0$, $Mdn = 2,5$. Gruppe B: $M = 2,6$, $Mdn = 3,0$.
(Post-Fragebogen, Skala: 1 = keine Kenntnisse, 5 = professionelle Kenntnisse)

6.3 Teilnahme, Abschluss und Abbruch

In der gefilterten Analyse-Stichprobe schlossen in beiden Gruppen jeweils acht von neun Teilnehmenden den Pflichtkurs (*BMI-Rechner*) erfolgreich ab. Die Abbruchquote ist mit 11,1 % ($n = 1$) pro Gruppe identisch. Aufgrund dieser minimalen Fallzahl an Abbrüchen sind keine statistisch belastbaren Rückschlüsse auf die Wirksamkeit der KI-Funktionen hinsichtlich des Kursverbleibs möglich. Der Befund wird daher als rein deskriptiver Basiswert für die weiteren Analysen festgehalten.

Die Bearbeitungszeit bis zum Abbruch unterscheidet sich zwischen den beiden Abbruchfällen erheblich: Die abbrechende Person in Gruppe A verließ den Kurs nach 2,13 Minuten, die abbrechende Person in Gruppe B erst nach 45,27 Minuten. Da aus jeder Gruppe nur eine Person abgebrochen hat, ist ein Gruppenvergleich dieser Kennzahl statistisch nicht sinnvoll. Die Werte werden der Vollständigkeit halber berichtet.

6.4 Bearbeitungszeit im BMI-Kurs

Die Gesamtbearbeitungszeit des BMI-Kurses unterscheidet sich zwischen den Gruppen deutlich. Abbildung 3 zeigt die AFK-bereinigte Gesamtdauer pro Teilnehmenden ($N = 17$, ein Ausreißer über 60 Minuten gefiltert). Gruppe A wies einen Mittelwert von 19,5 Minuten auf ($Mdn = 18,7$ min), während Gruppe B einen Mittelwert von 32,9 Minuten erreichte ($Mdn = 32,4$ min). Das entspricht einer Mehrzeit von durchschnittlich 13,4 Minuten in Gruppe B, was rund 68 % länger ist als in Gruppe A. Die Box von Gruppe B liegt nahezu vollständig oberhalb der Box von Gruppe A. Selbst die schnellste teilnehmende Person in Gruppe B benötigt eine vergleichbare Zeit wie der Medianwert in Gruppe A.

Abbildung 4 zeigt die schrittweise Bearbeitungszeit für alle 12 Kursabschnitte (*Steps*) nach Gruppe ($N = 18$). In den frühen Steps (1 bis 4) liegen beide Gruppen nah beieinander. Ab Step 5, der die Einführung von Variablen markiert, steigt die Bearbeitungszeit in Gruppe B deutlich an und erreicht ihre Peaks in Step 5 (Gruppe B: $M = 5,6$ min vs. Gruppe A: $M = 2,2$ min, Delta: +3,4 min) und Step 6 (Gruppe B: $M = 6,0$ min vs. Gruppe A: $M = 1,8$ min, Delta: +4,2 min). Ab Step 7 nähern sich die Werte wieder an, bleiben in Gruppe B jedoch bis zum letzten Schritt durchgängig höher, außer einer kleinen Ausnahme in Step 9 mit Delta: -0,2 min. Die mittlere Bearbeitungszeit pro Step beträgt für Gruppe A 1,9 Minuten ($Mdn = 1,4$ min) und für Gruppe B 2,9 Minuten ($Mdn = 2,4$ min).

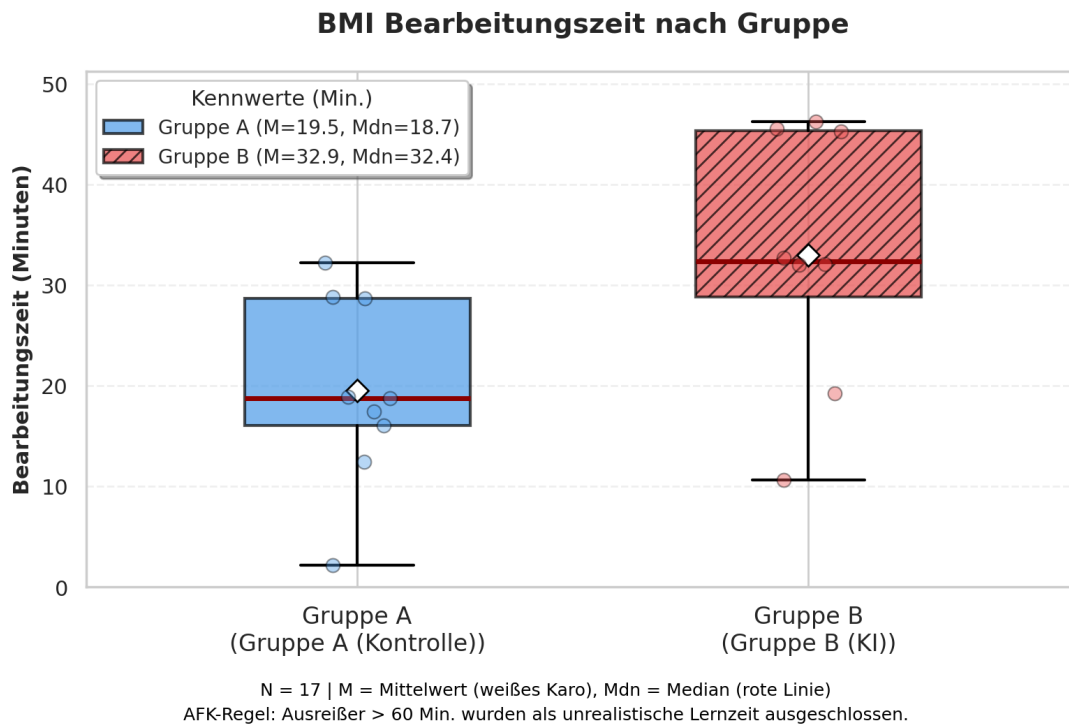


Abbildung 3: Gesamtbearbeitungszeit des BMI-Kurses nach Gruppe (AFK-bereinigt, $N = 17$). Gruppe A: $M = 19,5$ min, $Mdn = 18,7$ min. Gruppe B: $M = 32,9$ min, $Mdn = 32,4$ min.

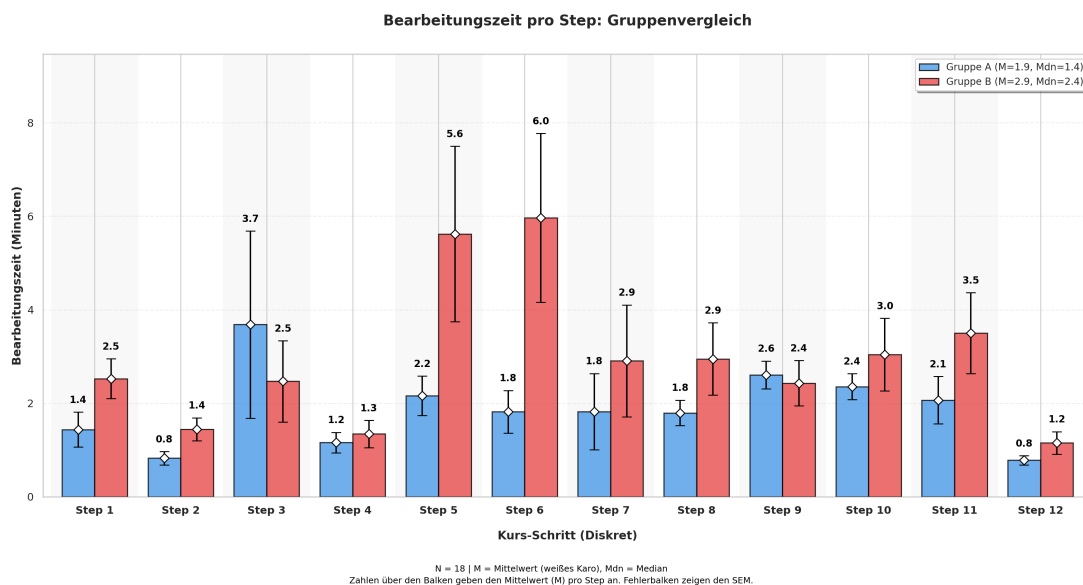


Abbildung 4: Durchschnittliche Bearbeitungszeit pro Step nach Gruppe ($N = 18$, Fehlerbalken: ± 1 SEM). Gruppe A: $M = 1,9$ min. Gruppe B: $M = 2,9$ min. Peaks in Step 5 (+3,4 min) und Step 6 (+4,2 min).

6.5 Fehlerprofile im BMI-Kurs

Neben der Bearbeitungszeit wurden Python-Fehler (Syntaxfehler und Laufzeitfehler) als Leistungsindikator ausgewertet. Abbildung 5 zeigt die kumulierte Fehleranzahl pro teilnehmender Person über den gesamten BMI-Kurs ($N = 18$). Gruppe A erzielte einen Mittelwert von 2,7 Fehlern (Mdn = 2,0), Gruppe B einen Mittelwert von 7,1 Fehlern (Mdn = 5,0). Die *Interquartilsbreite*² in Gruppe B ist erheblich größer; ein einzelner Ausreißer verursachte über 25 Fehler und zieht den Mittelwert nach oben. Auch der Median von 5,0 liegt in Gruppe B deutlich über dem Vergleichswert von 2,0 in Gruppe A.

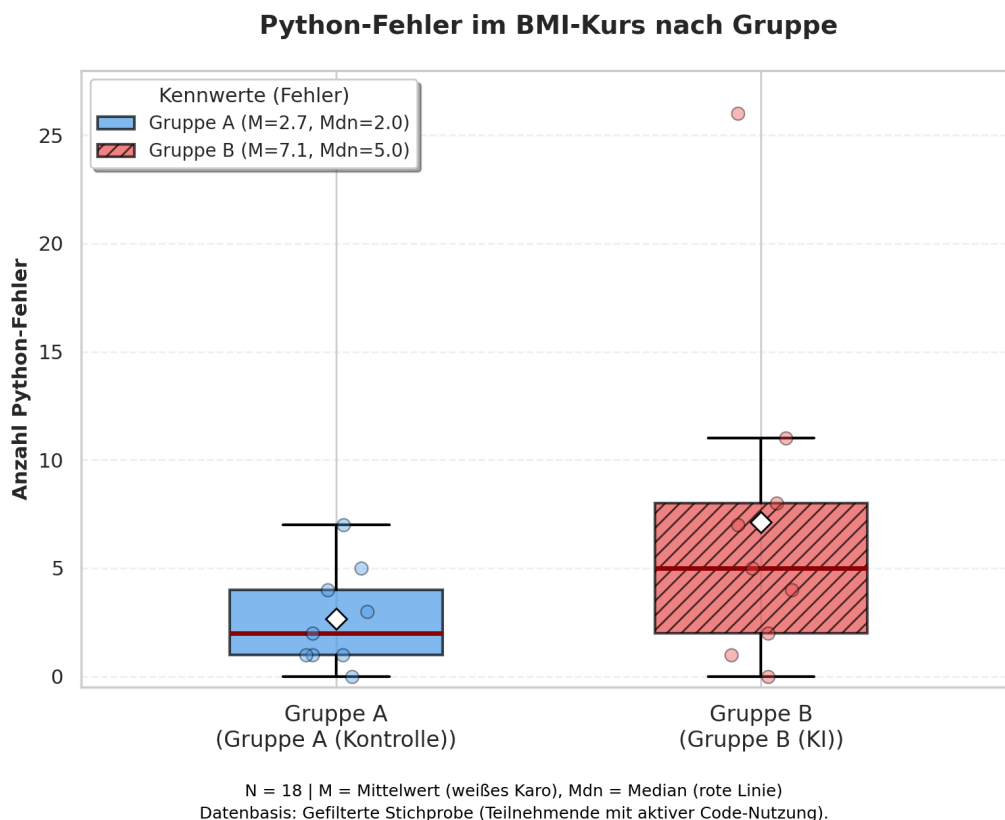


Abbildung 5: Gesamtanzahl Python-Fehler im BMI-Kurs nach Gruppe ($N = 18$, Datenbasis: Teilnehmende mit aktiver Code-Nutzung). Gruppe A: $M = 2,7$, $Mdn = 2,0$. Gruppe B: $M = 7,1$, $Mdn = 5,0$.

Abbildung 6 stellt die durchschnittliche Fehleranzahl pro Step im Zeitverlauf gegenüber. Auffällig ist ein markanter Peak in Gruppe B bei Step 1 ($M = 2,5$ vs. $M = 0,3$ in Gruppe A), obwohl dieser Step lediglich eine einfache Textausgabe mit `print` verlangt. Ab Step 5 steigen die Fehler in beiden Gruppen mit der zunehmenden Aufgabenkomplexität (Variablen, Arithmetik, Bedingungen) an. In Step 8, dem Einstieg in bedingte Anweisungen, liegen die Gruppen mit $M = 0,8$ und $M = 0,9$ nahezu gleichauf. Die mittlere Fehleranzahl pro Step beträgt $M = 0,3$ für Gruppe A und $M = 0,6$ für Gruppe B (Median jeweils 0,0).

²IQR (*Interquartile Range*): Abstand zwischen dem 25. und 75. Perzentil, umfasst die mittleren 50 % der Datenpunkte.

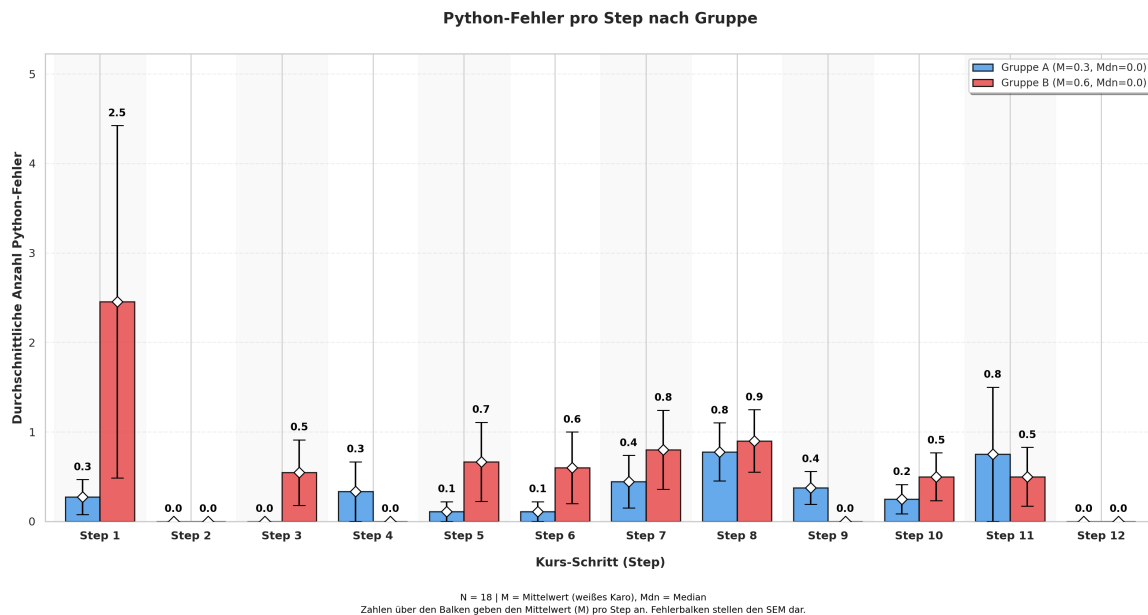


Abbildung 6: *Durchschnittliche Python-Fehleranzahl pro Step nach Gruppe* ($N = 18$, Fehlerbalken: ± 1 SEM). Gruppe A: $M = 0,3$. Gruppe B: $M = 0,6$. Markanter Peak in Gruppe B bei Step 1 ($M = 2,5$).

6.6 Drill-Auswahl und Leistung

Gruppe A nutzte eine gewichtete Zufallsauswahl, Gruppe B eine KI-gestützte adaptive Auswahl. Abbildung 7 zeigt zwei Kennzahlen zur Drill-Vielfalt: Gruppe A bearbeitete 97 unterschiedliche Drill-IDs, Gruppe B lediglich 62.

Die *Shannon-Entropie* [53] als Maß für die Gleichverteilung über den Drill-Pool³ beträgt in Gruppe A 4,4 Bit gegenüber 3,8 Bit in Gruppe B, was auf eine stärkere Wiederholung bestimmter Aufgaben durch die KI-Auswahl hindeutet.

Abbildung 8 zeigt die Verteilung der benötigten Versuche zur Lösung der Drills, basierend auf 433 erfolgreich gelösten Drills beider Gruppen. Die überwiegende Mehrheit der Aufgaben wurde im ersten Versuch gelöst, in Gruppe A zu 86,0 % und in Gruppe B zu 83,6 %. Bei Aufgaben, die vier oder mehr Versuche erforderten, weist Gruppe B mit 5,8 % gegenüber 2,4 % in Gruppe A einen deutlich höheren Anteil auf.

6.7 KI-Nutzung in Gruppe B

Für Gruppe B liegen Daten zur Nutzung des KI-Chat-Tutors vor. Gruppe A hatte keinen Zugang zu dieser Funktion, weshalb Chat-Metriken ausschließlich für Gruppe B berichtet werden. Im Mittel wurden pro Person 6,2 Nachrichten gesendet (Mdn = 6,0, SEM: $\pm 1,1$, $n = 9$). Die enge Übereinstimmung von Mittelwert und Median deutet auf eine relativ gleichmäßige Nutzung ohne extreme Ausreißer hin.

³Berechnet als $H = -\sum p_i \log_2 p_i$; höhere Werte bedeuten gleichmäßigere Verteilung über den Aufgabenpool.

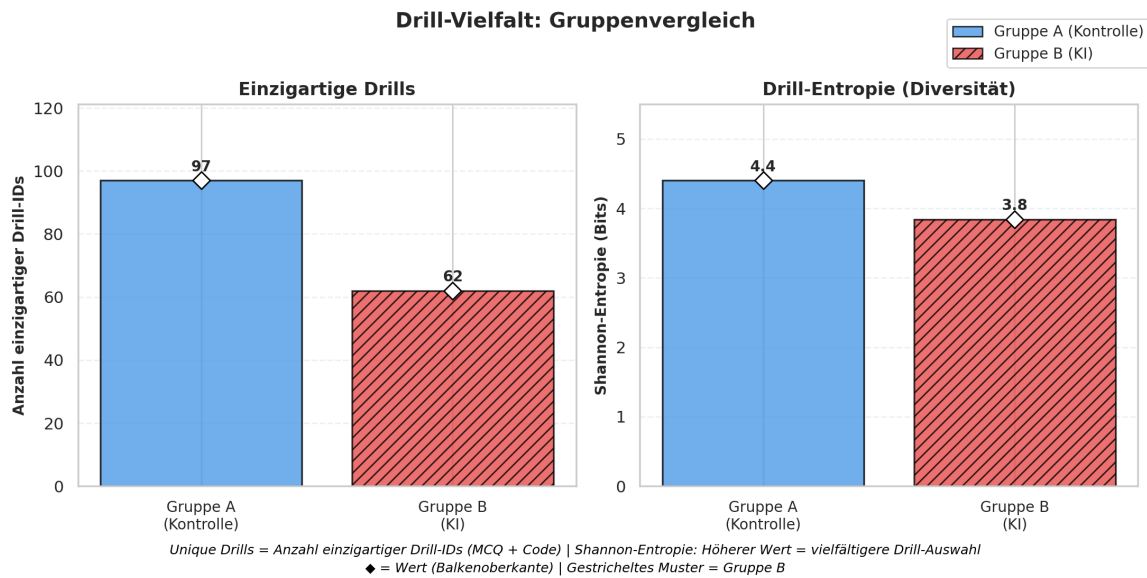
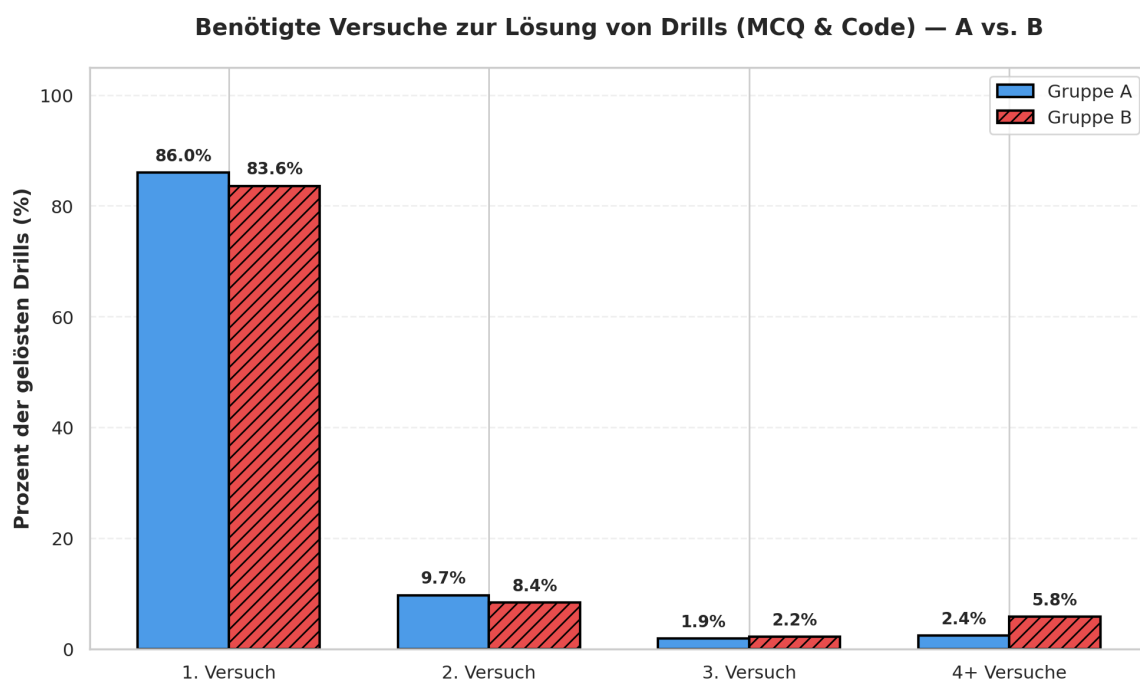


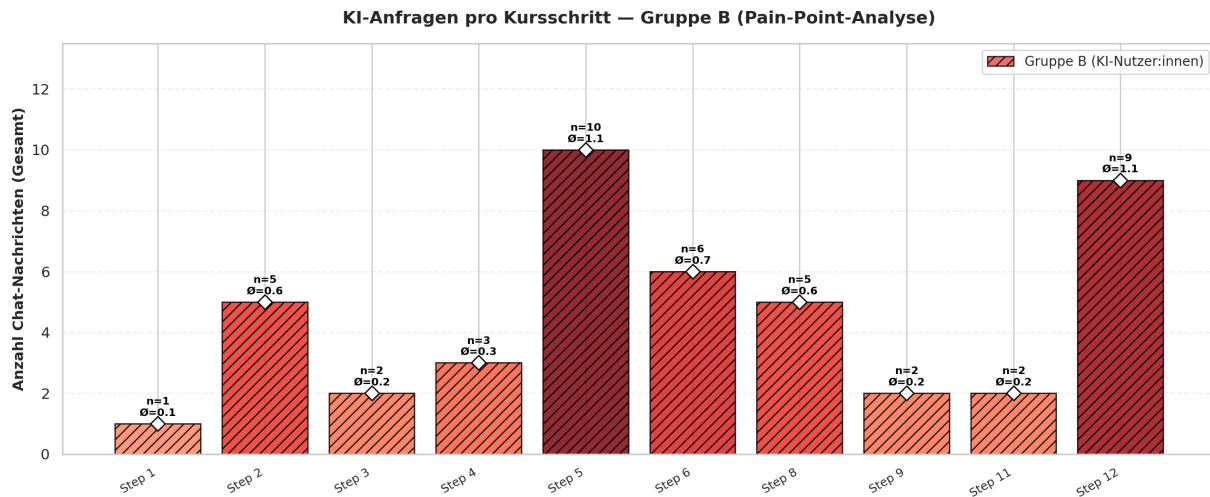
Abbildung 7: Drill-Vielfalt nach Gruppe: Anzahl einzigartiger Drill-IDs (links) und Shannon-Entropie der Aufgabenverteilung in Bit (rechts). Gruppe A: 97 einzigartige Drills, Entropie 4,4. Gruppe B: 62 einzigartige Drills, Entropie 3,8.



Basierend auf 433 erfolgreich gelösten Drills (MCQ & Code kombiniert).
Ein Versuch entspricht einmaligem Code-Ausführen bzw. Antwort-Prüfen bis zur ersten korrekten Lösung.

Abbildung 8: Anteil der Drills nach benötigter Versuchsanzahl (Basis: 433 erfolgreich gelöste Drills, MCQ (Multiple-Choice-Fragen) und Code kombiniert). Gruppe A: 86,0 % im 1. Versuch, 2,4 % bei 4+ Versuchen. Gruppe B: 83,6 % im 1. Versuch, 5,8 % bei 4+ Versuchen.

Abbildung 9 zeigt die Verteilung der KI-Anfragen über die einzelnen Kursschritte als *Pain-Point-Analyse*⁴. Die Chat-Nutzung ist nicht gleichmäßig über den Kurs verteilt, sondern konzentriert sich auf spezifische Steps. Step 5 weist mit insgesamt 10 Nachrichten ($\bar{x}$ 1,1 pro Person) den höchsten Wert auf und entspricht jenem Kursabschnitt, in dem Variablen und Arithmetik neu eingeführt werden. Step 12 folgt mit 9 Nachrichten ($\bar{x}$ 1,1), obwohl dieser Schritt kein neues Konzept einführt. In den ersten vier Schritten ist die KI-Nutzung mit insgesamt 11 Nachrichten gering.



Nur Gruppe B | n = Gesamtanzahl Chat-Nachrichten je Step, Ø = Durchschnitt pro Teilnehmer:in
Zuordnung via Zeitfenster (step_started_at ≤ timestamp ≤ step_completed_at) | ♦ = Balkenhöhe (weißes Karo) | Hohe Balken = Kursschritte mit viel KI-Bedarf

Abbildung 9: KI-Anfragen pro Kursschritt in Gruppe B (Pain-Point-Analyse). n = Gesamtanzahl Chat-Nachrichten je Step, Ø = Durchschnitt pro Teilnehmenden. Peaks in Step 5 (n=10, Ø=1,1) und Step 12 (n=9, Ø=1,1).

Abbildung 10 zeigt den Zusammenhang zwischen der Anzahl gesendeter Chat-Nachrichten und der Gesamtzahl Python-Fehler innerhalb von Gruppe B. Der Pearson-Korrelationskoeffizient beträgt $r = 0,59$ ($p = 0,096$), statistisch nicht signifikant, zeigt jedoch einen deutlichen positiven Trend: Teilnehmende mit mehr Python-Fehlern wandten sich häufiger an den KI-Chat-Tutor.

6.8 Post-Evaluation: Fragebogenauswertung

Der abschließende Evaluationsbogen wurde von $N = 16$ Teilnehmenden ausgefüllt ($n = 8$ je Gruppe). Er erfasst kognitive Belastung, Gebrauchstauglichkeit, Nutzungserfahrung und Weiterempfehlungsbereitschaft. Gruppe B beantwortete zusätzlich einen KI-spezifischen Abschnitt.

⁴Als *Pain Points* werden Stellen im Lernpfad bezeichnet, an denen Lernende überproportional häufig Schwierigkeiten haben. Die Häufung von KI-Anfragen an bestimmten Kursschritten dient hier als Indikator für kognitiv anspruchsvolle Abschnitte.

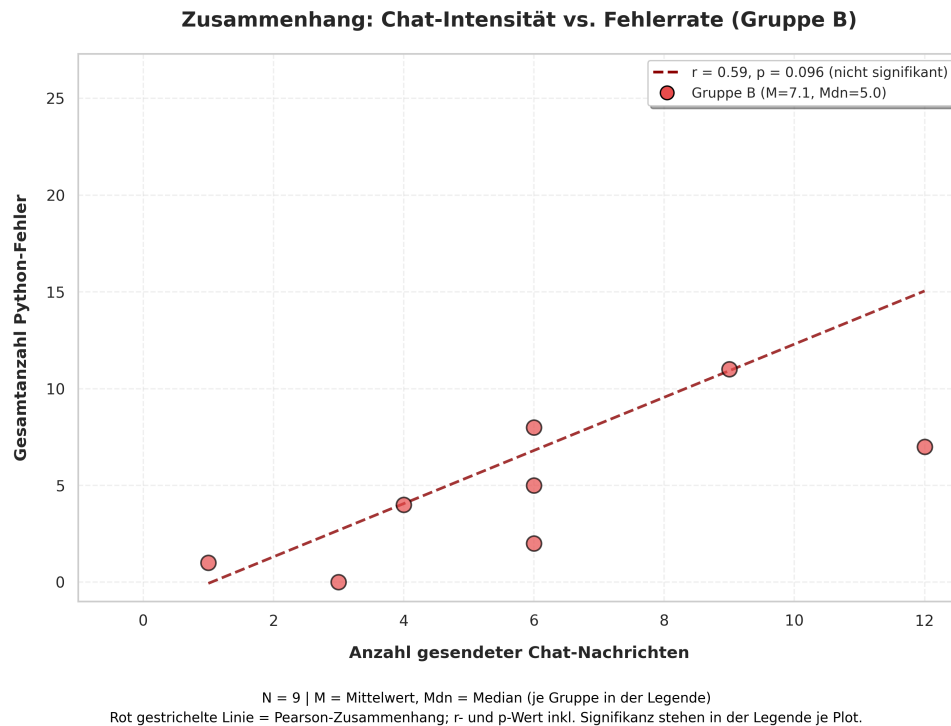


Abbildung 10: Zusammenhang zwischen Chat-Intensität und Python-Fehleranzahl in Gruppe B ($n = 9$).
Pearson $r = 0,59$, $p = 0,096$ (nicht signifikant).

6.8.1 Kognitive Belastung (NASA-TLX)

Abbildung 11 zeigt die Ausprägung der Dimensionen *Mentaler Aufwand* (Skala 1–9) und *Frustration* (Skala 0–100) auf den entsprechenden NASA-TLX-Subskalen [54]. Die Ergebnisse zeigen ein charakteristisches Überkreuz-Muster: Gruppe B berichtet einen höheren mentalen Aufwand ($M = 3,1$, $Mdn = 2,5$) als Gruppe A ($M = 2,5$, $Mdn = 1,5$), gleichzeitig aber eine niedrigere Frustration (Gruppe B: $M = 14,9$, $Mdn = 6,5$ gegenüber Gruppe A: $M = 16,2$, $Mdn = 10,0$). Der Median der Frustration liegt in Gruppe B damit rund 35 % unterhalb des Vergleichswertes in Gruppe A. Hinsichtlich der Extremwerte zeigt Gruppe A einen Ausreißer beim mentalen Aufwand (Wert: 8,0) sowie einen einzelnen Frustrations-Ausreißer (Wert: 70).

In Gruppe B ist das Frustrations-Maximum auf ca. 61 begrenzt, während die überwiegende Mehrheit der Teilnehmenden Frustrationswerte unter 10 aufweist.

6.8.2 Gebrauchstauglichkeit (SUS) und Weiterempfehlung (NPS)

Abbildung 12 stellt die *System Usability Scale* (SUS) und den *Net Promoter Score* [55] (NPS) im Gruppenvergleich gegenüber. Beide Gruppen erzielten SUS-Werte deutlich oberhalb des Benchmarks von 68 Punkten für gute Gebrauchstauglichkeit. Gruppe A erreichte $M = 85,0$ ($Mdn = 88,8$), Gruppe B $M = 85,9$ ($Mdn = 90,0$). Die Adjektiv-Bewertung nach Bangor et al. [56] ordnet beide Gruppen in die Kategorie *Good* ein.

Beim NPS (Skala 0–10) übertrifft Gruppe B mit einem Mittelwert von 8,5 ($Mdn = 9,0$) die Kontroll-

Gruppe A mit $M = 7,5$ ($Mdn = 8,5$). Der Median von 9,0 in Gruppe B liegt im *Promoter*-Bereich (Werte 9–10).⁵ In Gruppe A gab es zwei *Detraktoren* (Wert 1), während Gruppe B fünf *Promoter* mit einem Score von 10 aufweist.

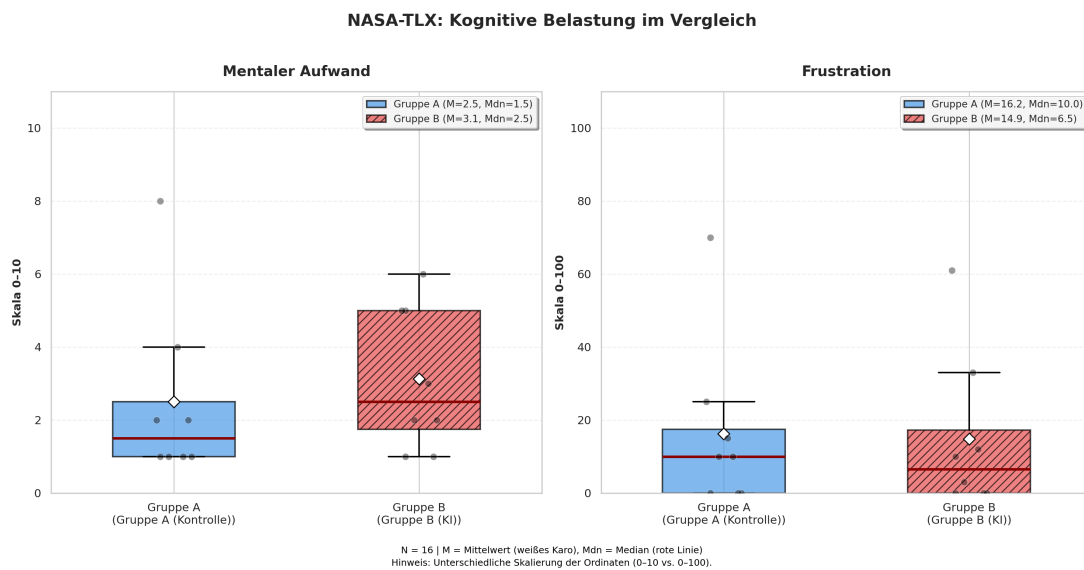


Abbildung 11: NASA-TLX: Mentaler Aufwand (Skala 1–9, links) und Frustration (Skala 0–100, rechts) nach Gruppe ($N = 16$). Die y-Achsen beider Teilgrafiken sind unterschiedlich skaliert.

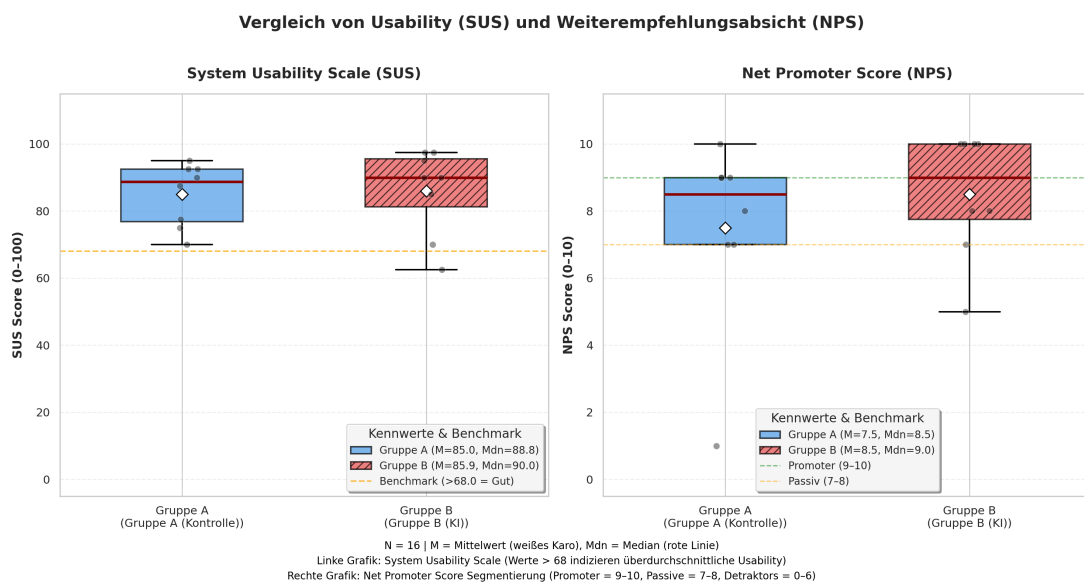


Abbildung 12: SUS-Scores (Skala 0–100, links) und NPS-Scores (Skala 0–10, rechts) nach Gruppe ($N = 16$). Gestrichelte Linien markieren die Benchmarks: $SUS \geq 68$ (gelb) sowie NPS-Schwellen für Passive (7) und Promoter (9).

⁵Der NPS teilt Befragte in drei Kategorien ein: *Promoter* (Score 9–10), *Passive* (Score 7–8) und *Detraktoren* (Score 0–6). Der NPS-Wert ergibt sich aus der Differenz von Promoter-Anteil und Detraktoren-Anteil in Prozentpunkten.

6.8.3 User Experience Questionnaire Short (UEQ-S)

Der UEQ-S [52] erfasst die Nutzungserfahrung anhand von acht Item-Paaren auf einer Skala von 1 bis 7. Abbildung 13 zeigt das Item-Profil beider Gruppen im Vergleich. Gruppe B erreicht über alle Items hinweg einen höheren Gesamtmittelwert von $M = 5,9$ gegenüber $M = 5,2$ in Gruppe A. Die stärksten Unterschiede zeigen sich bei den hedonischen Items: Das Attributpaar *uninteressant* / *interessant* fällt in Gruppe B um 1,3 Punkte höher aus (6,2 vs. 4,9), das Paar *herkömmlich* / *neuartig* um 1,2 Punkte (6,1 vs. 4,9). Bei den pragmatischen Items (*behindernd* / *unterstützend* und *kompliziert* / *einfach*) sind die Gruppenunterschiede geringer.

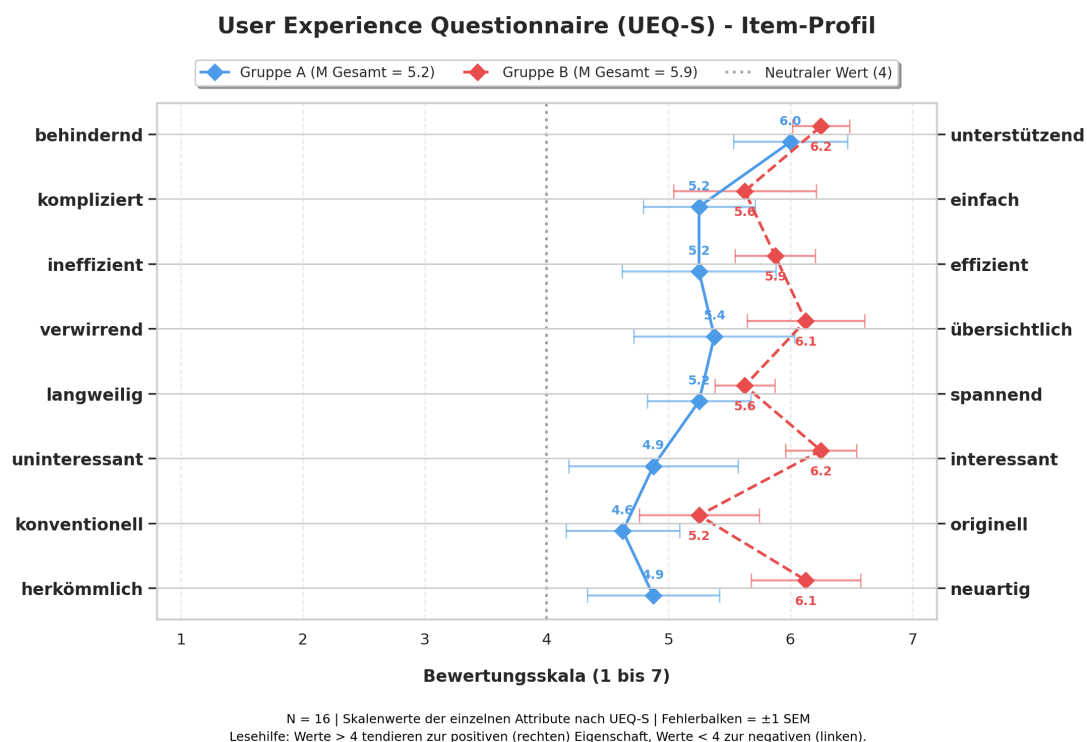


Abbildung 13: UEQ-S Item-Profil nach Gruppe ($N = 16$, Fehlerbalken: ± 1 SEM). Gruppe A: M Gesamt=5,2. Gruppe B: M Gesamt=5,9. Größte Differenzen bei den Items *interessant* (+1,3) und *neuartig* (+1,2).

6.8.4 KI-spezifischer Fragebogen (nur Gruppe B)

Teilnehmende in Gruppe B beurteilten zusätzlich sechs Items zur KI-Wahrnehmung auf einer *Likert-Skala*⁶ von 1 (trifft gar nicht zu) bis 7 (trifft voll zu). Abbildung 14 zeigt die Verteilung über alle sechs Items.

Die positivsten Bewertungen erzielten die Items zur Wiederverwendungsbereitschaft ($M = 5,38$, $Mdn = 6,5$), zum Vertrauen in die Antworten ($M = 5,38$, $Mdn = 5,5$) sowie zur Erklärungsqualität ($M = 5,25$, $Mdn = 6,0$). Auch die Hilfreichkeit bei der Fehlersuche liegt mit einem Mittelwert von 5,12

⁶Eine *Likert-Skala* ist ein psychometrisches Format, bei dem Befragte ihre Zustimmung oder Ablehnung zu einer Aussage auf einer mehrstufigen Ordinalskala angeben.

(Mdn = 5,5) deutlich im positiven Bereich. Im Gegensatz dazu wurden die allgemeine Hilfreichkeit ($M = 3,62$, Mdn = 4,0) und die fachliche Korrektheit der KI-Antworten ($M = 3,50$, Mdn = 3,0) kritischer beurteilt; beide Items liegen im Mittel nahe oder unterhalb des Neutralpunkts von 4,0.

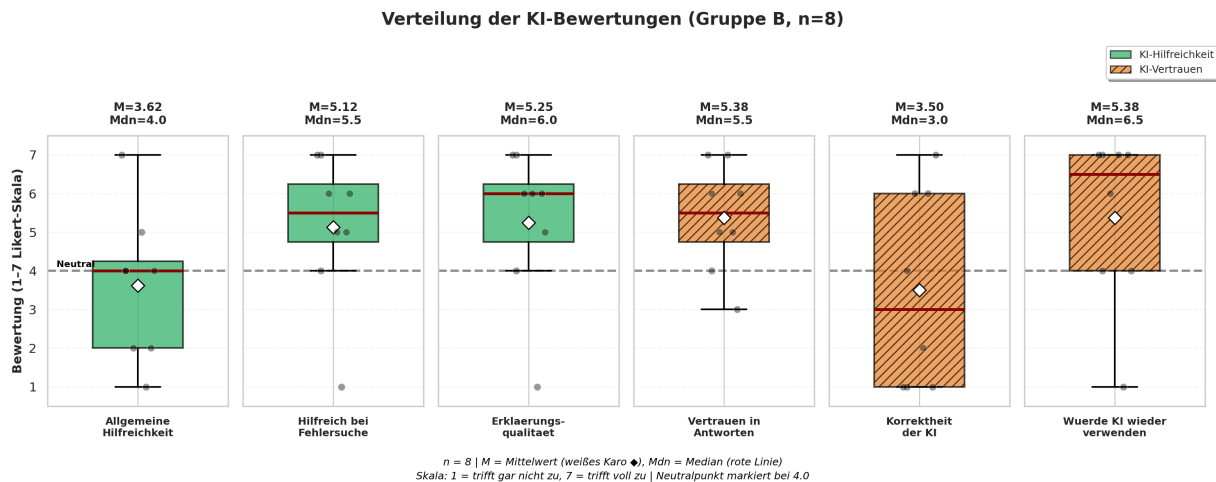


Abbildung 14: Bewertung der KI-spezifischen Items in Gruppe B ($n = 8$, Skala: 1–7). Die gestrichelte graue Linie markiert den Neutralpunkt bei 4,0.

Die grafische Darstellung der Einzelwerte (Jitter-Punkte) offenbart eine hohe Heterogenität innerhalb der Gruppe: Während einzelne Teilnehmende die KI in allen Dimensionen mit dem Maximalwert von 7,0 bewerteten, finden sich bei der Fehlersuche und der Erklärungsqualität deutliche Ausreißer im Minimalbereich von 1,0.

6.9 Korrelationsanalysen

Um Zusammenhänge zwischen der Programmiererfahrung der Teilnehmenden und ihren Leistungs- und Belastungsmetriken zu untersuchen, wurden *Pearson-Korrelationen*⁷ berechnet. Da die Stichprobengröße für alle drei Analysen $N \leq 16$ beträgt, sind die Befunde als explorative deskriptive Kennzahlen zu verstehen.

Abbildung 15 zeigt den Zusammenhang zwischen der Programmiererfahrung (Skala 1–5) und der Drill-Performance. Die *First-Try-Rate*⁸ korreliert schwach positiv mit der Erfahrung ($r = 0,33$, $p = 0,215$, nicht signifikant). Die durchschnittliche Anzahl benötigter Versuche weist hingegen eine starke, statistisch signifikante negative Korrelation auf ($r = -0,63$, $p = 0,010$): Erfahrenere Teilnehmende benötigen deutlich weniger Versuche zur Lösung der Drill Tasks.

Abbildung 16 zeigt den Zusammenhang zwischen Erfahrung und kognitiver Belastung. Frustration wie auch mentaler Aufwand sinken mit steigender Erfahrung stark: Die Korrelation mit Frustration beträgt $r = -0,68$ ($p = 0,004$), die mit mentalem Aufwand $r = -0,73$ ($p = 0,001$).

⁷Der *Pearson-Korrelationskoeffizient* r misst die Stärke und Richtung eines linearen Zusammenhangs zwischen zwei Variablen. Er nimmt Werte zwischen -1 (vollständig negativer Zusammenhang) und $+1$ (vollständig positiver Zusammenhang) an; $r = 0$ bedeutet kein linearer Zusammenhang.

⁸Als *First-Try-Rate* wird der Anteil der Drills bezeichnet, die im ersten Lösungsversuch korrekt gelöst wurden.

Beide Zusammenhänge sind statistisch signifikant. Teilnehmende mit der niedrigsten Erfahrungsstufe (1) erreichen Frustrationswerte von bis zu 70 und mentale Aufwandswerte, die deutlich über dem Gesamtdurchschnitt liegen.

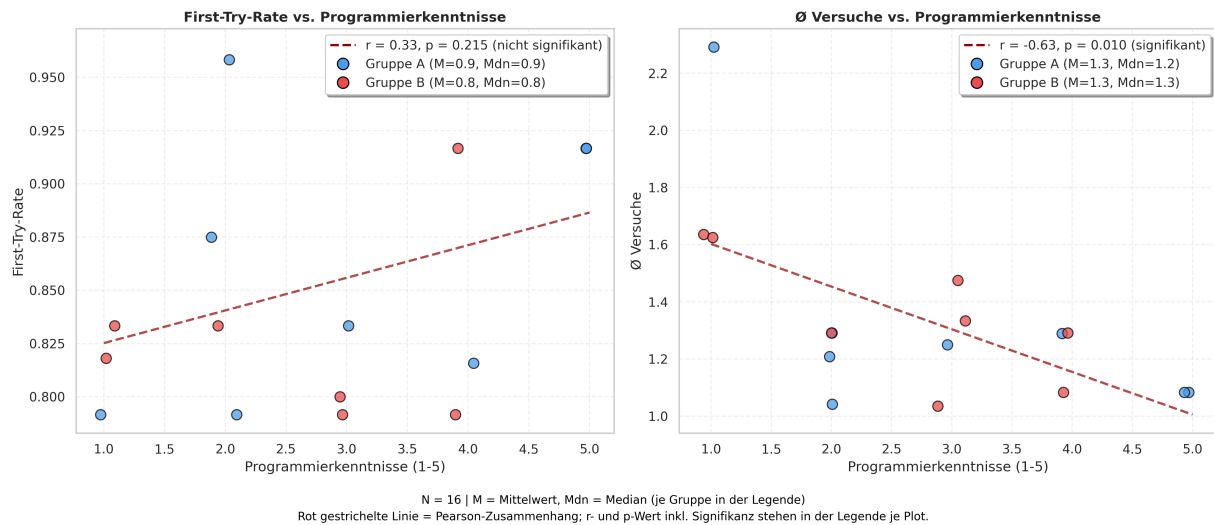


Abbildung 15: Zusammenhang zwischen Programmiererfahrung und Drill-Performance ($N = 16$). Links: First-Try-Rate ($r = 0,33$, $p = 0,215$, n.s.). Rechts: Durchschnittliche Versuche ($r = -0,63$, $p = 0,010$).

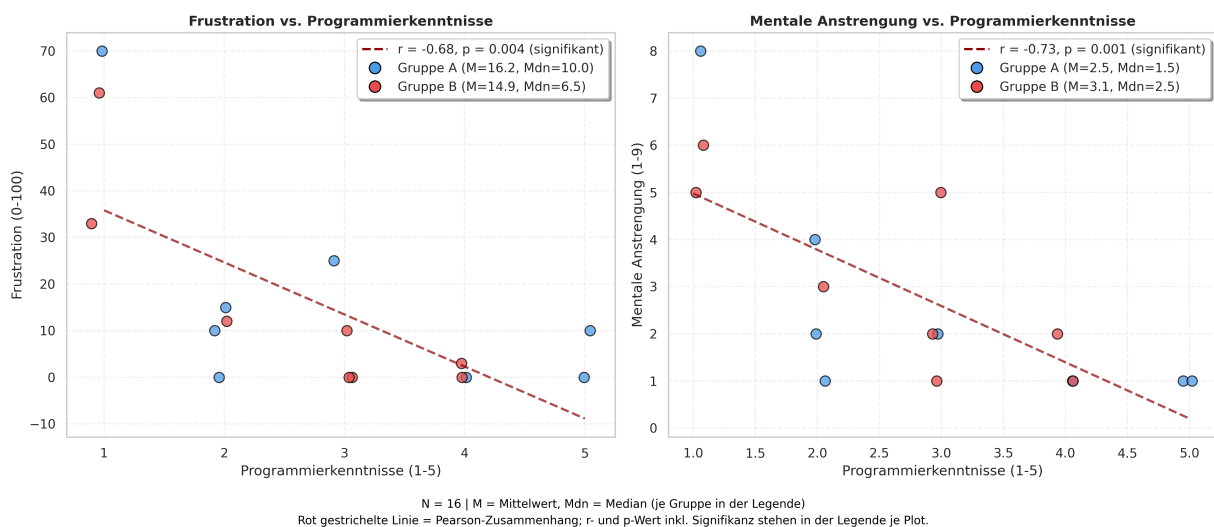


Abbildung 16: Zusammenhang zwischen Programmiererfahrung und kognitiver Belastung ($N = 16$). Links: Frustration ($r = -0,68$, $p = 0,004$). Rechts: Mentaler Aufwand ($r = -0,73$, $p = 0,001$).

Abbildung 17 zeigt den Zusammenhang zwischen Erfahrung sowie Bearbeitungszeit und Fehleranzahl im BMI-Kurs ($N = 16$). Beide Korrelationen sind negativ, jedoch statistisch nicht signifikant: Bearbeitungszeit $r = -0,46$ ($p = 0,071$) und Fehleranzahl $r = -0,50$ ($p = 0,051$). Die Streuung in beiden Scatterplots ist erheblich.

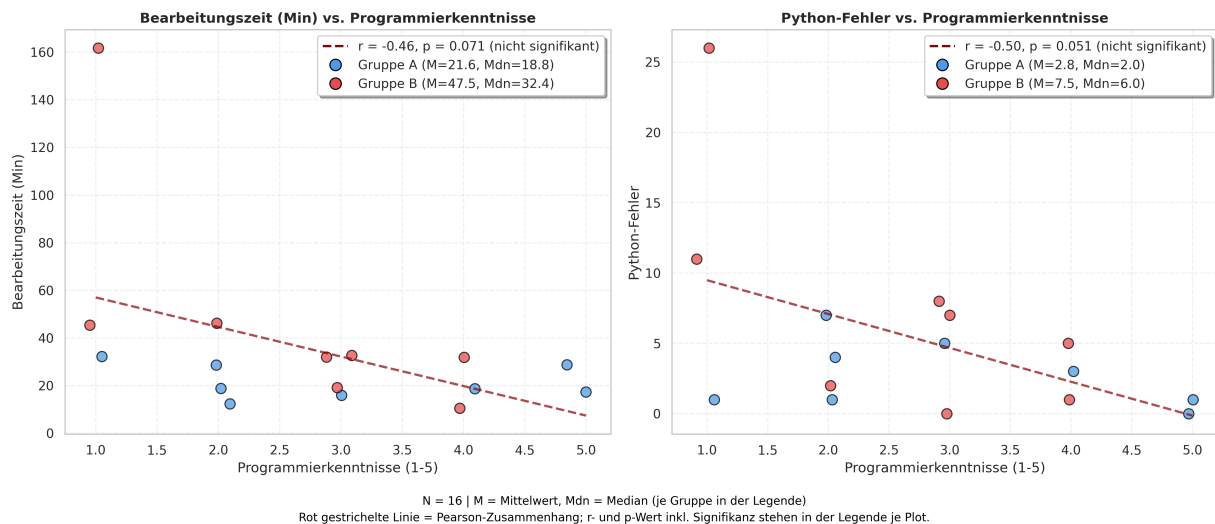


Abbildung 17: Zusammenhang zwischen Programmiererfahrung und Kursleistung ($N = 16$). Links: Bearbeitungszeit ($r = -0,46$, $p = 0,071$, n.s.). Rechts: Gesamtfehler ($r = -0,50$, $p = 0,051$, n.s.).

6.10 Qualitative Ergebnisse (Freitext)

Der Fragebogen enthielt ein offenes Textfeld für freies Feedback. Alle $N = 16$ Teilnehmenden, die den Fragebogen abgaben, nutzten dieses Feld. Die Antworten wurden thematisch kategorisiert. Da keine inhaltliche Auswertung ohne Interpretation möglich ist, werden hier ausschließlich wiederkehrende Themenfelder und exemplarische, anonymisierte Zitate berichtet.

Bedienbarkeit und Kursdesign Mehrere Teilnehmende beider Gruppen kommentierten positiv die schrittweise Führung durch den Kurs sowie die sofortige Rückmeldung nach jeder Code-Ausführung. Aus Gruppe A wurde geschrieben: „Ich fand es gut, dass man durch das System geführt wurde. Am Anfang dachte ich, das schaffe ich nicht, und durch die Unterstützung wurde es Schritt für Schritt leichter.“ Aus Gruppe B hielt eine weitere Person fest: „Die Handhabung und das allgemeine UI ist sehr einfach und übersichtlich gestaltet.“ Kritisch wurde angemerkt, dass die Lösungsvalidierung in einigen Fällen zu streng sei: „Etwas starre Überprüfung der Ergebnisse. Tipps, was genau an der Antwort falsch war, wären schön.“

Drill-Qualität und Aufgabenrelevanz Aus Gruppe A wurde festgestellt, dass die Drill Tasks in einzelnen Fällen nicht eng genug mit dem vorherigen Kursschritt verknüpft schienen: „Die Drill-Tasks haben oft das Thema zur Aufgabe verfehlt, beispielsweise wurde im achten Schritt gefordert, 'Willkommen' mithilfe der print-Funktion auszugeben.“ Aus Gruppe B wurde der Wunsch geäußert, zusätzliche Code-Fehlersuche-Aufgaben als Drill-Format anzubieten.

KI-Tutor (nur Gruppe B) Die Bewertungen des KI-Tutors fielen in Gruppe B heterogen aus. Einzelne Teilnehmende berichteten, der KI-Tutor habe ihnen geholfen, alternative Lösungswege zu erkunden und Rückfragen zur Python-Syntax zu stellen. Andere Teilnehmende empfanden den KI-Tutor als überflüssig, da die Aufgaben ohne Hilfe lösbar waren. Zudem wurde angemerkt, der KI-Tutor habe in einem Fall eine umständliche Antwort gegeben, obwohl ein einfacherer Weg möglich gewesen wäre.

Technische Hinweise Vereinzelt wurden konkrete technische Probleme gemeldet, darunter ein inkorrektes Benachrichtigungs-Interface, das einen Navigationsbutton verdeckte, sowie eine Inkompatibilität mit Safari auf macOS (Chrome funktionierte problemlos).

6.11 Zusammenfassung der Ergebnisse

Die Befunde der Studie lassen sich in vier Bereichen bündeln. In den **Leistungsmetriken** wies Gruppe B mit einer mittleren Bearbeitungszeit von 32,9 Minuten eine um 68 % längere Kursbearbeitungszeit auf als Gruppe A ($M = 19,5$ min) und produzierte dabei im Mittel mehr als doppelt so viele Python-Fehler ($M = 7,1$ vs. $M = 2,7$). Die Abschlussquote war mit je acht von neun Teilnehmenden in beiden Gruppen identisch.

Bei der **Drill-Performance** zeigte die KI-gestützte Auswahl in Gruppe B eine geringere Aufgabenvielfalt (62 vs. 97 einzigartige Drill-IDs, Entropie 3,8 vs. 4,4 Bits) bei gleichzeitig höherem Anteil an Aufgaben mit vier oder mehr Versuchen (5,8 % vs. 2,4 %). In Gruppe B wurden im Mittel 6,2 KI-Nachrichten pro Person gesendet, mit einem deutlichen Nutzungsspeak in Step 5.

In den **Fragebogendaten** ($N = 16$) erreichten beide Gruppen vergleichbar hohe SUS-Werte (85,0 bzw. 85,9). Im NASA-TLX berichtete Gruppe B einen tendenziell höheren mentalen Aufwand ($M = 3,1$ vs. $M = 2,5$) bei gleichzeitig niedrigerer Frustration (Median 6,5 vs. 10,0). Die hedonischen UEQ-S-Items *interessant* und *neuartig* lagen in Gruppe B um jeweils über einen Punkt höher. Der NPS fiel mit $M = 8,5$ ebenfalls besser aus als in Gruppe A ($M = 7,5$).

Die **Korrelationsanalysen** identifizierten Programmiererfahrung als stärksten Prädiktor: Erfahrung korrelierte signifikant negativ mit mentalem Aufwand ($r = -0,73$) und Drill-Versuchen ($r = -0,63$). Die Zusammenhänge mit Bearbeitungszeit und Fehleranzahl waren negativ, aber nicht signifikant.

Die Interpretation dieser Befunde, ihre Einordnung in den theoretischen Rahmen aus Kapitel 2 sowie die Bewertung der Haupthypothese folgen in Kapitel 7 der Diskussion.

7 Diskussion

Dieses Kapitel interpretiert die in Kapitel 6 berichteten Befunde, ordnet sie in den theoretischen Rahmen aus Kapitel 2 ein und benennt die zentralen Limitationen der Studie. Im Mittelpunkt steht die Frage, inwiefern die beobachteten Unterschiede zwischen Gruppe A und Gruppe B auf die KI-Komponenten zurückzuführen sind und welche Schlüsse daraus für die Lernwirksamkeit gezogen werden können.

7.1 Interpretation der Ergebnisse

Die Forschungsfrage untersucht, inwiefern die Integration einer KI-gestützten Drill-Auswahl und eines KI-Chat-Tutors die lernbezogenen Indikatoren sowie die wahrgenommene Lernunterstützung beim Erlernen von Python-Grundlagen verbessert. Die erhobenen Daten zeichnen hierzu ein differenziertes Bild, das in mehreren Dimensionen von der formulierten Haupthypothese abweicht.

7.1.1 Das Effizienz-Paradoxon: Mehrzeit als Zeichen vertiefter Auseinandersetzung

Ein besonders auffälliger Befund der Studie ist der Zeitunterschied zwischen den Gruppen. Gruppe B benötigte im AFK-bereinigten Mittel 32,9 Minuten für den BMI-Kurs, Gruppe A nur 19,5 Minuten (Abbildung 3). Die Differenz von rund 13 Minuten entspricht einer Mehrzeit von 68 % und widerspricht auf den ersten Blick der Erwartung, dass KI-Unterstützung die Aufgabenbewältigung erleichtert oder beschleunigt.

Bei näherer Betrachtung lässt sich diese Beobachtung jedoch mit der *Cognitive Load Theory* (CLT, Abschnitt 2.2.1) als Zeichen produktiver Auseinandersetzung deuten. Die CLT unterscheidet *Germane Load*, also die kognitive Energie, die für den Aufbau mentaler Schemata aufgewendet wird, von *Extraneous Load*, der nutzlosen Belastung durch unklare Instruktionen oder Hilflosigkeit [13]. Während Gruppe A über keinen externen Mechanismus verfügt, der einen oberflächlichen Umgang mit den Aufgaben verhindert, könnte das progressive Hint-System des KI-Tutors (vgl. Abschnitt 3.2.2) bei konsequenter Nutzung zur schrittweisen Auseinandersetzung anleiten: Die Lösung ist erst nach Durchlaufen vorgelagerter Hinweisstufen zugänglich. Die Zeit, die Teilnehmende in Gruppe B investierten, ist dann nicht Ausdruck von Ineffizienz, sondern von erhöhtem *Germane Load*, also der Investition in das Verstehen.

Diese Interpretation wird durch die schrittweise Analyse gestützt. In Abbildung 4 konzentrieren sich die größten Zeitdifferenzen auf Step 5 (+3,4 min) und Step 6 (+4,2 min), also genau jene Kursabschnitte, in denen Variablen und Arithmetik eingeführt werden und die KI-Anfragen ihr Maximum erreichen (Abbildung 9, Peak bei Step 5 mit $\emptyset=1,1$ Nachrichten). In den späteren Steps, in denen die Konzepte gefestigt sind, nähern sich die Gruppenzeiten wieder an. Dieses Muster deutet darauf hin, dass der KI-Tutor nicht generell, sondern gezielt an den konzeptionell anspruchsvollen Stellen in Anspruch genommen wurde.

Im Sinne von Jenkins [3] unterscheidet sich *Surface Learning*, das bloße Auswendiglernen syntaktischer Muster, fundamental von *Deep Learning*, dem Durchdringen der zugrundeliegenden Logik. Gruppe A hatte keinen äußeren Anreiz, über eine korrekte Codezeile hinaus nachzudenken. Gruppe B wurde durch die Gesprächsführung des KI-Tutors regelmäßig mit Rückfragen zum Verarbeitungsprozess konfrontiert, was dem Konzept des *Process Feedback* nach Hattie und Timperley [11] entspricht. Die Mehrzeit ist daher als strukturell erwünschtes Merkmal des KI-Designs und nicht als Fehler des Systems zu werten.

7.1.2 Das Belastungs-Dilemma: Höherer Aufwand, geringere Frustration

Ein zweiter, theoretisch bedeutsamer Befund ist das Überkreuz-Muster in den NASA-TLX-Daten (Abbildung 11). Gruppe B zeigt gegenüber Gruppe A einen tendenziell höheren mentalen Aufwand ($M = 3,1$ vs. $M = 2,5$), aber gleichzeitig eine niedrigere wahrgenommene Frustration ($Mdn = 6,5$ vs. $Mdn = 10,0$ in Gruppe A). Besonders auffällig ist der einzelne Frustrations-Ausreißer in Gruppe A mit einem Wert von 70, der in Gruppe B in dieser Ausprägung nicht auftritt.

Dieses Muster lässt sich aus der CLT-Perspektive kohärent erklären. Der tendenziell höhere mentale Aufwand in Gruppe B lässt sich als Indikator für einen angestiegenen *Germane Load* interpretieren: Die Interaktion mit dem KI-Tutor, das Lesen mehrstufiger Hinweise und das Nachvollziehen von Erklärungen sind kognitiv anstrengend. Gleichzeitig wird der *Extraneous Load* gesenkt, da die spezifische Belastung durch Hilflosigkeit, also das Gefühl, nicht weiterzukommen und keine Orientierung zu haben, durch die verfügbare, kontextgebundene Unterstützung abgebaut wird. Der KI-Tutor übernimmt in dieser Hinsicht die Funktion eines *affektiven Scaffoldings* und wirkt dem von Jenkins [3] beschriebenen Phänomen der *Learned Helplessness* entgegen: Teilnehmende, die ohne Hilfe in einer Aufgabe feststecken, riskieren den Absprung in Frustration und Demotivation. Er hält sie in einem Zustand der Herausforderung, ohne sie in die Hoffnungslosigkeit abrutschen zu lassen.

Diese Interpretation stützt die in Abschnitt 3.2.2 formulierte Designentscheidung für ein progressives, nicht-direktives Hint-System. Ein System, das sofort die Lösung liefert, hätte den mentalen Aufwand gesenkt, ohne den produktiven Frustrationsabbau zu leisten, da die Nutzenden nichts erarbeitet hätten. Die vorliegenden Daten deuten darauf hin, dass genau dieser Gleichgewichtszustand aus hohem *Germane Load* und niedrigem *Extraneous Load* durch die Designentscheidungen erreicht wurde.

7.1.3 Fehlerkultur und Explorationsbereitschaft

Gruppe B produzierte im Mittel mehr als doppelt so viele Python-Fehler wie Gruppe A ($M = 7,1$ vs. $M = 2,7$, Abbildung 5). Die schrittweise Analyse zeigt einen besonders auffälligen Peak in Gruppe B bereits bei Step 1 ($M = 2,5$ gegenüber $M = 0,3$ in Gruppe A, Abbildung 6), obwohl dieser Schritt lediglich eine einfache Textausgabe verlangt.

Eine mögliche Erklärung ist die von Dijkstra beschriebene Fehlerintoleranz der Programmierung:

In Python führt bereits ein fehlendes Klammerpaar oder eine falsche Einrückung zu einem vollständigen Programmabbruch, was *Radical Novelty* als inhärentes Merkmal der Disziplin kennzeichnet [3]. In einem klassischen Lernsetting führt diese binäre Fehlerkultur häufig zu einer erhöhten Abbruchneigung. Die vorliegenden Daten legen nahe, dass die KI-Unterstützung die Einstiegshürde dieser radikalen Neuartigkeit senkt: Da Teilnehmende in Gruppe B wissen, dass sie im Fehlerfall sofortige Unterstützung erhalten, sinkt die Hemmschwelle gegenüber explorativem Ausprobieren und Fehler werden von einer Barriere zu einem Interaktionsanlass transformiert. Diese Entwicklung ist theoretisch konsistent mit der Funktion von *Supportive Information* im 4C/ID-Modell (Abschnitt 2.2.4): Die KI liefert das nötige Hintergrundwissen und unterstützt den Aufbau mentaler Modelle, um Programmierfehler konzeptuell zu durchdringen, anstatt nur die Lösung vorzugeben.

Die Korrelationsanalyse deutet tendenziell auf einen Zusammenhang zwischen Chat-Intensität und Fehleranzahl hin ($r = 0,59$, $p = 0,096$, Abbildung 10): Teilnehmende mit mehr Fehlern nutzten den KI-Tutor häufiger. Da der p-Wert die Signifikanzschwelle von 0,05 knapp verfehlt, ist dieser Befund als tendenzielle Evidenz zu werten, nicht als statistisch gesicherter Nachweis. Die Richtung des Zusammenhangs im Sinne von Ursache und Wirkung ist zudem nicht eindeutig bestimmbar. Es wäre gleichermaßen plausibel, dass mehr Fehler zu mehr Chat-Anfragen führen (reaktive Nutzung) oder dass eine KI-vermittelte Explorationsbereitschaft sowohl zu mehr Versuchen als auch zu mehr Fehlern führt, was in seiner Summe einen intensiveren Lernprozess darstellt.

7.1.4 Drill-Adaptivität: Fokussierung statt Streuung

Die Drill-Daten legen nahe, dass der KI-Algorithmus qualitativ anders arbeitete als die gewichtete Zufallsauswahl. Gruppe A bearbeitete 97 unterschiedliche Drill-IDs mit einer Shannon-Entropie von 4,4 Bits, Gruppe B lediglich 62 mit einer Entropie von 3,8 Bits (Abbildung 7). Die geringere Diversität in Gruppe B ist nach dem 4C/ID-Modell als Qualitätsmerkmal zu werten, nicht als Defizit.

Van Merriënboer beschreibt die Funktion der *Part-task Practice* als gezielte Automatisierung spezifischer *Recurrent Skills*, also der wiederkehrenden Teilfertigkeiten, die im Laufe eines Lernprozesses flüssig beherrschbar werden müssen [20]. Eine zufällige Streuung über 97 verschiedene Aufgaben fördert Breite, nicht Tiefe. Gemäß dem Designziel aus Abschnitt 3.2.2 ist das Modell angewiesen, bei niedriger Erfolgsrate einfachere, thematisch einschlägige Aufgaben zu priorisieren und dadurch jene Wiederholungsintensität zu erzeugen, die für Schemaautomatisierung notwendig ist. Die Entropiedifferenz von 0,6 Bits ist damit ein Indikator für diagnostische Entscheidungen des Algorithmus.

Dieses Bild wird durch die Verteilung der Lösungsversuche bei den Drills ergänzt (Abbildung 8): Gruppe B weist mit 5,8 % einen mehr als doppelt so hohen Anteil an Aufgaben mit vier oder mehr Versuchen aufweist als Gruppe A (2,4 %).

Dieser Befund lässt sich als Hinweis auf eine *Optimal Challenge*¹ im Sinne des *Scaffolding* interpretieren: Die KI wählt Aufgaben nahe an der oberen Grenze des aktuellen Kompetenzbereichs, was zu mehr Versuchen führt, zugleich aber die Automatisierung der Zielfertigkeiten besonders effektiv fördert. Der Zusammenhang zum NASA-TLX-Befund ist hier bedeutsam: Trotz mehr Versuchen und mehr Fehlern war die Frustration in Gruppe B nicht höher, sondern niedriger. Das System scheint das Scheitern pädagogisch aufzufangen.

7.1.5 Nutzungserfahrung und Akzeptanz

Beide Gruppen bewerteten die Gebrauchstauglichkeit der Plattform mit SUS-Werten von 85,0 (Gruppe A) und 85,9 (Gruppe B), was nach Bangor et al. [56] der Kategorie *Good* entspricht (Abbildung 12). Der nahezu identische SUS-Wert ist ein relevanter Befund: Die Integration der KI-Komponente hat die Bedienbarkeit der Plattform nicht beeinträchtigt. Zusätzliche Interaktionselemente wie das Chat-Fenster oder die KI-gesteuerte Drillauswahl haben keinen messbaren *Extraneous Load* durch Interfacekomplexität verursacht.

Der Net Promoter Score zeigt hingegen einen Unterschied zugunsten von Gruppe B ($M = 8,5$, $Mdn = 9,0$ gegenüber $M = 7,5$, $Mdn = 8,5$). Fünf der acht Befragten in Gruppe B erzielten einen Score von 10, was der Promoter-Kategorie entspricht und auf eine ausgeprägte Weiterempfehlungsbereitschaft hinweist. Dieses Ergebnis deutet darauf hin, dass die KI-Integration einen emotionalen Mehrwert erzeugt, der über die reine Bedienbarkeit hinausgeht.

Das UEQ-S-Item-Profil (Abbildung 13) macht diesen Effekt sichtbar. Während sich die Gruppenunterschiede bei den pragmatischen Items (*unterstützend*, *einfach*, *effizient*) in Grenzen halten, sind die Abstände bei den hedonischen Items erheblich: Das Attribut *interessant* fällt in Gruppe B um 1,3 Punkte höher aus (6,2 vs. 4,9), das Attribut *neuartig* um 1,2 Punkte (6,1 vs. 4,9). In der subjektiven Wahrnehmung der Teilnehmenden scheint die KI-Unterstützung die Lernumgebung von einem statischen Übungssystem zu einem interaktiven Lernpartner zu transformieren. Auffällig ist, dass Gruppe B die Plattform trotz objektiv längerer Bearbeitungszeit als *effizienter* bewertet (5,9 vs. 5,2 beim Item *effizient*). Dieses subjektive Zeit-Paradoxon ist ein bekanntes Phänomen aus der Usability-Forschung: Wer gut unterstützt wird und inhaltlich engagiert ist, empfindet die investierte Zeit als gut genutzt, unabhängig von der absoluten Dauer.

7.1.6 Wahrnehmung des KI-Tutors in Gruppe B

Die Wahrnehmung des KI-Tutors durch die Teilnehmenden in Gruppe B wird im Folgenden anhand quantitativer Fragebogendaten sowie einer qualitativen Sichtung der Chat-Verläufe beleuchtet.

¹Konzept der Aufgabenschwierigkeit unmittelbar oberhalb des aktuellen Kompetenzniveaus; theoretische Grundlage der Flow-Theorie (Csikszentmihalyi) und der *Zone of Proximal Development* (Vygotsky).

Quantitative Fragebogenauswertung Die KI-spezifischen Fragebogenitems in Gruppe B (Abbildung 14) zeichnen ein differenziertes Bild, das für das Verständnis der Nutzungsqualität bedeutsam ist. Die höchsten Zustimmungswerte erzielten auf die Erklärungsqualität ($M = 5,25$), das Vertrauen in die Antworten ($M = 5,38$) und die Wiederverwendungsbereitschaft ($Mdn = 6,5$). Das kritischste Item war die wahrgenommene Korrektheit der KI-Antworten mit $M = 3,50$, einem Wert unterhalb des Skalenneutralpunkts.

Das Nebeneinander von hohem Vertrauen und niedriger wahrgenommener Korrektheit ist zunächst widersprüchlich, lässt sich aber als Ausdruck eines kritisch-selektiven Nutzungsverhaltens interpretieren. Die Teilnehmenden nutzten den KI-Tutor als Orientierungshilfe, ohne seine Ausgaben unkritisch zu übernehmen. Das entspricht dem von Kasneci [4] beschriebenen Prinzip des *Trust but Verify*: Lernende schätzen die interaktive Unterstützung, hinterfragen aber die Belastbarkeit der generierten Inhalte. Für den Einsatz von KI in der Bildung ist dies eine grundsätzlich wünschenswerte Haltung, da sie das Risiko des unkritischen Übernehmens von Lösungen begrenzt.

Das unterdurchschnittliche Abschneiden des Items zur *allgemeinen Hilfreichkeit* ($M = 3,62$) lässt sich auf zwei Faktoren zurückführen. Einerseits ist das progressive Hint-System bewusst so gestaltet, dass es keine direkten Fehlerkorrekturen vornimmt (vgl. Abschnitt 3.2.2): Wer eine sofortige Fehlerbehebung erwartet, wird dieses Verhalten als wenig hilfreich empfinden, obwohl es didaktisch intendiert ist.

Andererseits zeigt die Sichtung der Chat-Verläufe, dass der KI-Tutor in einigen Situationen das tatsächliche Problem der Nutzenden nicht korrekt identifizieren konnte. Ein konkretes Beispiel: Die automatische Lösungsvalidierung prüfte den Code auf exakt vorgegebene Variablennamen, erwartet wurden etwa `weight` und `height`. Teilnehmende, die abweichende, aber semantisch äquivalente Namen wählten, erhielten eine Fehlermeldung, obwohl ihre Lösung funktional korrekt war. Da in Python die Bezeichnung einer Variablen für das Programmresultat keine Rolle spielt, handelt es sich dabei um eine Einschränkung der Validierungslogik, nicht um einen inhaltlichen Fehler. Der KI-Tutor war in solchen Situationen nicht in der Lage, diesen Zusammenhang zu erkennen: Er sah eine Fehlermeldung, konnte sie aber nicht als Artefakt der strikten Validierung einordnen und gab den Nutzenden keine zielführende Rückmeldung.

Das Nutzungserlebnis mit dem KI-Tutor war daher von erheblicher Inkonsistenz geprägt: Je nach Art und Komplexität des individuellen Problems reichten die Erfahrungen von effektiver Unterstützung bis hin zu allgemeiner Frustration. Beide Faktoren zusammen erklären das unterdurchschnittliche Ergebnis dieses Items: zum einen die didaktisch intendierte Erwartungsdiskrepanz, zum anderen die technische Inkonsistenz in der Fehlerdiagnose. Die Sichtung der Chat-Verläufe erlaubt darüber hinaus eine differenziertere Einschätzung der Stärken und Schwächen des Systems, als es die Fragebogendaten allein ermöglichen. Die Beobachtungen lassen sich in produktive und problematische Interaktionsmuster unterteilen.

Produktive Interaktionsmuster Auf der produktiven Seite nutzte ein Teil der Teilnehmenden den KI-Tutor zur aktiven Konzeptvertiefung: Sie fragten etwa nach der Funktionsweise von Variablen, nach dem Unterschied zwischen einem String-Literal und einer `print()`-Anweisung oder nach dem Verhalten von Kontrollstrukturen in konkreten Beispielsituationen. Diese Nutzungsform entspricht dem Designziel des Tutors als *Supportive Information*-Quelle im Sinne des 4C/ID-Modells (Abschnitt 2.2.4). Andere Teilnehmende fragten nach Möglichkeiten zur Code-Optimierung, beispielsweise wie eine geschachtelte `if-else`-Konstruktion mit `elif` kompakter geschrieben werden kann. Auch diese Anfragen gehen über bloßes Fehlersuchen hinaus und zeugen von einer explorativ-lernenden Grundhaltung.

Problematische Interaktionsmuster Daneben traten mehrere Klassen problematischer Interaktionen auf. Erstens konnte der KI-Tutor in einigen Fällen das eigentliche Problem nicht korrekt identifizieren, da er Fehlermeldungen aus der strikten Validierungslogik als inhaltliche Programmierfehler interpretierte, obwohl der Code der Lernenden funktional korrekt war (vgl. das `weight/height`-Beispiel oben). Zweitens fehlte dem KI-Tutor bei Fragen zu den Drill-Tasks der nötige Aufgabenkontext, weil dieser nicht in den Systemprompt übergeben wurde. Anfragen zu konkreten Übungsaufgaben konnte der Tutor daher nicht zuverlässig beantworten und wies die Lernenden nur allgemein auf Python-Konzepte hin, ohne auf die spezifische Aufgabe eingehen zu können. Drittens griff der Tutor in einzelnen Verläufen konzeptuell vor: In einem dokumentierten Fall wurden Konzepte zum Definieren eigener Funktionen erklärt, obwohl dieses Thema weder Teil des BMI-Kurses war noch an dieser Stelle benötigt wurde. Solche Vorwärtsreferenzen können den Fokus von den eigentlichen Kurszielen ablenken und zusätzlichen kognitiven Aufwand erzeugen. Viertens erwies sich das nicht-deterministische Verhalten des Sprachmodells als Inkonsistenzquelle: Kommunikationsstil und Detailtiefe der Antworten variierten zwischen den Teilnehmenden erheblich, da alle Ausgaben zur Laufzeit generiert wurden. In einem dokumentierten Fall antwortete das System trotz deutschsprachigen Kontexts auf Englisch.

Diese Beobachtungen zeigen, dass das KI-System im Rahmen dieser Studie noch nicht ausreichend kalibriert werden konnte, um eine konsistent positive Lernerfahrung zu gewährleisten. Die genannten Schwachstellen sind technischer Natur und durch gezieltes Prompt-Engineering, eine vollständigere Kontextübergabe und intensiveres Testing behebbar. Die positiven Nutzungsmuster belegen, wo das Potenzial des Systems liegt; die negativen Erfahrungen definieren den Entwicklungsbedarf für eine Folgeversion. Ausgewählte Beispielverläufe sind in Abbildung 20 im Anhang dokumentiert. Sämtliche anonymisierten Chat-Daten sind darüber hinaus im öffentlichen Projekt-Repository archiviert und stehen für weiterführende Analysen zur Verfügung (vgl. Anhang D).

7.1.7 Kursabschluss und Abbruchquote

Beide Gruppen wiesen eine identische Abbruchquote von 11,1 % auf. Aus dieser Gleichheit lässt sich weder eine Überlegenheit noch ein Nachteil der KI-Funktionen hinsichtlich der Kurspersistenz ableiten. Bemerkenswert ist, dass die abbrechende Person in Gruppe A die Plattform nach 2,13 Minuten verließ, während die abbrechende Person in Gruppe B erst nach 45,27 Minuten abbrach. Bei je einem einzigen Abbruchfall pro Gruppe sind diese Einzelfälle nicht als gruppencharakteristisch zu werten, sie zeigen jedoch, dass Abbrüche unterschiedlichen Mustern folgen können. Die Abbruchquote ist grundsätzlich eine der aussagekräftigsten Metriken, um den Einfluss von KI-Unterstützung auf die Kurspersistenz zu untersuchen, da hohe Drop-out-Raten in Einführungskursen ein bekanntes und gut dokumentiertes Problem darstellen [2]. Mit je einem Abbruchfall pro Gruppe ist eine belastbare Aussage auf Basis dieser Studie jedoch nicht möglich. Eine größere Stichprobe würde hier deutlich mehr Aufschluss geben.

7.1.8 Bewertung der Haupthypothese

Die zentrale Forschungsfrage dieser Arbeit lautet, inwiefern der Einsatz KI-generierter Übungsaufgaben und eines KI-gestützten Chat-Tutors die Effektivität beim Erlernen von Python-Grundlagen steigert. Auf Basis der erhobenen Daten lässt sich diese Frage wie folgt beantworten: Die KI-Komponenten steigern die Effektivität nicht im Sinne objektiver Effizienz, wohl aber im Sinne der emotionalen Qualität der Lernerfahrung. H1 nimmt an, dass die KI-Funktionen günstigere lernbezogene Indikatoren und eine höhere wahrgenommene Lernunterstützung erzeugen. Die Gegenhypothese H0 besagt, dass zwischen den Gruppen keine systematischen Unterschiede bestehen. Die erhobenen Daten bestätigen H1 nicht einheitlich. In den objektiven Log-Daten (Fehlerrate, Abschlussquote) zeigen sich keine klaren Vorteile für Gruppe B. In den subjektiven Metriken bestätigt der Fragebogen eine tendenziell positivere Nutzungserfahrung in Gruppe B, insbesondere bei hedonischer Qualität und Weiterempfehlungsbereitschaft. Die Zeitdaten, der auffälligste Unterschied, können je nach theoretischer Rahmung als Indikator für tieferes Engagement oder als Mehraufwand interpretiert werden.

Entscheidend ist die Differenzierung der Hypothese selbst: Wenn unter *besseren lernbezogenen Indikatoren* ausschließlich objektive Effizienzmetriken wie Fehlerrate und Bearbeitungszeit verstanden werden, ist H1 nicht bestätigt. Wenn jedoch die Qualität der Auseinandersetzung, die emotionale Stabilität und die Akzeptanz der Lernumgebung als Indikatoren einbezogen werden, liefern die Daten ein konsistentes Muster zugunsten der KI-Bedingung. Mit den vorliegenden Stichprobengrößen von je neun Personen sind jedoch keine belastbaren statistischen Schlüsse möglich. Die Ergebnisse sind als erste explorative Hinweise zu verstehen, die in einer größeren Folgestudie überprüft werden müssten.

7.2 Einordnung in den Forschungsstand

Die Befunde der vorliegenden Studie lassen sich auf mehreren Ebenen in den Forschungsstand einordnen.

Cognitive Load Theory und 4C/ID Das beobachtete Muster aus erhöhter Bearbeitungszeit, höherem mentalem Aufwand und niedrigerer Frustration in Gruppe B ist theoretisch kohärent mit den Vorhersagen der CLT. Van Merriënboer und Sweller [13] beschreiben das Ziel effektiver Instruktion als die Maximierung des *Germane Load* bei gleichzeitiger Minimierung des *Extraneous Load*. Die vorliegenden Daten legen nahe, dass das KI-Design genau dieses Gleichgewicht befördert hat: mehr Energie in das Verstehen, weniger Energie in das Hilfloose-Feststecken. Aus der Perspektive des 4C/ID-Modells [20] erfüllten die Kurs-Steps die Funktion authentischer *Learning Tasks*, der KI-Tutor lieferte *Just-in-Time Information* bedarfsgerecht (nachgewiesen durch die Konzentration der Chat-Anfragen auf den Konzept-Peak in Step 5) und die Drills übernahmen die *Part-task Practice* mit einer in Gruppe B diagnostisch fokussierten Auswahl.

Intelligente Tutorsysteme VanLehn [27] zeigt in seiner Meta-Analyse, dass *Intelligent Tutoring Systems* in der Regel zu keiner Beschleunigung, aber zu einer Verbesserung der Lernqualität führen. Das Zeitparadoxon der vorliegenden Studie, also die Mehrzeit in Gruppe B bei gleichzeitig besserer Bewertung, entspricht exakt diesem etablierten Muster. Die KI-Unterstützung ist kein Beschleunigungs-, sondern ein Qualitätsinstrument.

Vergleich mit PyTutor Der direkte Vergleich mit der PyTutor-Studie von Yang et al. [32] (vgl. Abschnitt 2.4.1) ist aufschlussreich. PyTutor setzte ebenfalls auf ein mehrstufiges Scaffolding-System (Pseudocode, Lückentext, Basislösung, Erweiterung) und berichtete für leistungsschwächere Lernende signifikant höhere Abschlussraten. Ein zentraler Befund der Studie war jedoch das *Frequenz-Paradoxon*: Moderate KI-Nutzung führte zu besseren Lernergebnissen, sehr hohe Nutzung hingegen nicht. In der vorliegenden Studie lag die durchschnittliche Chat-Nutzung bei $M = 6,2$ Nachrichten über 12 Aufgaben, also weniger als eine Anfrage pro Aufgabe. Dieser Wert liegt im moderaten Bereich und deutet darauf hin, dass die Stichprobe den von Yang et al. als optimal beschriebenen Nutzungsbereich traf, ohne in eine Überabhängigkeit abzugleiten.

Tabelle 3 stellt die wichtigsten Merkmale beider Studien vergleichend gegenüber.

Ein bedeutsamer Unterschied zwischen beiden Studien ist das Messinstrument für Frustration. Während Yang et al. das Frustrationsvermeidungsargument theoretisch herleiten, liefert die vorliegende Studie mit den NASA-TLX-Boxplots (Abbildung 11) erste empirische Hinweise darauf: Der Ausreißer mit einem Frustrationswert von 70 in Gruppe A veranschaulicht, was das Fehlen einer KI-Unterstützung für einzelne Teilnehmende bedeuten kann.

Merkmal	PyTutor (Yang et al., 2024)	Vorliegende Studie
Stichprobe	71 Studierende	18 Teilnehmende
Dauer	11 Wochen	Einmalige Sitzung
Sprache	Python	Python
KI-System	ChatGPT-basiert, 4 Hint-Stufen	OpenAI API, 5 Hint-Stufen
Hauptbefund	Bessere Abschlussraten in leist.-schwächerer Gruppe	Mehr Engagement, niedrigere Frustration in KI-Gruppe
Nutzungsfrequenz	Moderate Nutzung optimal	M = 6,2 Nachrichten (moderat)
Frustrationsmessung	Theoretisch argumentiert	Empirisch erhoben (NASA-TLX)

Tabelle 3: Vergleich der vorliegenden Studie mit PyTutor (Yang et al., 2024) [32].

Erfahrung als Moderator Die Korrelationsanalysen (Abschnitt 6.9) bestätigen Programmiererfahrung als den stärksten Prädiktor für kognitive Belastung ($r = -0,73$, $p = 0,001$ für mentalen Aufwand, Abbildung 16) und Drill-Performance ($r = -0,63$, $p = 0,010$ für Versuche pro Aufgabe, Abbildung 15). Dieser Befund deckt sich mit dem von Yang et al. beschriebenen *Expertise-Reversal-Effekt*²: Einsteigende profitieren stärker von externer Unterstützung als Fortgeschrittene, die bereits über automatisierte Schemata verfügen. Die beiden Teilnehmenden in Gruppe A mit Selbsteinschätzungswert 5 (höchstes Vorwissen) könnten die Ergebnisse der Kontrollgruppe, insbesondere die Bearbeitungszeit, nach unten und die Abschlussrate nach oben verzerrt haben. Diese *Confound-Variable*³ ist bei der Interpretation des Gruppenvergleichs zu berücksichtigen. Die Korrelationen zwischen Erfahrung und Bearbeitungszeit ($r = -0,46$, $p = 0,071$) sowie Fehleranzahl ($r = -0,50$, $p = 0,051$) sind dagegen nicht signifikant und zeigen eine erhebliche Streuung, was darauf hindeutet, dass Bearbeitungszeit und Fehleraufkommen durch individuelle Faktoren beeinflusst werden, die über die Programmiererfahrung hinausgehen.

²Kalyuga et al. (2003): Instruktionsmaßnahmen, die für Einsteigende lernförderlich sind, können für Lernende mit höherem Vorwissen wirkungslos oder sogar kontraproduktiv sein.

³Variable, die sowohl mit der unabhängigen als auch mit der abhängigen Variable korreliert und den beobachteten Zusammenhang verzerren kann.

7.3 Limitationen

Die vorliegende Studie unterliegt mehreren Einschränkungen, die bei der Interpretation der Befunde zu berücksichtigen sind.

Die wichtigste Einschränkung ist die geringe Stichprobengröße von neun Personen je Gruppe. Alle berichteten Gruppenunterschiede sind deskriptiv und erlauben keine Kausalaussagen. Statistische Inferenztests sind bei dieser Stichprobengröße nicht aussagekräftig und wurden daher nicht berichtet.

Eine Replikation mit deutlich größerer Stichprobe ist Voraussetzung für belastbare Schlussfolgerungen.

Das *Convenience Sampling*⁴ im persönlichen sozialen Umfeld birgt das Risiko von *Demand Characteristics*⁵: Teilnehmende, die die Studienleitung kennen, könnten Fragebögen positiv verzerrt ausgefüllt haben. Dieser Effekt ist nicht kontrollierbar, aber für die Einordnung der subjektiven Bewertungen relevant.

Die demografische Ungleichheit zwischen den Gruppen ist eine weitere Einschränkung. Gruppe B war im Mittel rund 11 Jahre jünger als Gruppe A ($M = 25,1$ vs. $M = 36,1$ Jahre, Abbildung 1). Jüngere Teilnehmende tendieren laut Kasneci [4] zu einer höheren Affinität gegenüber KI-Systemen. Es ist daher nicht auszuschließen, dass die positiveren UEQ-S-Werte in Gruppe B teilweise auf den Altersunterschied und nicht ausschließlich auf die KI-Komponente zurückzuführen sind. Außerdem wurde kein Pre-Test zur objektiven Erfassung des Vorwissens erhoben. Bjork [17] weist darauf hin, dass Selbsteinschätzungen im Lernkontext systematisch verzerrt sein können. Die Vergleichbarkeit der Gruppen ist daher nur über die Selbsteinschätzung (Gruppe A: $M = 3,0$, Gruppe B: $M = 2,6$) abgesichert, die zudem in Gruppe A eine deutlich größere Streuung aufweist. Die Studie beschränkt sich auf eine einmalige Lernsitzung von unter 45 Minuten. Langzeiteffekte auf Wissensspeicherung, Transfer oder kumulativen Lernerfolg wurden nicht gemessen. Ob die beobachteten Muster bei wiederholter Nutzung oder in einem anderen inhaltlichen Kontext replizierbar sind, kann auf Basis dieser Studie nicht beurteilt werden. Yang et al. [32] untersuchten PyTutor über 11 Wochen und zeigten erst dann deutlichere Lernerfolge, was für die Relevanz des Zeitfaktors spricht.

Schließlich basiert die KI-gestützte Drill-Auswahl auf einem Prompt-Design, das nicht formell validiert wurde. Die tatsächlich realisierten Auswahlentscheidungen des Sprachmodells sind eine *Black Box* und können nur indirekt über die beobachteten Entropie-Unterschiede erschlossen werden. Darüber hinaus wurden im freien Feedback vereinzelte technische Probleme gemeldet, darunter eine Browser-Inkompatibilität mit Safari auf macOS. Solche Stabilitätslücken sind als typisches Merkmal eines Forschungsprototyps einzuordnen, können die Nutzungserfahrung einzelner Teilnehmender jedoch beeinflusst haben.

⁴Stichprobenverfahren, bei dem Teilnehmende nach Verfügbarkeit und Zugänglichkeit rekrutiert werden, nicht nach dem Zufallsprinzip.

⁵Tendenz von Versuchspersonen, das erwartete Verhalten der Studienleitung zu antizipieren und ihr eigenes Verhalten entsprechend anzupassen.

7.4 Implikationen für Forschung und Praxis

Trotz der genannten Einschränkungen lassen sich aus den Ergebnissen Hinweise für Forschung und Praxis ableiten.

Aus Praxisperspektive zeigt die Studie, dass ein autarkes KI-gestütztes Lernsystem für Python-Grundlagen technisch realisierbar ist und von Teilnehmenden mit hohen Usability-Scores von $SUS \geq 85$ in beiden Gruppen grundsätzlich positiv aufgenommen wird. Für den Einsatz in der Hochschullehre ist das Konzept der KI-gestützten Drill-Auswahl besonders relevant, da es ohne zusätzlichen Workload für Lehrende eine individualisierte *Part-task Practice* ermöglicht. Das progressive Hint-System des KI-Tutors kann als Vorlage für ein nicht-direktives, scaffolding-basiertes KI-Tutoring dienen.

Für die Praxis relevant ist auch der Befund zur Frustrationsvermeidung. In Einführungskursen für Programmierung, in denen hohe Abbruchquoten ein bekanntes Problem sind [2], könnte die Verfügbarkeit eines reaktiven KI-Tutors insbesondere für Einsteigende ohne Vorkenntnisse (Erfahrungsstufe 1–2, Abbildung 16) die kritischen Frustrationsschwellen abfedern und damit die Persistenz im Kurs erhöhen.

Alle erhobenen Rohdaten, einschließlich Chat-Verläufe, Aufgaben-Log-Daten und Fragebogenantworten, sind in anonymisierter Form im öffentlichen Projekt-Repository archiviert und stehen für weiterführende Auswertungen zur Verfügung (vgl. Anhang D).

8 Fazit und Ausblick

Dieses abschließende Kapitel fasst die zentralen Erkenntnisse der Arbeit zusammen und benennt konkrete Ansatzpunkte für weiterführende Forschung und Systementwicklung.

8.1 Zusammenfassung

Diese Arbeit untersuchte, inwiefern die Integration eines KI-Tutors und einer adaptiven Drill-Auswahl die Effektivität beim Erlernen von Python-Grundlagen steigert. Zu diesem Zweck wurde ein webbasiertes Lernsystem entwickelt, das auf dem 4C/ID-Modell aufbaut und in einer kontrollierten Studie mit 18 Teilnehmenden erprobt wurde. Die Ergebnisse lassen sich in drei Kernaussagen bündeln.

Erstens konnten keine eindeutigen Unterschiede in den objektiven Leistungsmetriken festgestellt werden. Fehlerrate, Kursabschluss und Drill-Performance unterschieden sich zwischen der Kontrollgruppe (Gruppe A) und der Experimentalgruppe (Gruppe B) nicht systematisch. Die Haupthypothese, dass KI-Funktionen günstigere lernbezogene Indikatoren erzeugen, ließ sich bei $N = 18$ weder bestätigen noch widerlegen.

Zweitens liefern die subjektiven und qualitativen Daten ein konsistentes Muster zugunsten der KI-Bedingung. Gruppe B benötigte im Mittel 68 % mehr Zeit, wies dabei aber eine deutlich niedrigere Frustration auf (NASA-TLX-Median: 6,5 vs. 10,0). Die KI-gestützte Drill-Auswahl zeigte eine messbar geringere Shannon-Entropie (3,8 vs. 4,4 Bits), was auf eine diagnostisch fokussierte, lernwirksame Aufgabenauswahl hindeutet. Die hedonischen UEQ-S-Items *interessant* und *neuartig* lagen in Gruppe B um über einen Punkt höher, und der Net Promoter Score fiel deutlich besser aus. Die KI-Integration verbesserte die Bedienbarkeit nicht, aber sie veränderte die emotionale Qualität des Lernerlebnisses spürbar.

Drittens zeigt die Sichtung der Chat-Verläufe, dass das System bereits in seiner ersten Version ein breites Spektrum von Nutzungsformen ermöglicht: von der vertiefenden Konzeptanfrage bis zur Code-Optimierungsfrage. Gleichzeitig wurden klare technische Schwachstellen sichtbar, darunter fehlender Drill-Kontext im Systemprompt, konzeptuelle Vorwärtsreferenzen und nicht-deterministisches Verhalten des Sprachmodells. Diese Befunde sind weniger ein Urteil über das Potenzial von KI im Lernen als eine präzise Zielscheibe für die Weiterentwicklung.

Zusammenfassend lässt sich festhalten: KI-Unterstützung beim Erlernen von Python verändert den Lernprozess qualitativ, ohne ihn messbar effizienter zu machen. Sie senkt Frustration, erhöht das Engagement und macht Fehler zu einem Interaktionsanlass statt zu einer Barriere. Ob sich daraus auch messbare Lernvorteile ergeben, bleibt eine offene Frage, die nur mit einer größeren Stichprobe und längerem Beobachtungszeitraum beantwortet werden kann.

8.2 Ausblick

Aus den Befunden und Limitationen dieser Arbeit ergeben sich konkrete Ansatzpunkte für weiterführende Forschung und Systementwicklung.

Auf methodischer Ebene sollte eine Folgestudie mit deutlich größerer Stichprobe, echter Randomisierung und einem objektiven Pre-Test zur Vorwissenserhebung durchgeführt werden. Erst ab einem Stichprobenumfang von mindestens 30 Personen je Gruppe werden inferenzstatistische Aussagen möglich, die über explorative Hinweise hinausgehen. Ergänzend wäre eine Untersuchung über mehrere Lernsituationen sinnvoll, um Langzeiteffekte auf Wissensretention und Transfer zu erfassen, die in einer einmaligen Sitzung von unter 45 Minuten strukturell nicht sichtbar werden können.

Auf Studiendesign-Ebene wäre ein verblindetes Parallel-Design empfehlenswert, um den in dieser Studie nicht kontrollierbaren Gruppenerwartungseffekt zu eliminieren. Beide Gruppen würden dabei eine KI-Schnittstelle erhalten, sodass keine Gruppe weiß, in welcher Bedingung sie sich befindet. Die Kontrollgruppe würde eine generische KI ohne Kurskontext und ohne adaptive Drillauswahl nutzen, die Experimentalgruppe hingegen die vollständige, kontextgebundene Tutorfunktion. Ein solches Design würde die spezifischen Effekte des intelligenten KI-Designs isolieren, anstatt lediglich KI-Nutzung mit keiner KI-Nutzung zu vergleichen.

Auf Systemebene bieten die Chat-Verlauf-Analysen eine konkrete Entwicklungsagenda: Der Drill-Kontext sollte vollständig in den Systemprompt integriert werden, das konzeptuelle Vorgreifen durch eine Themenfilterung eingeschränkt werden und die Validierungslogik sollte semantisch flexibler gestaltet werden, um äquivalente Lösungen zu erkennen. Darüber hinaus war die Chat-Datenbasis mit insgesamt rund 56 Nachrichten von neun Teilnehmenden zu klein, um daraus statistisch belastbare Nutzungsmuster abzuleiten. Eine umfangreichere Datengrundlage würde es erlauben, mittels strukturierter qualitativer Inhaltsanalyse zu untersuchen, ob Lernende die Hinweis-Eskalation als Lerngelegenheit oder als Umgehungsstrategie nutzen, was den Kern der didaktischen Wirksamkeit des Systems berührt.

Das entwickelte System ist ein erster, funktionsfähiger Prototyp, der zeigt, dass KI-gestütztes Lernen in einem Python-Einführungskurs technisch realisierbar und von Lernenden grundsätzlich positiv aufgenommen wird. Die offenen Fragen sind damit nicht Ausdruck eines Scheiterns, sondern das natürliche Ergebnis eines ersten empirischen Zugangs zu einem noch wenig erforschten Gestaltungsraum.

Literaturverzeichnis

- [1] Stack Overflow, “Stack Overflow Developer Survey 2024,” [Online] Available: <https://survey.stackoverflow.co/2024/> (Accessed: Feb. 14, 2026).
- [2] J. Bennedsen and M. E. Caspersen, “Failure Rates in Introductory Programming,” in *ACM SIGCSE Bulletin*, vol. 39, no. 2, 2007, pp. 32–36, doi: <https://doi.org/10.1145/1272848.1272879>.
- [3] T. Jenkins, “On the Difficulty of Learning to Program,” in *Proceedings of the 3rd Annual Conference of the LTSN Centre for Information and Computer Sciences*, 2002, pp. 53–58.
- [4] E. Kasneci, K. Seßler, S. Küchemann, M. Bannert, D. Dementieva, F. Fischer, U. Gasser, G. Groh, S. Günemann, E. Hüllermeier, S. Kruber, G. Kutyniok, T. Michaeli, C. Nerdel, J. Pfeffer, O. Poquet, M. Saber, A. Schmidt, T. Seidel, M. Stadler, J. Weller, J. Kuhn, and G. Kasneci, “ChatGPT for Good? On Opportunities and Challenges of Large Language Models for Education,” *Learning and Individual Differences*, vol. 103, p. 102274, 2023, doi: <https://doi.org/10.1016/j.lindif.2023.102274>.
- [5] M. Tomaschitz, “Erlernen von Programmieren an Oberstufen der Gymnasien,” Ph.D. dissertation, Universität für Weiterbildung Krems, 2020.
- [6] Y. Qian and J. Lehman, “Students’ Misconceptions and Other Difficulties in Introductory Programming: A Literature Review,” in *ACM Transactions on Computing Education*, vol. 18, no. 1, 2017, pp. 1–24, doi: <https://doi.org/10.1145/3077618>.
- [7] J. Sorva, “Notional Machines and Introductory Programming Education,” *ACM Transactions on Computing Education*, vol. 13, no. 2, pp. 1–31, 2013, doi: <https://doi.org/10.1145/2483710.2483713>.
- [8] A. Luxton-Reilly, Simon, I. Albluwi, B. A. Becker, M. Giannakos, A. N. Kumar, L. Ott, J. Paterson, M. J. Scott, J. Sheard, and C. Szabo, “Introductory Programming: A Systematic Literature Review,” *Proceedings Companion of the 23rd Annual ACM Conference on Innovation and Technology in Computer Science Education*, pp. 55–106, 2018, doi: <https://doi.org/10.1145/3293881.3295779>.
- [9] TIOBE Software, “TIOBE Index,” [Online] Available: <https://www.tiobe.com/tiobe-index/> (Accessed: Feb. 14, 2026).
- [10] K. A. Ericsson, R. T. Krampe, and C. Tesch-Römer, “The Role of Deliberate Practice in the Acquisition of Expert Performance,” *Psychological Review*, vol. 100, no. 3, pp. 363–406, 1993, doi: <https://doi.org/10.1037/0033-295X.100.3.363>.

- [11] J. Hattie and H. Timperley, "The Power of Feedback," *Review of Educational Research*, vol. 77, no. 1, pp. 81–112, 2007, doi: <https://doi.org/10.3102/003465430298487>.
- [12] J. Sweller, "Cognitive Load During Problem Solving: Effects on Learning," *Cognitive Science*, vol. 12, no. 2, pp. 257–285, 1988, doi: https://doi.org/10.1207/s15516709cog1202_4.
- [13] J. Sweller, J. J. G. van Merriënboer, and F. G. W. C. Paas, "Cognitive Architecture and Instructional Design," *Educational Psychology Review*, vol. 10, no. 3, pp. 251–296, 1998, doi: <https://doi.org/10.1023/A:1022193728205>.
- [14] B. J. Zimmerman, "Becoming a Self-Regulated Learner: An Overview," *Theory Into Practice*, vol. 41, no. 2, pp. 64–70, 2002, doi: https://doi.org/10.1207/s15430421tip4102_2.
- [15] E. Panadero, "A Review of Self-regulated Learning: Six Models and Four Directions for Research," *Frontiers in Psychology*, vol. 8, 2017, doi: <https://doi.org/10.3389/fpsyg.2017.00422>.
- [16] P. H. Winne, "A Perspective on State-of-the-art Research on Self-regulated Learning," *Instructional Science*, vol. 33, no. 5–6, pp. 559–565, 2005, doi: <https://doi.org/10.1007/s11251-005-1280-9>.
- [17] R. A. Bjork, J. Dunlosky, and N. Kornell, "Self-Regulated Learning: Beliefs, Techniques, and Illusions," *Annual Review of Psychology*, vol. 64, no. 1, pp. 417–444, 2013, doi: <https://doi.org/10.1146/annurev-psych-113011-143823>.
- [18] M. Kazemitabaar, X. Hou, A. Henley, B. J. Ericson, D. Weintrop, and T. Grossman, "How Novices Use LLM-based Code Generators to Solve CS1 Coding Tasks in a Self-Paced Learning Environment," in *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research (Koli Calling '23)*. ACM, 2023, pp. 1–12, doi: <https://doi.org/10.1145/3631802.3631806>.
- [19] R. Azevedo and A. F. Hadwin, "Scaffolding Self-regulated Learning and Metacognition: Implications for the Design of Computer-based Scaffolds," *Instructional Science*, vol. 33, no. 5–6, pp. 367–379, 2005, doi: <https://doi.org/10.1007/s11251-005-1272-9>.
- [20] J. J. G. van Merriënboer and P. A. Kirschner, *Ten Steps to Complex Learning: A Systematic Approach to Four-Component Instructional Design*, 3rd ed. Routledge, 2018, isbn: 978-1-138-08079-5.
- [21] J. J. G. van Merriënboer, R. E. Clark, and M. B. M. de Croock, "Blueprints for Complex Learning: The 4C/ID-Model," *Educational Technology Research and Development*, vol. 50, no. 2, pp. 39–61, 2002, doi: <https://doi.org/10.1007/BF02504993>.
- [22] J. Frerejean, J. L. van Strien, P. A. Kirschner, and S. Brand-Gruwel, "Designing Instruction for Complex Learning: 4C/ID in Higher Education," *European Journal of Education*, vol. 54, no. 4, pp. 513–524, 2019, doi: <https://doi.org/10.1111/ejed.12363>.

- [23] H. M. Niegemann and A. Weinberger, Eds., *Handbuch Bildungstechnologie: Konzeption und Einsatz digitaler Lernumgebungen*. Springer, 2020, doi: <https://doi.org/10.1007/978-3-662-54368-9>.
- [24] L. W. Anderson and D. R. Krathwohl, *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. Longman, 2001.
- [25] OpenAI, "OpenAI API Documentation," [Online] Available: <https://platform.openai.com/docs> (Accessed: Feb. 14, 2026).
- [26] —, "Structured Outputs," [Online] Available: <https://platform.openai.com/docs/guides/structured-outputs> (Accessed: Feb. 16, 2026).
- [27] K. VanLehn, "The Relative Effectiveness of Human Tutoring, Intelligent Tutoring Systems, and Other Tutoring Systems," *Educational Psychologist*, vol. 46, no. 4, pp. 197–221, 2011, doi: <https://doi.org/10.1080/00461520.2011.611369>.
- [28] K. Scheiter, E. Bauer, Y. Omarchevska, C. Schumacher, and M. Sailer, *Künstliche Intelligenz in der Schule: Eine Handreichung zum Stand in Wissenschaft und Praxis*. Bonn: Rahmenprogramm empirische Bildungsforschung, BMBF, 2025, [Online] Available: https://www.empirische-bildungsforschung-bmbf.de/img/KI_Review.pdf.
- [29] B. A. Becker, P. Denny, J. Finnie-Ansley, A. Luxton-Reilly, J. Prather, and E. A. Santos, "Programming Is Hard – Or at Least It Used to Be: Educational Opportunities and Challenges of AI Code Generation," *Proceedings of the 54th ACM Technical Symposium on Computer Science Education (SIGCSE)*, pp. 500–506, 2023, doi: <https://doi.org/10.1145/3545945.3569759>.
- [30] S. Sarsa, P. Denny, A. Hellas, and J. Leinonen, "Automatic Generation of Programming Exercises and Code Explanations Using Large Language Models," in *Proceedings of the 2022 ACM Conference on International Computing Education Research (ICER)*, 2022, pp. 27–43, doi: <https://doi.org/10.1145/3501385.3543957>.
- [31] S. MacNeil, A. Tran, A. Hellas, J. Kim, S. Sarsa, P. Denny, S. Bernstein, and J. Leinonen, "Experiences from Using Code Explanations Generated by Large Language Models in a Web Software Development E-Book," in *Proceedings of the 54th ACM Technical Symposium on Computer Science Education (SIGCSE)*, 2023, pp. 931–937, doi: <https://doi.org/10.1145/3545945.3569785>.
- [32] A. C. M. Yang, J.-Y. Lin, C.-Y. Lin, and H. Ogata, "Enhancing Python Learning with PyTutor: Efficacy of a ChatGPT-Based Intelligent Tutoring System in Programming Education," *Computers and Education: Artificial Intelligence*, vol. 7, p. 100309, 2024, doi: <https://doi.org/10.1016/j.caeai.2024.100309>.

- [33] S. Mun and I.-H. Jo, “Evolving the 4C/ID Model Through Learning Analytics Approaches: Teaching and Learning System Design Framework for Supporting Learners’ Complex Problem-Solving,” in *Proceedings of the 15th International Learning Analytics and Knowledge Conference (LAK '25)*, 2025, pp. 611–619, doi: <https://doi.org/10.1145/3706468.3706549>.
- [34] Europäisches Parlament und Rat der Europäischen Union, “Verordnung (EU) 2016/679 (Datenschutz-Grundverordnung),” 2016, [Online] Available: <https://eur-lex.europa.eu/eli/reg/2016/679/oj> (Accessed: Feb. 16, 2026).
- [35] Meta Platforms, “React: A JavaScript Library for Building User Interfaces,” [Online] Available: <https://react.dev> (Accessed: Feb. 14, 2026).
- [36] Vercel, “Next.js: The React Framework,” [Online] Available: <https://nextjs.org> (Accessed: Feb. 14, 2026).
- [37] Microsoft, “TypeScript: JavaScript With Syntax For Types,” [Online] Available: <https://www.typescriptlang.org> (Accessed: Feb. 14, 2026).
- [38] Tailwind Labs, “Tailwind CSS: A Utility-First CSS Framework,” [Online] Available: <https://tailwindcss.com> (Accessed: Feb. 16, 2026).
- [39] Chakra UI, “Chakra UI: Simple, Modular and Accessible UI Components,” [Online] Available: <https://chakra-ui.com> (Accessed: Feb. 16, 2026).
- [40] Microsoft, “Monaco Editor: The Code Editor that Powers VS Code,” [Online] Available: <https://microsoft.github.io/monaco-editor/> (Accessed: Feb. 16, 2026).
- [41] —, “Visual Studio Code,” [Online] Available: <https://code.visualstudio.com> (Accessed: Feb. 16, 2026).
- [42] Google, “Firebase: Build and Run Apps,” [Online] Available: <https://firebase.google.com> (Accessed: Feb. 16, 2026).
- [43] Pyodide Development Team, “Pyodide: Python with the Scientific Stack, Compiled to WebAssembly,” [Online] Available: <https://pyodide.org> (Accessed: Feb. 16, 2026).
- [44] W3C, “WebAssembly,” [Online] Available: <https://webassembly.org> (Accessed: Feb. 16, 2026).
- [45] Canonical, “Ubuntu: The Leading Operating System for PCs, IoT Devices, Servers and the Cloud,” [Online] Available: <https://ubuntu.com> (Accessed: Feb. 16, 2026).
- [46] Hetzner Online GmbH, “Hetzner Online: Hosting, Webhosting, Managed Server, Dedicated Server,” [Online] Available: <https://www.hetzner.com> (Accessed: Feb. 16, 2026).

- [47] Coolify, “Coolify: An Open-Source and Self-Hostable Heroku / Netlify / Vercel Alternative,” [Online] Available: <https://coolify.io> (Accessed: Feb. 16, 2026).
- [48] Docker Inc., “Docker: Accelerated Container Application Development,” [Online] Available: <https://www.docker.com> (Accessed: Feb. 16, 2026).
- [49] J. Gruber, “Markdown,” [Online] Available: <https://daringfireball.net/projects/markdown/> (Accessed: Feb. 16, 2026).
- [50] Anthropic, “Claude Code: An Agentic Coding Tool,” [Online] Available: <https://docs.anthropic.com/en/docs/claude-code> (Accessed: Feb. 16, 2026).
- [51] J. Brooke, “SUS: A “Quick and Dirty” usability scale,” in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. London: Taylor and Francis, 1996, pp. 189–194.
- [52] M. Schrepp, A. Hinderks, and J. Thomaschewski, “Design and evaluation of a short version of the user experience questionnaire (UEQ-S),” *International Journal of Interactive Multimedia and Artificial Intelligence*, vol. 4, no. 6, pp. 103–108, 2017, doi: <https://doi.org/10.9781/ijimai.2017.09.001>.
- [53] C. E. Shannon, “A mathematical theory of communication,” *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948, dOI: 10.1002/j.1538-7305.1948.tb01338.x.
- [54] S. G. Hart and L. E. Staveland, “Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research,” in *Human Mental Workload*, P. A. Hancock and N. Meshkati, Eds. Amsterdam: Elsevier, 1988, pp. 139–183, dOI: 10.1016/S0166-4115(08)62386-9.
- [55] F. F. Reichheld, “The one number you need to grow,” *Harvard Business Review*, vol. 81, no. 12, pp. 46–54, 2003.
- [56] A. Bangor, P. Kortum, and J. Miller, “Determining what individual sus scores mean: Adding an adjective rating scale,” *Journal of Usability Studies*, vol. 4, no. 3, pp. 114–123, 2009.

Abkürzungsverzeichnis

4C/ID	Four-Component Instructional Design
AFK	Away from Keyboard
API	Application Programming Interface
BMI	Body-Mass-Index
BMBF	Bundesministerium für Bildung und Forschung
CLT	Cognitive Load Theory
CSS	Cascading Style Sheets
DSGVO	Datenschutz-Grundverordnung
GPT	Generative Pre-trained Transformer
IQR	Interquartile Range
ITS	Intelligentes Tutorsystem
JSON	JavaScript Object Notation
KI	Künstliche Intelligenz
LLM	Large Language Model
MCQ	Multiple-Choice-Frage
Mdn	Median
MOOC	Massive Open Online Course
NASA-TLX	NASA Task Load Index
NoSQL	Not only SQL
NPS	Net Promoter Score
n. s.	nicht signifikant ($p \geq 0,05$)
SRL	Selbstreguliertes Lernen
SSL	Secure Sockets Layer
SUS	System Usability Scale
UI	User Interface
UEQ-S	User Experience Questionnaire – Short

Abbildungsverzeichnis

1	Altersverteilung nach Gruppe (Post-Fragebogen, $n = 8$ je Gruppe). Gruppe A: $M = 36,1$, $Mdn = 34,5$. Gruppe B: $M = 25,1$, $Mdn = 21,0$	45
2	Selbsteinschätzung der Programmierkenntnisse nach Gruppe. Gruppe A: $M = 3,0$, $Mdn = 2,5$. Gruppe B: $M = 2,6$, $Mdn = 3,0$. (Post-Fragebogen, Skala: 1 = keine Kenntnisse, 5 = professionelle Kenntnisse)	45
3	Gesamtbearbeitungszeit des BMI-Kurses nach Gruppe (AFK-bereinigt, $N = 17$). Gruppe A: $M = 19,5$ min, $Mdn = 18,7$ min. Gruppe B: $M = 32,9$ min, $Mdn = 32,4$ min.	47
4	Durchschnittliche Bearbeitungszeit pro Step nach Gruppe ($N = 18$, Fehlerbalken: ± 1 SEM). Gruppe A: $M = 1,9$ min. Gruppe B: $M = 2,9$ min. Peaks in Step 5 (+3,4 min) und Step 6 (+4,2 min).....	47
5	Gesamtanzahl Python-Fehler im BMI-Kurs nach Gruppe ($N = 18$, Datenbasis: Teilnehmende mit aktiver Code-Nutzung). Gruppe A: $M = 2,7$, $Mdn = 2,0$. Gruppe B: $M = 7,1$, $Mdn = 5,0$	48
6	Durchschnittliche Python-Fehleranzahl pro Step nach Gruppe ($N = 18$, Fehlerbalken: ± 1 SEM). Gruppe A: $M = 0,3$. Gruppe B: $M = 0,6$. Markanter Peak in Gruppe B bei Step 1 ($M = 2,5$).	49
7	Drill-Vielfalt nach Gruppe: Anzahl einzigartiger Drill-IDs (links) und Shannon-Entropie der Aufgabenverteilung in Bit (rechts). Gruppe A: 97 einzigartige Drills, Entropie 4,4. Gruppe B: 62 einzigartige Drills, Entropie 3,8.	50
8	Anteil der Drills nach benötigter Versuchsanzahl (Basis: 433 erfolgreich gelöste Drills, MCQ (Multiple-Choice-Fragen) und Code kombiniert). Gruppe A: 86,0 % im 1. Versuch, 2,4 % bei 4+ Versuchen. Gruppe B: 83,6 % im 1. Versuch, 5,8 % bei 4+ Versuchen.....	50
9	KI-Anfragen pro Kursschritt in Gruppe B (Pain-Point-Analyse). n = Gesamtanzahl Chat-Nachrichten je Step, $\bar{O}$ = Durchschnitt pro Teilnehmenden. Peaks in Step 5 ($n=10$, $\bar{O}=1,1$) und Step 12 ($n=9$, $\bar{O}=1,1$).	51
10	Zusammenhang zwischen Chat-Intensität und Python-Fehleranzahl in Gruppe B ($n = 9$). Pearson $r = 0,59$, $p = 0,096$ (nicht signifikant).	52
11	NASA-TLX: Mentaler Aufwand (Skala 1–9, links) und Frustration (Skala 0–100, rechts) nach Gruppe ($N = 16$). Die y-Achsen beider Teilgrafiken sind unterschiedlich skaliert.....	53
12	SUS-Scores (Skala 0–100, links) und NPS-Scores (Skala 0–10, rechts) nach Gruppe ($N = 16$). Gestrichelte Linien markieren die Benchmarks: $SUS \geq 68$ (gelb) sowie NPS-Schwellen für Passive (7) und Promoter (9).	53
13	UEQ-S Item-Profil nach Gruppe ($N = 16$, Fehlerbalken: ± 1 SEM). Gruppe A: M Gesamt=5,2. Gruppe B: M Gesamt=5,9. Größte Differenzen bei den Items <i>interessant</i> (+1,3) und <i>neuartig</i> (+1,2).....	54

14	Bewertung der KI-spezifischen Items in Gruppe B ($n = 8$, Skala: 1–7). Die gestrichelte graue Linie markiert den Neutralpunkt bei 4,0.	55
15	Zusammenhang zwischen Programmiererfahrung und Drill-Performance ($N = 16$). Links: First-Try-Rate ($r = 0,33$, $p = 0,215$, n.s.). Rechts: Durchschnittliche Versuche ($r = -0,63$, $p = 0,010$).	56
16	Zusammenhang zwischen Programmiererfahrung und kognitiver Belastung ($N = 16$). Links: Frustration ($r = -0,68$, $p = 0,004$). Rechts: Mentaler Aufwand ($r = -0,73$, $p = 0,001$).	56
17	Zusammenhang zwischen Programmiererfahrung und Kursleistung ($N = 16$). Links: Bearbeitungszeit ($r = -0,46$, $p = 0,071$, n.s.). Rechts: Gesamtfehler ($r = -0,50$, $p = 0,051$, n.s.).	57
18	Dashboard mit Kursübersicht. Abgeschlossene Schritte werden grün hervorgehoben; der Zinsrechner ist als optionaler Aufbaukurs gesperrt, bis der BMI-Rechner abgeschlossen wurde.	84
19	Kursansicht (Schritt 1 von 12): Links der Monaco-Code-Editor mit Startercode und Python-Ausgabebereich, rechts die Aufgabenstellung mit farbigen Hinweisboxen. Der blaue Chat-Button (roter Pfeil) öffnet den KI-Tutor (nur Gruppe B).	84
20	KI-Tutor-Chat (Gruppe B): Das progressive Hint-System antwortet auf eine Frage zur <code>print()</code> -Funktion mit Rückfragen statt direkter Lösung.	85
21	Part-task Practice – Multiple-Choice-Aufgabe (Schritt 1): Konzeptabfrage zur <code>print()</code> -Funktion aus dem statischen Drill-Pool.	86
22	Part-task Practice – Code-Aufgabe (Schritt 1): Die Lernende hat <code>Willkommen</code> ohne Anführungszeichen eingegeben; das System zeigt den Python-Fehler und meldet die Lösung als noch nicht korrekt.	86

Tabellenverzeichnis

1	Vergleich der Versuchsgruppen	24
2	Stichprobengrößen auf den einzelnen Filterebenen	38
3	Vergleich der vorliegenden Studie mit PyTutor (Yang et al., 2024) [32].	67
4	Protokoll der verwendeten Werkzeuge und KI-Systeme	87

Anhang

A: Fragebogen

A.1 Instrumente und Skalen

Der eingesetzte Fragebogen bestand aus fünf Blöcken. Die Implementierung findet sich in `evaluation/page.tsx`, die deutschen Wortlaute in `messages/de.json`.

1. **Vorerfahrung & Selbsteinschätzung:** Altersgruppe, Beschäftigungsstatus, IT/Software-Hintergrund, Programmierkenntnisse (Skala 1–5), Selbstwirksamkeit (3 Items, Likert 1–7)
2. **Cognitive Load:** Mentale Anstrengung (Paas-Skala, 1–9), Frustration (NASA-TLX-Teilskala, 0–100)
3. **Systemqualität:** SUS (10 Items, Likert 1–5), UEQ-S (8 Gegensatzpaare, 1–7)
4. **KI-Assistent (nur Gruppe B):** Nützlichkeit (3 Items), Vertrauen (3 Items, darunter ein invertiertes Item)
5. **Abschluss:** Net Promoter Score (0–10), offenes Freitext-Feedback (Pflichtfeld)

A.2 Beispielhafte Original-Items (Deutsch)

SUS-Beispiele:

- Ich denke, dass ich das System gerne häufig benutzen würde.
- Ich fand das System einfach zu benutzen.
- Ich fühlte mich bei der Benutzung des Systems sehr sicher.

UEQ-S Gegensatzpaare (Auszug):

- behindernd ↔ unterstützend
- kompliziert ↔ einfach
- langweilig ↔ spannend
- konventionell ↔ neuartig

Cognitive-Load-Items:

- Wie viel geistige Anstrengung mussten Sie aufbringen, um die Aufgaben zu lösen? (Paas, 1–9)
- Wie unsicher, entmutigt, irritiert oder gestresst fühlten Sie sich während der Aufgaben? (NASA-TLX Frustration, 0–100)

B: Ausgewählte Drill-Aufgaben (Beispiele)

Die folgenden Beispiele entstammen dem statischen Drill-Pool (`src/courses/pythonDrillTasks.ts`) und repräsentieren die zwei eingesetzten Aufgabentypen.

B.1 Multiple-Choice-Aufgabe: Konzeptabfrage

ID: `mcq-print-1`

Frage: Was macht die `print()`-Funktion?

Antwortoptionen:

- Speichert Daten auf der Festplatte
- **Gibt Text in der Konsole aus** (korrekt)
- Erstellt eine Variable
- Rechnet Zahlen aus

B.2 Multiple-Choice-Aufgabe: Output vorhersagen

ID: `mcq-print-3`

Aufgabe: Welche Ausgabe erzeugt der folgende Code?

```
print('Python')  
print('Lernen')
```

Korrekte Antwort: `Python` (Zeile 1), `Lernen` (Zeile 2)

B.3 Code-Drill: Schreiben

ID: `code-print-1`

Titel: Text ausgeben

Aufgabenstellung: Gib den Text `'Willkommen'` in der Konsole aus. Nutze `print()` direkt mit dem Text – du brauchst keine Variable.

Starter-Code:

```
# Schreibe hier deinen Code
```

Erwartete Ausgabe: `Willkommen`

C: Beispiel-Prompts für die KI-Generierung

Die folgenden Auszüge dokumentieren die Kernprinzipien der eingesetzten Systemprompts. Da die vollständigen Prompts mehrere hundert Token umfassen, werden hier nur die strukturellen Prinzipien zusammengefasst. Die vollständigen Prompt-Texte sind im Quellcode des Projekts einsehbar (vgl. Anhang D, Repository-Link).

C.1 Systemprompt: KI-Tutor (Lernassistenz)

Datei: `src/actions/openaiChatActionV3.ts`

Kernprinzipien des Prompts:

- Rolle: hilfreicher KI-Assistent für das Python-Bootcamp
- Didaktisches Prinzip: keine direkte Komplettlösung, sondern schrittweise, progressive Hinweise
- Strikte Ausgabeformatierung: strukturiertes JSON mit den Typen `text`, `admonition` und `codeblock`
- Kontextinjektion: aktuelle Aufgabe, Startercode, Konsolenausgabe, Hint-Level und Benutzerprofilierung werden je Anfrage als Nutzerkontext übergeben
- Hint-Eskalation: mit jeder Anfrage wird eine Stufe weiter eskaliert; Solution-Strings werden erst ab Level 5 mitgesendet

C.2 Systemprompt: KI-gestützte Drill-Auswahl (Pool-basiert)

Datei: `src/actions/aiDrillSelectionFromPoolAction.ts`

Kernprinzipien des Prompts:

- Auswahl einer didaktisch passenden Kombination aus einer MCQ- und einer Code-Aufgabe aus dem statischen Pool
- Performance-adaptive Schwierigkeit: Fehlerhistorie und Erfolgsrate der lernenden Person werden übergeben
- Priorisierung noch nicht gezeigter Aufgaben (`timesShown = 0`)
- Keine Wiederholung bereits korrekt gelöster Drills
- Strukturierte Ausgabe über striktes JSON-Schema (*Structured Outputs*, `strict: true`)

D: Screenshots der Web-Anwendung und Verfügbarkeit der Daten

Der vollständige Quellcode des Lernsystems sowie alle anonymisierten Studiendaten (Chat-Verläufe, Aufgaben-Log-Daten, Fragebogenantworten) sind öffentlich zugänglich unter:

<https://github.com/Shaco74/4CID-Adaptive-Python-Tutor>

Die folgenden Abbildungen zeigen die wichtigsten Ansichten der implementierten Lernplattform.

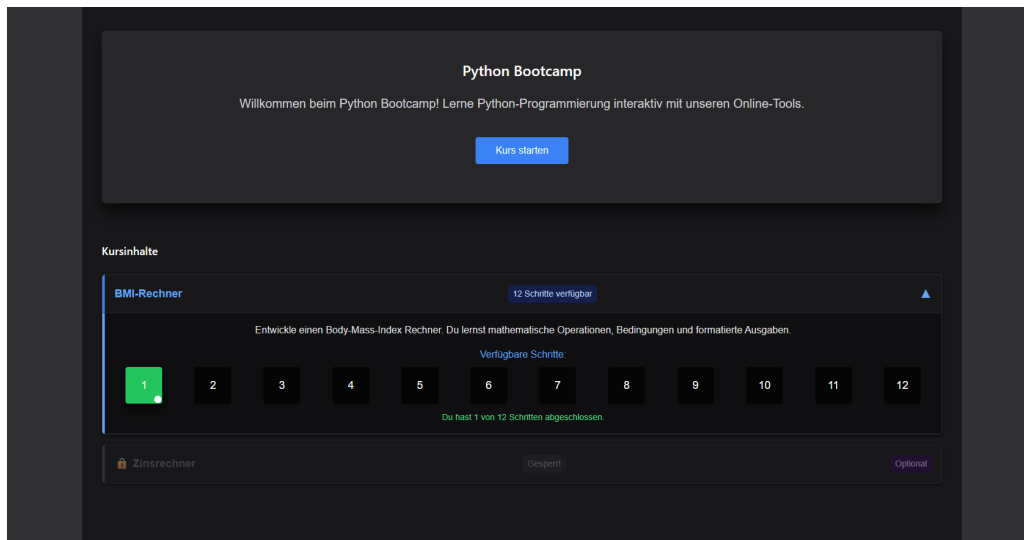


Abbildung 18: *Dashboard mit Kursübersicht. Abgeschlossene Schritte werden grün hervorgehoben; der Zinsrechner ist als optionaler Aufbaukurs gesperrt, bis der BMI-Rechner abgeschlossen wurde.*

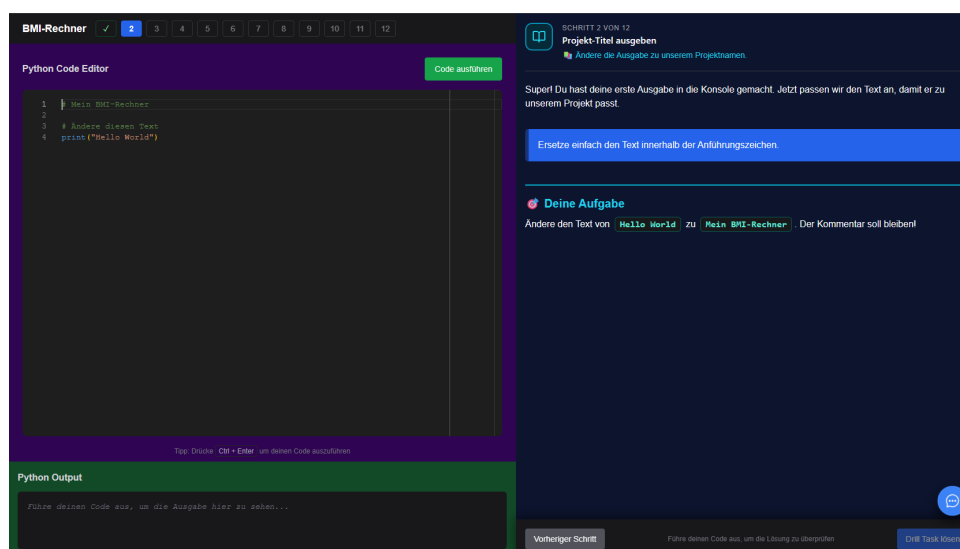


Abbildung 19: *Kursansicht (Schritt 1 von 12): Links der Monaco-Code-Editor mit Startercode und Python-Ausgabebereich, rechts die Aufgabenstellung mit farbigen Hinweisboxen. Der blaue Chat-Button (roter Pfeil) öffnet den KI-Tutor (nur Gruppe B).*

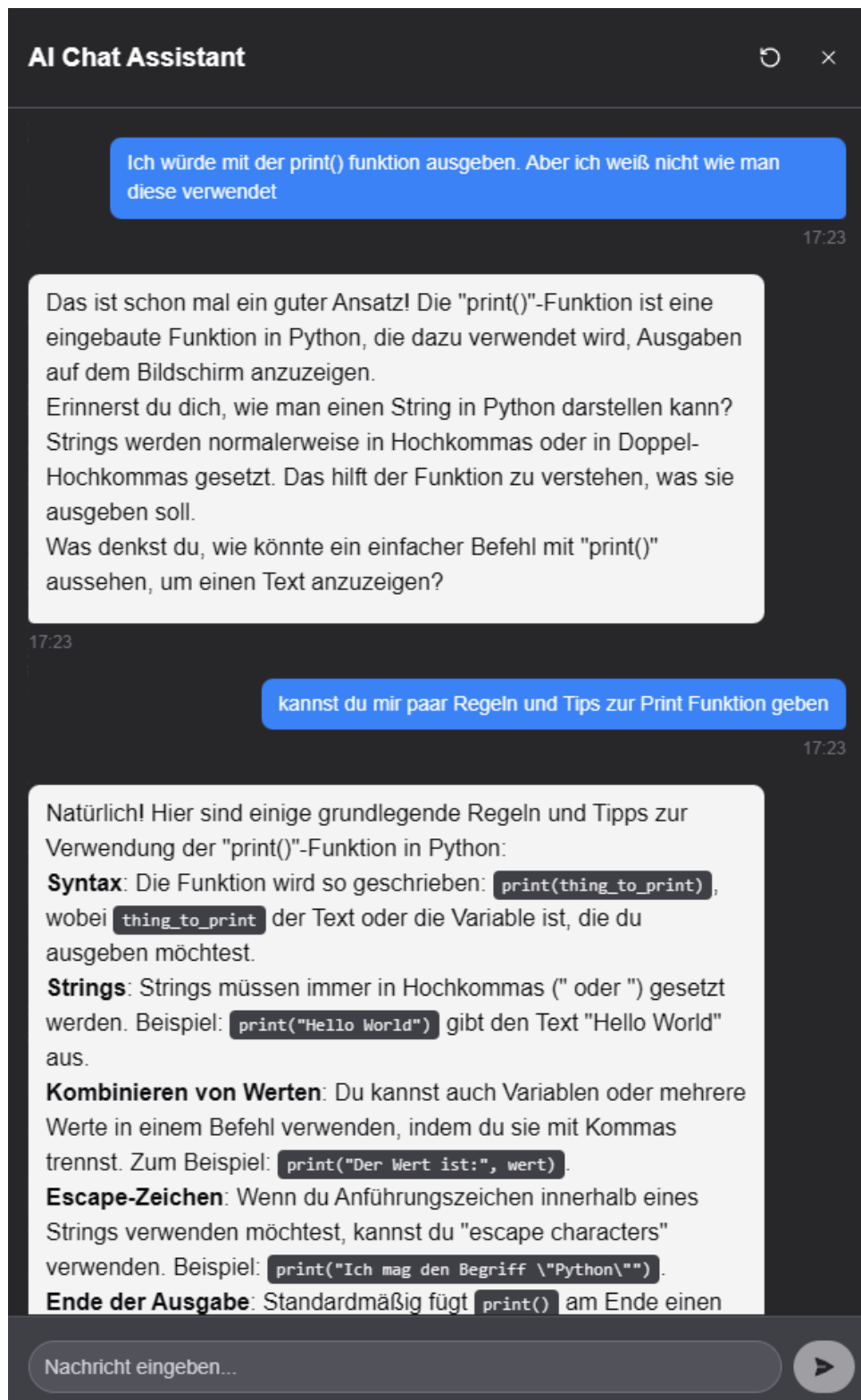


Abbildung 20: KI-Tutor-Chat (Gruppe B): Das progressive Hint-System antwortet auf eine Frage zur `print()`-Funktion mit Rückfragen statt direkter Lösung.

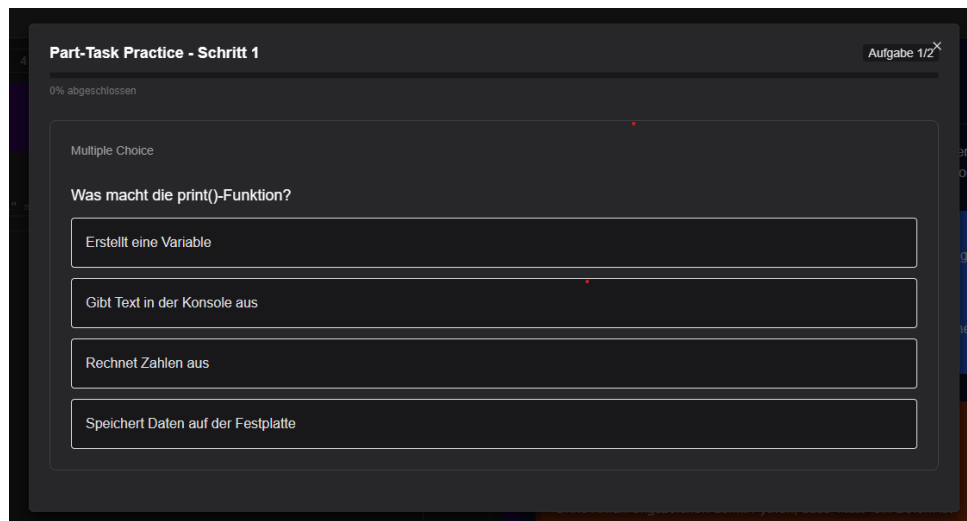


Abbildung 21: Part-task Practice – Multiple-Choice-Aufgabe (Schritt 1): Konzeptabfrage zur `print()`-Funktion aus dem statischen Drill-Pool.

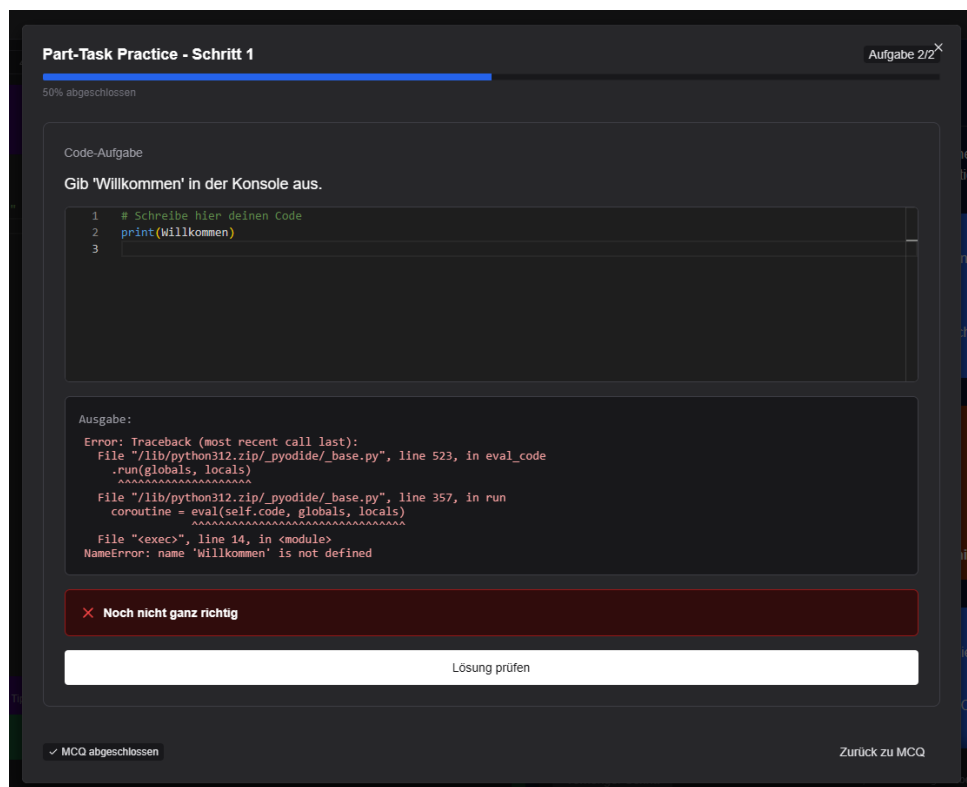


Abbildung 22: Part-task Practice – Code-Aufgabe (Schritt 1): Die Lernende hat `Willkommen` ohne Anführungszeichen eingegeben; das System zeigt den Python-Fehler und meldet die Lösung als noch nicht korrekt.

E: Werkzeug- und KI-Protokoll

Nachfolgend sind alle wesentlichen digitalen Werkzeuge und KI-Systeme aufgeführt, die bei der Erstellung dieser Arbeit zum Einsatz kamen. Die Auflistung folgt der Empfehlung des Betreuers, alle verwendeten Hilfsmittel transparent auszuweisen.

Tabelle 4: *Protokoll der verwendeten Werkzeuge und KI-Systeme*

Werkzeug / System	Version / Zeitraum	Einsatzbereich	Validierung
GitHub Copilot (Microsoft) mit GPT-4o, GPT-5.1 sowie Claude Sonnet 4.6 & Claude Opus 4.6 (Anthropic)	Copilot in VS Code, Juni 2025–Mrz. 2026	Formulierungsvorschläge, Gliederungsstruktur, $\LaTeX$ -Syntax, Erarbeitung von Kapitelgerüsten und Leitfragen, Code-Generierung und -Analyse, BibTeX-Bereinigung; KI-Unterstützung bei der Entwicklung der Lernplattform-Codebase	Alle Ausgaben manuell geprüft und inhaltlich angepasst; Quellen und DOIs separat verifiziert
Visual Studio Code [41] mit LaTeX Workshop Extension	VS Code 1.x, Feb.–Mrz. 2026	Entwicklungsumgebung für $\LaTeX$ -Ausarbeitung und Programmierprototyp; Build, Formatierung, PDF-Vorschau	Kein KI-Einsatz; reines Editierwerkzeug
latexmk / $\TeX$ Live (WSL)	$\TeX$ Live 2024, Feb.–Mrz. 2026	Kompilierung der $\LaTeX$ -Arbeit zu PDF	Kein KI-Einsatz
OpenAI API (gpt-4o-mini) [25]	API, Jan.–Mrz. 2026	KI-Tutor und Aufgabenauswahl im entwickelten Lernprototyp (Forschungsgegenstand der Arbeit)	Prototyp-intern; Ausgaben im Rahmen der Studie erhoben und ausgewertet

Anmerkung: Triviale Werkzeuge (Browser, Dateiverwaltung) sind gemäß den Hinweisen des Betreuers nicht gesondert aufgeführt. Die inhaltliche Verantwortung für alle Aussagen, Interpretationen und Schlussfolgerungen liegt beim Autor.